

COGANT: Codebase-to-GNN Translation

Theory of the Program Graph IR and Practice of the Python Pipeline

Daniel Ari Friedman

Active Inference Institute

`daniel@activeinference.institute`

ORCID: 0000-0001-6232-9096

2026-04-10

2026-04-10

Contents

1	Abstract	6
2	Introduction and Scope	7
2.1	Terminology: “GNN” in this manuscript	7
2.2	Problem	7
2.3	What COGANT is not	7
2.4	Positioning	8
2.5	Manuscript versus package docs	8
2.6	Documentation map (package hubs)	8
2.7	Dual Python/Rust architecture	9
2.8	Roadmap of the following sections	9
3	Program graph and formal foundations	10
4	Methodology: Program Graphs and Intermediate Representations	10
4.1	Program graph	10
4.1.1	Structural equivalence	10
4.2	Formal definitions	10
4.2.1	Theorems	12
5	IR progression, translation engine, and algorithms	14
5.1	Progressive IRs	14
5.2	Translation rules	14
5.2.1	Fixpoint iteration, conflict resolution, and coverage	14
6	Confidence scoring, evidence tiers, and state-space compilation	16
6.1	Confidence Scoring and Evidence Tiers	16

6.2	State space and behavior	16
6.2.1	Worked example: temperature controller	18
7	GNN export and error-handling philosophy	20
7.1	GNN export (Generalized Notation Notation)	20
7.2	Error handling philosophy	20
8	API and Workflows	21
8.1	Session-oriented workflow	21
8.2	Pipeline-oriented workflow	21
8.3	Bundle accessors	21
8.4	Command-line interface	22
8.5	Review API	22
9	Examples and Failure Modes	23
9.1	Example outputs	23
9.1.1	Concrete walkthrough: Flask REST API	23
9.1.2	Example semantic mapping output	24
9.1.3	Example state-space excerpt	25
9.1.4	Dynamically enriched excerpt	26
9.1.5	Failure modes and graceful degradation	27
9.2	Rust layer	29
10	Conclusion	30
10.1	Shipped Capabilities	30
10.2	Roadmap and Future Extensions	32
11	Experimental setup: environment, API, and configuration	34
12	Experimental setup	34
12.1	Environment	34
12.2	Running the API	34
12.3	Configuration files	34
12.4	CLI	35
13	Exports, parser capabilities, and progressive IR stages	36
13.1	Export targets	36
13.2	Python AST parser capabilities	36
13.3	Progressive IR stages	36
14	Performance targets and measured runs on packaged fixtures	39
14.1	Performance characteristics	39
14.2	Measured runs on packaged fixtures	39
15	Test matrix, mutation testing, and benchmark suite	43
15.1	Test matrix and coverage	43
15.2	Mutation testing	44
15.3	Benchmark suite (shipped)	45

16 What to record for reproducible experiments	47
16.1 Minimum recording checklist	47
16.2 How to re-run the canonical fixture experiments	48
16.3 What the manifest.json index records	48
16.4 Worked example: regenerating the Table 4 row for <code>flask_app</code>	48
16.5 Data ethics and licensing for exported bundles	49
16.6 Cross-references	49
17 Experimental setup	50
17.1 Environment	50
17.2 Running the API	50
17.3 Configuration files	50
17.4 CLI	51
17.5 Export targets	51
17.6 Python AST parser capabilities	51
17.7 Progressive IR stages	52
17.8 Performance characteristics	53
17.9 Measured runs on packaged fixtures	54
17.10 Test matrix and coverage	57
17.11 Mutation testing	58
17.12 Benchmark suite (shipped)	59
17.13 What to record	60
18 Reproducibility	61
18.1 Version pinning	61
18.2 Artifact layout	61
18.3 Determinism	61
18.4 Relation to the template repository	61
18.5 Validation gates	61
18.6 Current runner behavior vs template checkpointing	63
18.7 Data ethics and licensing	63
19 Scope and related work: landscape and tool categories	64
19.1 COGANT in the program-analysis landscape	64
19.2 Related tool categories	64
20 Program analysis for machine learning and comparison tables	65
20.1 Program analysis for ML	65
20.1.1 Feature matrix: COGANT vs. related tools	66
20.1.2 Input/output comparison vs prior art	67
21 Bidirectional lenses, synthesis, and categorical framing	70
21.1 Bidirectional transformations and the lens view of COGANT	70
22 World models, active inference, boundaries, and forward compatibility	72
22.1 World models from code	72
22.2 Active inference and program behavior	72

22.3	Boundaries	72
22.4	Forward compatibility	72
23	Scope and related work	74
23.1	COGANT in the program-analysis landscape	74
23.2	Related tool categories	74
23.3	Where the full comparison lives	74
24	Ablation study	75
24.1	Rule-family ablation	75
24.2	Fixpoint-iteration ablation	82
24.3	Matrix-fallback ablation	84
24.4	Summary	85
25	Appendix A — Full Roundtrip Table (per-role breakdown)	86
25.0.1	A.1 All 23 targets, post wave 14 (canonical)	86
25.0.2	A.2 Pre-fix vs post-fix for affected repositories (wave 14 CONSTRAINT fix)	87
25.0.3	A.3 POMDP shape match across all 23 targets	88
26	Appendix B — Full Ablation Table	89
26.0.1	B.1 Rule-family ablation on zoo/01_simple_state	89
26.0.2	B.2 Cross-reference with Section 9 rule-family ablation	91
27	Appendix C — Galois Connection Proof Sketch	93
27.0.1	C.1 Categories	93
27.0.2	C.2 Forward and reverse functors	93
27.0.3	C.3 Role multiset functor	93
27.0.4	C.4 Adjunction (approximate)	94
27.0.5	C.5 -isomorphism theorem	94
27.0.6	C.6 ISOMORPHIC threshold corresponds to majority role preservation	95
28	Appendix D — Inference Loop Mathematics	96
28.0.1	D.1 POMDP formulation	96
28.0.2	D.2 Variational free energy functional	96
28.0.3	D.3 Variational inference via belief propagation	97
28.0.4	D.4 VFE = 0.0 in the identity model	97
28.0.5	D.5 Other regimes observed in the empirical runs	98
28.0.6	D.6 Multi-episode D update rule and convergence	98
28.0.7	D.7 Expected free energy and policy selection	99
29	Appendix E — Extended Related Work	100
29.0.1	E.1 Program analysis $\rightarrow$ GNN (learned and symbolic)	100
29.0.2	E.2 Active inference tooling and implementations	101
29.0.3	E.3 Code understanding and learned code models	102
29.0.4	E.4 Graph kernels and structural similarity for code	104
29.0.5	E.5 Formal methods: abstract interpretation and Galois connections in static analysis	104

29.0.6	E.6 POMDP solvers and planning	105
29.0.7	E.7 Program synthesis and reverse engineering	106
29.0.8	E.8 Bidirectional transformations, lenses, and the categorical frame	107
29.0.9	E.9 Markov blankets and active inference foundations	108
30	References back to COGANT source material	110
31	Manuscript Syntax Reference (COGANT)	111
31.1	Citations	111
31.2	Equations	111
31.3	Figures	111
31.4	Section files	111
31.5	Cross-references to the package	111
31.6	See also	111
32	COGANT — Supplementary Materials	112
32.1	Supplemental files	112

1 Abstract

COGANT (Codebase-to-GNN Translation) converts software repositories into structured Active Inference artifacts expressed in the Active Inference Institute’s **Generalized Notation Notation** (GNN)¹. It sits between classical program analysis, which already reasons over graphs such as call graphs and code property graphs [Yamaguchi et al., 2014], and data-driven methods that expect tensors, consistent node and edge vocabularies, and exportable training bundles [Allamanis et al., 2018a, Wu et al., 2021, Scarselli et al., 2009].

The design centres on a **program graph IR** whose nodes and edges carry **confidence** and **provenance**. A **fixpoint translation engine** applies **19 declarative rules** (five structural, five semantic, two control, three behavioural, four resilience) that map nodes onto **7 Active Inference semantic roles** — `HIDDEN_STATE`, `OBSERVATION`, `ACTION`, `POLICY`, `PREFERENCE`, `CONSTRAINT`, and `CONTEXT` — with conflict resolution by priority and score. Each bundle includes a **Markov blanket partition** in $O(V+E)$ time. A **reverse synthesizer** (`cogant.reverse`) reconstructs a runnable Python package from an emitted GNN bundle. After the v0.5.0 `POLICY/CONTEXT` stub-emission fix (`../cogant/docs/evaluation/ROUNDTRIP_IMPROVEMENT.md`), the forward–reverse–forward round-trip on the canonical evaluation set reports **14** isomorphic ($\varepsilon \geq 0.8$), **6** approximate ($\varepsilon \geq 0.5$), and **3** divergent targets (of **23** total; twelve zoo fixtures, three real-world-example fixtures, eight third-party libraries in scope); details and tables are in `../cogant/docs/evaluation/ROUNDTRIP_EVAL.md` and §9.

The v0.5.0 **Python** pipeline uses the standard-library `ast` module. **JavaScript** and **TypeScript** are **optional**: install `cogant[multilang]` and an available `tree-sitter` grammar (including the JS-grammar fallback path for `.ts` on mixed repositories); integration tests cover this path when those pieces are installed. HTTP service, container image, incremental analysis, and multi-episode agent runtime are documented under `../cogant/docs/index.md` and the package `README.md` rather than repeated here. The test suite reports **2129** passing tests, **86** skips, **2** expected xfail, and **1** xpass; `py/cogant/` line coverage is **83.42%** (canonical run aligned to `METRICS.yaml`, generated **2026-04-10T21:03:18.504378Z**) under `pytest --cov`. Interpreters: Python 3.11–3.13. This manuscript gives the theory of the IR, running practice, and extension points (parsers, rules, validators, exporters) alongside that documentation.

¹Throughout this manuscript **GNN** refers to the Active Inference Institute’s **Generalized Notation Notation** (**repository**), a structured notation for state-space and process models — *not* graph neural networks. Downstream consumers, including graph neural network training pipelines, can ingest the emitted artifacts, but COGANT itself produces Generalized Notation Notation bundles.

2 Introduction and Scope

Machine learning on source code has matured alongside static analysis: both need structured representations of programs, but learning pipelines additionally need fixed feature layouts, batchable graphs, and reproducible exports [Allamanis et al., 2018a]. COGANT addresses that gap by making the path from repository checkout to **Generalized Notation Notation (GNN) bundles** explicit, inspectable, and configurable.

2.1 Terminology: “GNN” in this manuscript

In this manuscript, **GNN** refers to the Active Inference Institute’s **Generalized Notation Notation (repository)** [Active Inference Institute, 2024], a structured notation for state-space and process models—not graph neural networks. COGANT translates source code into this notation, producing program-graph and state-space artifacts (state space, observation modalities, actions/policies, likelihood structure, preferences, time settings, parameterization) that downstream tools can consume. Those downstream tools include graph neural network training pipelines, which can ingest the exported tensor views of the program graph, but such neural networks are a *consumer* of COGANT’s output, not its output format. Where the manuscript discusses graph neural networks as related work, it uses the phrase “graph neural networks” in full; the acronym GNN always refers to Generalized Notation Notation.

2.2 Problem

A research or engineering group typically has:

- **Source code** in one or more languages.
- **Analysis goals** that combine classical queries (who calls whom?) with learned models (bug detection, retrieval, summarization).
- **Tooling fragmentation**: compilers and linters produce one shape of facts; PyTorch or JAX expect another.

COGANT unifies these behind a single pipeline whose intermediate artifacts are documented as a progression of IRs (repo IR, program graph, semantic mapping, state space, process model, validation), as described in `../cogant/docs/architecture/README.md` and `../cogant/docs/reference/implementation_status.md`.

2.3 What COGANT is not

COGANT is not a replacement for a full compiler IR, a theorem prover, or a security scanner by itself. It **extracts and serializes** program graphs and behavioral sketches for downstream use. Language coverage and rule depth follow `../cogant/docs/reference/implementation_status.md`: Python is the primary front end at v0.5.x. JavaScript/TypeScript run through optional `cogant[multilang]` plus `tree-sitter` grammars (including a JS-grammar fallback path for `.ts` on mixed repositories) when installed. Rust acceleration is partially wired — `cogant._rust` exposes a PyO3 `connected_components` FFI behind the `COGANT_USE_RUST` feature flag, with a pure-Python fallback for all other code paths as documented there.

2.4 Positioning

COGANT occupies a specific niche in the program-analysis and code-modelling landscape that is worth making explicit from the outset. Unlike learned code-embedding approaches such as code2vec and code2seq [Alon et al., 2019], which map programs to opaque distributed representations via AST path attention, COGANT produces **interpretable rule-based role assignments** that a human reviewer can inspect, edit, or reject through the ReviewAPI and that require no training corpus, no GPU budget, and no curated supervision signal. Every mapping carries a provenance record and a confidence score derived from a transparent rule (Equation 10), so the output can be audited in the same way a conventional static analyser’s findings are audited [Yamaguchi et al., 2014]. While graph-learning methods such as gated graph neural networks for programs [Allamanis et al., 2018b, Li et al., 2016] train supervised models to predict variable names or detect bugs from raw graph topology, COGANT complements that line by supplying a typed, confidence-annotated program graph in a documented export contract that such models can consume directly (Section 2 and `../cogant/docs/export/README.md`).

On the Active Inference side, Friston et al.’s free energy principle [Friston, 2010] and the discrete-state synthesis of [Da Costa et al., 2020] provide the theoretical foundation for the A/B/C/D matrix formalism, and the PyMDP runtime [Heins et al., 2022] implements the simulation side of that formalism. COGANT sits between those layers: it extracts the same matrices directly from an existing codebase — rather than asking a human to hand-author them, as [Smith et al., 2022] walks through for small examples — and emits them in the Active Inference Institute’s Generalized Notation Notation [Active Inference Institute, 2024] so that PyMDP and compatible runtimes can execute them without manual translation. No other shipped tool combines all three properties: declarative rule-based role assignment, typed program-graph extraction with dynamic enrichment, and direct GNN/A/B/C/D export compatible with an active-inference runtime. Section 8 walks through the feature matrix that makes this positioning concrete.

2.5 Manuscript versus package docs

The canonical API and CLI details live in:

- `../cogant/docs/api/README.md` — Session, PipelineRunner, Bundle, ReviewAPI
- `../cogant/docs/cli/README.md` — commands and flags
- `../cogant/docs/export/README.md` — Generalized Notation Notation schema, section ordering, and companion interop field contracts (including optional PyG/DGL tensor views)
- `../cogant/docs/plugins/README.md` — parsers, rules, validators, exporters
- `../cogant/docs/architecture/README.md` — layers, crates, data flow

This manuscript summarizes theory and practice for readers who want a single linear narrative before diving into those files.

2.6 Documentation map (package hubs)

The modular docs tree under `../cogant/docs/` is the authoritative reference. The MkDocs entry page is `../cogant/docs/index.md`; the full module index is `../cogant/docs/reference/documentation_modules.md`. Beyond the API, CLI, export, plugins, and architecture indexes above, editors and agents should route through:

- [../cogant/docs/reference/README.md](#) — implementation status, pipeline narratives, onboarding-style how-tos
- [../cogant/docs/rules/README.md](#) — translation rule framework and mapping definitions
- [../cogant/docs/validation/README.md](#) — integrity, schema, and provenance checks
- [../cogant/docs/security/README.md](#) — threat model and sandboxing notes
- [../cogant/docs/roadmap/README.md](#) — releases and backlog
- [../cogant/specs/README.md](#) — machine-readable schemas and IR contracts

2.7 Dual Python/Rust architecture

COGANT employs a dual-language architecture. The **Python orchestration layer** handles session management, pipeline coordination, configuration, file discovery, AST parsing, and the plugin interface – tasks where developer ergonomics and extensibility matter more than raw throughput. The **Rust core**, organized as a workspace of eight crates (`cogant-core`, `cogant-graph`, `cogant-translate`, `cogant-statespace`, `cogant-store`, `cogant-trace`, `cogant-gnn`, `cogant-ffi`), implements typed graph operations, translation internals, trace-oriented plumbing, state-space structures, and Generalized Notation Notation (GNN) formatting – tasks where memory layout and cache locality dominate wall-clock time. Communication between layers flows through PyO3 bindings (`cogant-ffi`), which release the Python GIL during Rust execution to permit concurrent stage processing. Where Rust bindings are not yet wired for a code path (see the implementation-status table in [../cogant/docs/reference/implementation_status.md](#)), Python fallback implementations ensure the pipeline remains functional at the cost of higher latency on large repositories.

2.8 Roadmap of the following sections

Section 2 formalizes the program graph and IR progression, and states the five formal definitions and three termination/completeness/validity theorems that govern the rest of the manuscript. Section 3 describes the realized pipeline behavior and artifacts. Section 4 provides concrete examples and failure modes. Section 5 closes with limitations and roadmap. Section 6 covers experimental setup, Section 7 reproducibility, Section 8 related work and the COGANT-vs-prior-art feature and input/output comparison tables, and Section 9 an ablation study over rule families, fixpoint iterations, and A/B/C/D fallback paths.

3 Program graph and formal foundations

4 Methodology: Program Graphs and Intermediate Representations

4.1 Program graph

Let $G = (V, E)$ denote a directed graph whose vertices V represent program entities and whose edges E represent relationships. COGANT assigns each node a **kind** (for example function, variable, type) and a **semantic role** (for example definition versus use). Each node and edge may carry:

- A **stable identifier** for persistence across runs when hashing allows.
- **Type strings** and attributes recovered from the front end.
- A **confidence** score in $[0, 1]$ and **provenance** metadata (source code, type system, control flow, heuristic, external tool).

This follows the same philosophical line as code property graphs that fuse AST, control flow, and data flow into one analyzable structure [Yamaguchi et al., 2014], but orients the representation toward **Generalized Notation Notation (GNN) export** and, optionally, tensorized views of the same graph: node kinds and roles map to discrete indices in both forms, and optional embeddings can augment features as described in `../cogant/docs/export/README.md`.

4.1.1 Structural equivalence

Two program graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are usefully compared via typed graph isomorphism: a bijection $\phi : V_1 \rightarrow V_2$ preserving edge structure and relevant labels supports deduplication and cross-repository linking when interfaces align.

$$(u, v) \in E_1 \iff (\phi(u), \phi(v)) \in E_2 \quad (1)$$

Equation 1 formalizes the structural invariant we check during deduplication and cross-repository linking: a candidate bijection ϕ is accepted only when every edge in G_1 has a corresponding edge between the images of its endpoints in G_2 (and vice versa), so that label-preserving matches strictly preserve the adjacency structure of both program graphs.

4.2 Formal definitions

This subsection makes the objects manipulated by the pipeline mathematically explicit. The definitions are stated for the shipped v0.5.x engine; where the implementation differs from the general form (for example because an edge kind is not yet emitted by the Python front end), the difference is noted with a forward reference to Section 6.

Definition 1 (Program graph). A **program graph** is a tuple $G = (V, E, \lambda_V, \lambda_E, \tau)$ where

- V is a finite set of program nodes (modules, classes, methods, functions, and — on languages where the front end emits them — variables and control-flow sites);
- $E \subseteq V \times V \times K$ is a finite set of typed directed edges drawn from the edge-kind alphabet $K \supseteq \{\text{READS, WRITES, CALLS, CONTAINS, INHERITS, IMPORTS, OBSERVES, MUTATES, DEPENDS_ON}\}$;

- $\lambda_V : V \rightarrow \mathcal{N}$ labels each node with a node kind $\mathcal{N} \supseteq \{\text{MODULE, CLASS, METHOD, FUNCTION, CONFIGURATION}\}$
- $\lambda_E : E \rightarrow K$ is the (trivial) projection onto the edge kind;
- $\tau : V \rightarrow (T \cup \{\perp\})$ maps each node to a type annotation recovered from the front end or to $\perp$ when no annotation is available.

The shipped Python front end populates the kinds $\{\text{MODULE, CLASS, METHOD, FUNCTION}\}$ and the edge kinds $\{\text{CALLS, CONTAINS, READS, WRITES, IMPORTS, INHERITS}\}$; the remaining kinds in $\mathcal{N}$ and K are declared in `cogant.schemas.core` but currently emitted only when other parsers or dynamic enrichment stages fire. The empirical distribution on the six packaged fixtures is recorded in Tables 4 and 5 of Section 6.

Definition 2 (Translation rule). A **translation rule** is a quadruple $r = (\varphi_r, \kappa_r, w_r, p_r)$ where

- $\varphi_r : \mathcal{G} \rightarrow 2^{\mathcal{F}}$ is a computable predicate that, given a program graph $G \in \mathcal{G}$, returns a (possibly empty) set of matched fragments $\mathcal{F}$ — each fragment a finite tuple of node ids and optional edge ids;
- $\kappa_r \in \mathcal{K}_M$ is the mapping kind the rule assigns on success, drawn from the mapping-kind alphabet $\mathcal{K}_M = \{\text{HIDDEN_STATE, OBSERVATION, ACTION, POLICY, CONSTRAINT, PREFERENCE, CONTEXT}\}$;
- $w_r \in (0, 1]$ is the base confidence weight attached to mappings produced by r ;
- $p_r \in \mathbb{Z}$ is the rule priority consulted during conflict resolution (Algorithm 2).

Concretely, each `TranslationRule` subclass in `../cogant/py/cogant/translate/rules/` exposes φ_r as the `matches(graph, query)` method, encodes κ_r in the `mapping_kind` property, embeds w_r in the `confidence_score` field of the returned `SemanticMapping`, and exposes p_r through the `priority` property. The nineteen shipped rules span five families: five structural rules (`ReadOnlyInput`, `MutatingSubsystem`, `Inheritance`, `Containment`, `DataPipeline`), five semantic rules (`Observation`, `Action`, `Policy`, `Preference`, `Context`), two control rules (`Config`, `FeatureFlag`), three behavioural rules (`Orchestrator`, `TestAssertion`, `EventBus`), and four resilience rules (`RetryPattern`, `ErrorBoundary`, `SingletonAccess`, `CircuitBreaker`).

Definition 3 (Fixpoint semantics). Let $\mathcal{M}$ be the (finite) set of all possible semantic mappings that any finite composition of rules in R could emit on a fixed graph G . Define the rule-application operator $F_{G,R} : 2^{\mathcal{M}} \rightarrow 2^{\mathcal{M}}$ by

$$F_{G,R}(S) = S \cup \{r.\text{apply}(G, f) \mid r \in R, f \in \varphi_r(G), r.\text{apply}(G, f) \neq \perp\}. \quad (2)$$

The **translation** of G under R is the least fixpoint

$$T^*(G) = \bigsqcup_{k \geq 0} F_{G,R}^k(\emptyset) = \lim_{k \rightarrow \infty} F_{G,R}^k(\emptyset), \quad (3)$$

computed iteratively by `TranslationEngine.translate()` and then post-processed by `_resolve_conflicts()` (Algorithm 2). The post-processing step applies an anti-monotone pruning to the fixpoint, and is therefore specified outside of $F_{G,R}$ rather than folded into it.

Definition 4 (Markov blanket partition). Given a program graph $G = (V, E, \lambda_V, \lambda_E, \tau)$ and a **seed set** $S \subseteq V$ selected by one of the five strategies in `MarkovBlanketExtractor` (`explicit`, `module`, `kind`, `auto`, `mapping_kind`), the **Markov blanket partition** $\Pi_{G,S} : V \rightarrow \{\mu, s, a, \eta\}$ is defined on the undirected projection of G as follows. Let $N^{\text{in}}(v) = \{u : (u, v, k) \in E\}$ and

$N^{\text{out}}(v) = \{u : (v, u, k) \in E\}$. Then

$$\Pi_{G,S}(v) = \begin{cases} \mu & \text{if } v \in S \text{ and } (N^{\text{in}}(v) \cup N^{\text{out}}(v)) \subseteq S, \\ a & \text{if } v \in S \text{ and } N^{\text{out}}(v) \setminus S \neq \emptyset, \\ s & \text{if } v \in S \text{ and } N^{\text{out}}(v) \subseteq S \text{ and } N^{\text{in}}(v) \setminus S \neq \emptyset, \\ \eta & \text{if } v \notin S. \end{cases} \quad (4)$$

The four-way case split is realised verbatim by `partition_by_seeds()` in `../cogant/py/cogant/markov/blanket.py`: the function precomputes the in/out adjacency of every node in $O(|V| + |E|)$ time via `_bidirectional_adjacency()`, then walks V once and assigns each node to exactly one of μ, s, a, η . *Bidirectional boundary nodes (those with both external in – and out – neighbours) are assigned to a by convention, and are additionally tagged with a `bidirectional` metadata flag so that downstream consumers can recover the distinction.*

Definition 5 (A/B/C/D matrices). Given a translation $T^*(G)$ and a compiled state-space $(\mathbf{V}, \mathbf{O}, \mathbf{A})$ of hidden-state variables, observation modalities, and actions, the **generative-model matrices** of COGANT are

$$A \in \mathbb{R}^{|\mathbf{O}| \times |\mathbf{V}|}, \quad A_{ij} = P(o_i \mid s_j), \quad \sum_i A_{ij} = 1, \quad (5)$$

$$B \in \mathbb{R}^{|\mathbf{V}| \times |\mathbf{V}| \times |\mathbf{A}|}, \quad B_{ijk} = P(s'_i \mid s_j, a_k), \quad \sum_i B_{ijk} = 1, \quad (6)$$

$$C \in \mathbb{R}^{|\mathbf{O}|}, \quad C_i = \log \tilde{P}(o_i), \quad (7)$$

$$D \in \mathbb{R}^{|\mathbf{V}|}, \quad D_j = P(s_j \mid t = 0), \quad \sum_j D_j = 1. \quad (8)$$

A is derived from `{READS, OBSERVES, DEPENDS_ON}` edges between observation and hidden-state nodes; B is derived from `{WRITES, MUTATES}` edges from action to hidden-state nodes, with identity fallback when an action writes nothing; C is derived from the signed confidence scores of `CONSTRAINT/PREFERENCE` mappings adjacent to each observation; and D is derived from `CONFIGURATION`-neighbour bias on hidden-state variables or falls back to a uniform prior. The derivation is performed by `GNNMatrices` in `../cogant/py/cogant/gnn/matrices.py` and uses no numerical dependencies beyond the Python standard library. Rows (for A) and columns (for B) and the vectors D are normalised to valid probability distributions using the high-direct / low-indirect mass defaults (0.9, 0.1) imported from the upstream PyMDP placeholder convention; the C vector is a log-preference and is not normalised.

4.2.1 Theorems

Theorem 1 (Fixpoint termination). Let $G = (V, E, \lambda_V, \lambda_E, \tau)$ be a finite program graph with $|V| = n$, let R be a finite rule set with $|R| = k$, and let $F_{G,R}$ be the rule-application operator of Definition 3. Then the Kleene chain

$$\emptyset \subseteq F_{G,R}(\emptyset) \subseteq F_{G,R}^2(\emptyset) \subseteq \dots \quad (9)$$

stabilises in at most $|\mathcal{M}|$ iterations, where $|\mathcal{M}| \leq n \cdot |\mathcal{K}_M|$ is an upper bound on the number of distinct mapping ids any rule set can produce on G . In particular, the shipped engine with its default cap $K = 10$ converges on every fixture in `{calculator, event_pipeline, flask_mini, flask_app, requests_lib, json_stdlib}` within the cap.

Proof sketch. Each application step either adds at least one new mapping id to the accumulating set S or leaves S unchanged; the engine’s explicit `mapping.id not in self.mappings` guard (`engine.py` lines 299–303) is what enforces the monotone-plus-bounded invariant. Because $F_{G,R}$ is monotone on the finite lattice $(2^{\mathcal{M}}, \subseteq)$ and bounded above by $\mathcal{M}$, the Kleene chain is an ascending chain in a finite lattice and therefore stabilises in at most $|\mathcal{M}|$ steps. The stabilisation point is, by construction, the least fixpoint of $F_{G,R}$ above $\emptyset$. The worst-case bound $n \cdot |\mathcal{K}_M|$ is attained when every node receives at most one mapping of each kind, which is the maximum any rule set can inject before the conflict-resolution step prunes overlaps. Empirically every packaged fixture converges in a single pass because the shipped rules are disjoint on the node kinds they target: Tables 4 and 6 of Section 6 record `mappings_total` values that equal the sum of per-kind counts, which is only possible if no node was touched twice. ■

Theorem 2 (Markov blanket completeness). For any program graph $G = (V, E, \lambda_V, \lambda_E, \tau)$ with $V \neq \emptyset$ and any seed set $S \subseteq V$, the partition $\Pi_{G,S}$ of Definition 4 is **total** (every $v \in V$ receives exactly one role) and **mutually exclusive** (no v is assigned two roles).

Proof sketch. Totality: the case split in Equation 4 exhausts the boolean product $(v \in S) \times (N^{\text{out}}(v) \setminus S = \emptyset) \times (N^{\text{in}}(v) \setminus S = \emptyset)$. If $v \notin S$ the fourth case applies. If $v \in S$ there are three subcases depending on whether v has external out-neighbours, external in-neighbours only, or no external neighbours at all; the engine resolves the “both external in and external out” subcase to **ACTIVE** by convention, matching lines 205–211 of `blanket.py`. Every $v \in V$ therefore matches exactly one branch, so Π is a total function. Mutual exclusivity: the branches are pairwise disjoint because they condition on mutually exclusive predicates on $(v \in S, N^{\text{in}}(v) \setminus S, N^{\text{out}}(v) \setminus S)$. The implementation materialises the four role sets **internal**, **sensory**, **active**, **external** as disjoint Python sets and writes into exactly one on each iteration of the main loop, so the in-memory invariant matches the mathematical one. ■

Theorem 3 (Matrix validity). If $|\mathbf{O}| \geq 1$, $|\mathbf{V}| \geq 1$, and $|\mathbf{A}| \geq 1$, then the matrices (A, B, C, D) produced by `GNNMatrices.compute_A/B/C/D` satisfy the stochastic conditions of Definition 5 within a numerical tolerance of 10^{-6} .

Proof sketch. Rows of A are produced in Equation (5) with explicit normalisation (`_normalize_row()` at `matrices.py` line 277), which divides by the row sum when it exceeds $\varepsilon = 10^{-9}$ and returns a uniform row otherwise. Columns of each action slice $B[:, :, k]$ are produced by the same helper on lines 334–348, with an identity fallback ensuring the resulting column is never all-zero. The prior D is built as a weighted vector of confidence scores and passed through `_normalize_vector()`, again with a uniform fallback when all weights are below ε . The implementation therefore establishes the sum-to-one invariant by construction, and `validate_shapes()` (lines 554–603) enforces a tolerance of 10^{-6} on every computed matrix before the pipeline accepts the bundle; all six packaged fixtures pass `GNNValidator` with zero errors (Table 7), which is observationally the same statement as Theorem 3 on those fixtures. The C vector is a log-preference and is exempt from the sum-to-one requirement. ■

5 IR progression, translation engine, and algorithms

5.1 Progressive IRs

Processing proceeds through a sequence of representations. Which stages are complete for a given repository is summarized in `../cogant/docs/reference/implementation_status.md` (partial areas include translation rules, state space, and Rust acceleration):

1. **Repo IR** — entities and relationships extracted from parsers.
2. **Program graph IR** — consolidated graph with deduplication and metadata.
3. **Semantic mapping IR** — output of the translation rule engine.
4. **State space IR** — variables, actions, transitions, observations.
5. **Process model IR** — higher-level control patterns where implemented.
6. **Validation IR** — coverage, confidence analysis, schema checks.

COGANT’s program graph sits in the same conceptual space as compiler intermediate representations such as LLVM [Lattner and Adve, 2004] and MLIR [Lattner et al., 2021], but targets behavioral extraction and export for downstream learning rather than code generation or optimization.

5.2 Translation rules

The **translation** stage applies declarative rules that refine roles, attach labels, and adjust confidence. Concurrency targets and layering are described in `../cogant/docs/architecture/README.md`. The rule engine composes passes over the graph in a **fixpoint loop**: rules are re-applied until no new semantic mappings emerge or a configurable iteration cap is reached. The fixpoint loop follows the classical formulation of program analysis as fixpoint computation over a lattice of abstract states [Cousot and Cousot, 1977]; in our setting the lattice is the set of semantic mappings partially ordered by inclusion, and the monotone operator is the composition of all registered rule applications. In the shipped implementation, each pass applies rules in descending `rule.priority`, and conflict resolution later compares (`rule.priority`, `confidence_score`) tuples when two mappings overlap, following the principle that edit representations should be composable [Yin et al., 2019].

5.2.1 Fixpoint iteration, conflict resolution, and coverage

The shipped `TranslationEngine` in `../cogant/py/cogant/translate/engine.py` realizes the fixpoint loop concretely. Each iteration walks every registered rule once: the engine calls `rule.matches(graph, query)` to collect candidate fragments, then calls `rule.apply(graph, match)` on each, accumulating any resulting `SemanticMapping` objects keyed by their stable IDs. A per-pass counter tracks mappings that were genuinely new (not already present in the running set), and the loop terminates as soon as a pass completes with zero additions. The engine logs each iteration boundary through an internal match log, so the number of passes required to reach a fixed point is directly observable in the post-run diagnostics. The default iteration cap is `max_iterations = 10`; in testing, most repositories converge well before that bound, and the cap exists primarily as a safety valve against pathological rule sets that could otherwise oscillate indefinitely.

After fixpoint termination, the engine invokes `_resolve_conflicts()` to reconcile mappings whose `graph_fragment_node_ids` sets overlap. For each overlapping pair the engine retains

the mapping with the larger (`rule_priority`, `confidence_score`) key and discards the other, logging a `conflict_resolved` event that records the losing ID, the winning ID, and the specific overlap set. Most shipped rules use the default `rule.priority` of 0; the mutating-subsystem / hidden-state rule uses priority 1 so it survives overlaps with same-confidence class-level aggregates (for example containment summaries) on the same `CLASS` node. A companion entry point, `translate_with_confidence()`, runs the standard fixpoint loop, rescores every surviving mapping through the `ConfidenceModel`, and then re-resolves conflicts so that any ordering shifts induced by rescoring are honored.

Translation coverage – the fraction of graph nodes that received at least one semantic mapping – is reported by `get_coverage_report(graph)`. It returns the total node count, the number of covered nodes, the number of uncovered nodes, a `coverage_percent` value rounded to two decimal places, and the sorted list of uncovered node IDs. The uncovered list is intentionally emitted verbatim so that downstream tooling can target unmapped regions for manual review or rule extension, rather than burying the gap behind a single aggregate number [Allamanis et al., 2018a].

Algorithm 1: Fixpoint translation engine

Input: Program graph $G = (V, E)$; rule set R ; maximum iterations K (default 10)

Output: Set of semantic mappings $\mathcal{M}$ keyed by stable ID

```

1  $\mathcal{M} \leftarrow \emptyset$ ;
2 for  $k \leftarrow 1$  to  $K$  do
3    $n_{\text{new}} \leftarrow 0$ ;
4   foreach rule  $r \in R$  sorted by  $\text{priority}(r)$  descending do
5     foreach match  $m \in r.\text{matches}(G)$  do
6        $\mu \leftarrow r.\text{apply}(G, m)$ ;
7       if  $\mu \neq \perp$  and  $\mu.\text{id} \notin \mathcal{M}$  then
8          $\mathcal{M} \leftarrow \mathcal{M} \cup \{\mu\}$ ;
9          $n_{\text{new}} \leftarrow n_{\text{new}} + 1$ ;
10  if  $n_{\text{new}} = 0$  then
11    break ;                                     // fixed point reached
12  $\mathcal{M} \leftarrow \text{RESOLVECONFLICTS}(\mathcal{M})$ ;
13 return  $\mathcal{M}$ ;
```

Algorithm 1 summarizes the engine. Termination is guaranteed because each iteration either produces at least one new mapping (whose stable ID is then fixed in $\mathcal{M}$) or terminates by the break condition; the outer K bound serves as a safety valve for pathological rule sets.

Algorithm 2 detects conflicts via an inverted index in $O(\sum_v |\mathcal{I}(v)|^2)$ worst-case time, which is substantially faster than the naive all-pairs scan for graphs where most nodes carry at most one mapping.

Algorithm 2: Priority-ordered conflict resolution

Input: Mapping set $\mathcal{M}$; priority function $p(\cdot)$

Output: Reduced mapping set with no overlapping fragments

```
1 Build inverted index  $\mathcal{I} : V \rightarrow 2^{\mathcal{M}}$  from node IDs to mappings touching them;
2  $\mathcal{C} \leftarrow \emptyset$ ; // conflict pairs
3 foreach node  $v$  with  $|\mathcal{I}(v)| \geq 2$  do
4   foreach  $(\mu_a, \mu_b) \in \binom{\mathcal{I}(v)}{2}$  do
5      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\mu_a, \mu_b)\}$ ;
6  $\mathcal{R} \leftarrow \emptyset$ ; // removal set
7 foreach  $(\mu_a, \mu_b) \in \mathcal{C}$  do
8   if  $\mu_a \in \mathcal{R}$  or  $\mu_b \in \mathcal{R}$  then
9     continue
10   $k_a \leftarrow (p(\mu_a), c(\mu_a)); k_b \leftarrow (p(\mu_b), c(\mu_b));$ 
11  if  $k_a \geq k_b$  then
12     $\mathcal{R} \leftarrow \mathcal{R} \cup \{\mu_b\}$ 
13  else
14     $\mathcal{R} \leftarrow \mathcal{R} \cup \{\mu_a\}$ 
15 return  $\mathcal{M} \setminus \mathcal{R}$ ;
```

6 Confidence scoring, evidence tiers, and state-space compilation

6.1 Confidence Scoring and Evidence Tiers

The shipped `ConfidenceModel` in `../cogant/py/cogant/translate/confidence.py` computes a scalar score in $[0, 1]$ from:

- Average confidence over provenance records.
- An evidence-diversity term (scaled, capped).
- A parser certainty factor applied multiplicatively.
- Conflict penalties subtracted after scaling.

$$c = \max(0, \min(1, (\bar{e} + \delta_d) \cdot \kappa - \pi)) \quad (10)$$

Here $\bar{e}$ is the mean evidence confidence, δ_d is the diversity bonus (bounded), κ is parser certainty, and π aggregates conflict penalties. *only versus static—plus—runtime) are assigned from score thresholds and evidence source tags (determine_confidence_t)*

6.2 State space and behavior

Where traces or coverage are available, **dynamic** extraction feeds the state-space compiler. The goal is a compact behavioral model: states, actions, transitions, and observations that sit alongside the static graph for tasks that require execution-sensitive features. The tuple (S, A, T, O) of variables, actions, transitions, and observation modalities mirrors the structure of a partially observed

Markov decision process as used in discrete active inference [Parr et al., 2022, Da Costa et al., 2020, Smith et al., 2022], and at the level of reachable state/transition graphs it also resembles the Kripke structures traditionally used in model checking [Clarke et al., 1999], without attempting to discharge temporal-logic obligations. PyMDP [Heins et al., 2022] is one such downstream consumer that executes the compiled state-spaces as discrete active inference simulations over the exported Generalized Notation Notation bundles.

The shipped `StateSpaceCompiler` in `../cogant/py/cogant/stateSpace/compiler.py` constructs this model in several coordinated passes driven by the semantic mappings and the underlying program graph.

State variables are identified by a `StateVariableExtractor` that traverses graph nodes carrying READS and WRITES edges: any node that participates in a write is a candidate hidden-state variable, and its type, cardinality, and initial confidence are derived from the node’s recovered type string and the provenance of the rules that produced the associated mapping.

Actions are extracted from semantic mappings of kind ACTION, with parameters read from node metadata (typically the parser-recovered function signature), effects traced through outgoing WRITES edges on the controller node, and preconditions derived from parameter lists, docstring directives, and explicit metadata entries. For method-kind actions, the compiler also walks CONTAINS edges back to the enclosing class so that instance-level state mutations are attributed to the correct controller, mirroring how code property graphs fuse structural and data-flow views [Yamaguchi et al., 2014].

Transitions are inferred by a cross-reference pass (`_cross_reference_actions_and_variables`) that, for each action, partitions its adjacent variables into reads and writes based on edge kind (WRITES and MUTATES yielding writes; READS and OBSERVES yielding reads). The resulting `Transition` object records a `source_state` in which every touched variable is marked "pre" and a `target_state` in which written variables advance to "post" while read-only variables remain "pre"; this simple pre/post convention keeps the model faithful to the static evidence without overcommitting to symbolic value domains that cannot be recovered from AST analysis alone.

Trigger attribution follows incoming TRIGGERS and CALLS edges so that orchestration flow is preserved in the behavioral model.

Observation modalities are built from semantic mappings of kind OBSERVATION: each associated node becomes an `ObservationModality` whose modality type (`log`, `metric`, `event`, `sensor`, or a generic fallback) is inferred from the mapping’s description and the node’s name, giving the GNN export a typed observation channel aligned with the OBSERVES edges in the program graph.

The **temporal regime** of the model is determined by a companion `TemporalAnalyzer`. It classifies nodes as asynchronous when their metadata carries `is_async`/`async` flags or their names match patterns such as `async`, `callback`, `promise`, or `future`, and as event-related when their kind is `EVENT` or their names match `event`, `handler`, `listener`, or `trigger`. Temporal orderings are then extracted from CALLS and TRIGGERS edges, with each edge classified as `parallel` (when either endpoint is `async`) or `sequential` otherwise, and event patterns are assembled from triggers-in / triggers-out pairs around each event node.

A final decision rule selects among `SYNCHRONOUS`, `ASYNCHRONOUS`, `EVENT_DRIVEN`, and `HYBRID`: the presence of event triggers and patterns combined with `async` handlers yields `HYBRID`; event triggers

alone yield `EVENT_DRIVEN`; an async fraction above 30 percent or the presence of async handlers yields `ASYNCHRONOUS`; and all remaining cases default to `SYNCHRONOUS`. This regime is attached to the `StateSpaceModel` metadata so downstream consumers know which execution model the transition graph assumes.

When coverage and traces are available, `enrich_graph()` in `../cogant/py/cogant/dynamic/enrichment.py` feeds additional evidence into this pipeline. Coverage enrichment matches `.coverage` SQLite databases or Cobertura XML reports against nodes whose `path` and `source_range` overlap covered lines, attaching `coverage_hits` and, where branch data is available, `branch_coverage` metadata. Trace enrichment parses Chrome DevTools traces, writes `call_count`, `avg_duration_ms`, and `is_hot_path` onto matching callable nodes, and adds or reweights dynamic `CALLS` edges tagged with `evidence_sources=["dynamic_trace"]`.

Both steps also append `dynamic_coverage` and `dynamic_trace` markers to the program graph's evidence sources. The confidence model consumes these markers directly: any mapping whose evidence set now contains both static and dynamic entries becomes eligible for promotion from the `STATIC_ONLY` tier to `STATIC_PLUS_RUNTIME`, and hot-path and branch-coverage metadata raise the underlying score through the diversity bonus δ_d in Equation 10. This is the mechanism by which executing the target program, even partially, converts static heuristics into corroborated behavioral facts without rerunning the upstream rule engine [Allamanis et al., 2018a].

6.2.1 Worked example: temperature controller

Consider a small Python controller extracted from a HVAC codebase:

```
class TemperatureController:
    def __init__(self):
        self.current_temp: float = 20.0
        self.target_temp: float = 22.0
        self.heater_on: bool = False

    def set_target(self, t: float) -> None:
        self.target_temp = t

    def read_sensor(self, reading: float) -> None:
        self.current_temp = reading

    def actuate_heater(self) -> None:
        if self.current_temp < self.target_temp:
            self.heater_on = True
        else:
            self.heater_on = False
```

`StateVariableExtractor` identifies three **state variables** from `WRITES` edges on `__init__` and the three methods: `current_temp` (float, cardinality continuous), `target_temp` (float), and `heater_on` (bool, cardinality 2). Three **actions** are extracted from `ACTION`-kind mappings: `set_target` (writes `target_temp`), `read_sensor` (writes `current_temp`), and `actuate_heater` (reads `current_temp`, `target_temp`; writes `heater_on`). All three actions are attributed to

TemperatureController via CONTAINS edges.

The cross-reference pass yields three **transitions**. For `actuate_heater` the `source_state` records `{current_temp: "pre", target_temp: "pre", heater_on: "pre"}` and the `target_state` records `{current_temp: "pre", target_temp: "pre", heater_on: "post"}`, capturing that only `heater_on` advances while the read-only variables remain pinned. Because no node carries async flags or event-kind markers and no CALLS or TRIGGERS edges cross into async endpoints, the `TemporalAnalyzer` classifies the model as `SYNCHRONOUS` and attaches that regime to the `StateSpaceModel` metadata.

7 GNN export and error-handling philosophy

7.1 GNN export (Generalized Notation Notation)

Exports package the program graph and compiled state-space model into the Active Inference Institute’s **Generalized Notation Notation** plus a small set of interop targets:

- **GNN Markdown** (`model.gnn.md`) — canonical, human-readable Generalized Notation Notation organized into the section order defined in `../cogant/docs/export/README.md` (Model Metadata, Repository Metadata, Source Coverage, State Space, Observation Modalities, Actions/Policies, Connections, Factors, Transition Structure, Likelihood Structure, Preferences/Constraints, Time Settings, Parameterization, Ontology Mapping, Provenance, Confidence Scores, Rendering Hints, Validation Notes).
- **GNN JSON** (`model.gnn.json`) — machine-readable equivalent of the markdown sections, plus companion JSON files per section (`state_space.json`, `observations.json`, `actions.json`, `transitions.json`, and so on).
- **Interop exports** — GraphML and Parquet for analysis in Gephi/yEd and DuckDB, and optional tensor views (PyTorch Geometric Data objects [Fey and Lenssen, 2019], DGL graphs, HDF5 tables) for downstream graph neural network training pipelines that consume the program graph as a relational tensor.

Node and edge feature breakdowns, section contracts, and the optional framework targets are specified in `../cogant/docs/export/README.md`; they determine the structure of the emitted notation as well as the effective input dimensionality of any downstream model.

7.2 Error handling philosophy

The implementation distinguishes fatal configuration or parse failures from per-file errors that allow partial results. Validation aggregates warnings and inconsistencies into a report that travels with the bundle. This mirrors the layered error categories documented in the architecture (fatal, error, warning, info).

8 API and Workflows

This section describes the programmatic surface that the shipped Python package exposes in practice, aligned with `../cogant/docs/api/README.md` and the inventory-style notes under `../cogant/docs/reference/`. It is **not** an empirical benchmark section; COGANT does not ship comparative timing claims in the manuscript layer. Instead, it records the **surface area** users can rely on: two complementary entry points (a `Session` for stepwise work and a `Pipeline` for batch runs), the `Bundle` accessors that expose their artifacts, a command-line interface, and a Review API for human-in-the-loop curation.

8.1 Session-oriented workflow

Use `Session` for interactive exploration, notebook-driven debugging, and incremental re-runs where you want to materialize each stage’s output as a Python attribute. It is the right choice when a user wants to inspect intermediate artifacts between stages, iterate on a single repository in a notebook, or debug a specific extraction step without committing to a full scripted run. Each call returns control to the caller so that the graph, mappings, or state-space model can be examined before the next stage is invoked.

`Session.from_target` accepts a local path or URL, then supports a stepwise workflow:

- `extract_static` — AST-oriented extraction for supported languages.
- `extract_dynamic` — traces and coverage when inputs exist.
- `build_graph` — program graph construction.
- `translate_to_gnn` — Generalized Notation Notation (GNN) representation.
- `compile_state_space` — behavioral model when the pipeline has sufficient data.
- `export_all` — writes JSON artifacts under a chosen output directory.

This path suits interactive notebooks and incremental debugging.

8.2 Pipeline-oriented workflow

Use `PipelineRunner` for scripted, reproducible batch runs where all stages are configured up front and the end-state is a single `Bundle`. It is the right choice when a user wants to process many repositories with a fixed configuration, wire COGANT into CI, or guarantee that every run executes the same ordered stages with the same plugin settings. Because the whole run is described by a single `PipelineConfig`, it can be checked into version control and replayed without manual intervention.

`PipelineRunner` with `PipelineConfig` runs an ordered list of stages (ingest, static, normalize, graph, translate, state space, process, export, validate). Configuration can skip stages (for example dynamic analysis), attach **plugin** settings per language, set `output_dir`, verbosity, and dry-run mode. Results aggregate into a `Bundle` with `stage_results`, error lists, and accessors described below.

8.3 Bundle accessors

The bundle API exposes stage summaries and convenience render/export helpers, including:

- `repo_summary`, `program_graph`, `state_space_model`, `process_model`

- `gnn_markdown` — compact markdown summary built from `stage_results["translate"]`
- `validation_report`
- `render_site` — static HTML site with graph and model views
- `to_json` / `save_json`

For canonical 18-section Generalized Notation Notation artifacts (`model.gnn.md` plus companion JSON), use the export outputs documented in `../cogant/docs/export/README.md`; `Bundle.gnn_markdown()` is intentionally a lightweight report surface.

8.4 Command-line interface

The CLI entry point (`cogant.cli.main`) implements commands such as `init`, `scan`, `extract-static`, `extract-dynamic`, `graph`, `translate`, `statespace`, `validate`, and visualization/export helpers. Exact flags live in `../cogant/docs/cli/README.md`; the manuscript does not duplicate them to avoid drift.

8.5 Review API

`ReviewAPI` supports interactive curation: load a bundle, present mappings, accept, reject, or edit, then save a curated bundle. This closes the loop when human review is part of the ML dataset construction.

9 Examples and Failure Modes

This section walks through a concrete example run of the pipeline on a small Flask application, shows representative fragments of the artifacts it produces (semantic mappings and state-space transitions), and then documents the degradation behavior that users should expect when inputs are missing or partial. Together these illustrate both what a successful run looks like and how the system communicates partial success.

9.1 Example outputs

The package tree includes runnable examples under `../cogant/examples/` (for example `control_positive/`, `python-service/`, `workflow-engine/`, and `thin_orchestrated/`) and generated sample outputs under `../cogant/output/` (for example `roundtrip_calculator/` with `model.gnn.md`, `diagrams`, and `rendered charts`). Use `cogant translate --layout-output` or `PipelineConfig(layout_output=True)` to place pipeline JSON under `data/` automatically; run `python -m cogant.tools.render_output_figures` on the output root for PNGs. Regenerate these trees by running the packaged pipeline against the example sources when validating releases.

9.1.1 Concrete walkthrough: Flask REST API

To make the pipeline’s behavior tangible, consider the `flask_app` fixture distributed under `../cogant/examples/real_world/flask_app/`: a small six-module Flask application (`__init__.py`, `app.py`, `config.py`, `models.py`, `services.py`, `utils.py`) totalling 853 lines of Python analyzed end-to-end by the pipeline. Running `RoundtripOrchestrator` on this repository via `../cogant/examples/orchestrate_roundtrip.py` produces a program graph with the characteristics recorded in the canonical `../cogant/evaluation/figures/metrics.json`:

Metric	Value
Source files discovered	6
Lines analyzed	853
Nodes	98
Edges	597
Total semantic mappings	51
GNN package files	19
GNN validation	PASS (score 100.0, 0 errors, 0 warnings)

The node and edge populations distribute across the kinds the v0.5.x Python front end actually extracts (structural core: `MODULE` / `CLASS` / `METHOD` / `FUNCTION` plus `CONTAINS` / `WRITES` / `READS` / `CALLS` / `IMPORTS` / `INHERITS`), rather than the richer taxonomy that is declared in `cogant.schemas.core.NodeKind` and `EdgeKind` but remains roadmap for the Python parser.

Table 1. Node kind distribution (Flask API example).

Node kind	Count	Percentage
MODULE	6	6.1%
CLASS	25	25.5%
METHOD	57	58.2%
FUNCTION	10	10.2%

Table 2. Edge kind distribution (Flask API example).

Edge kind	Count	Percentage
CALLS	433	72.5%
CONTAINS	92	15.4%
READS	38	6.4%
WRITES	15	2.5%
IMPORTS	10	1.7%
INHERITS	9	1.5%

The CALLS-heavy distribution reflects the call-graph step in `CallGraphBuilder`, which traverses every `ast.Call` node in the module and attaches an edge between the enclosing function or method and its callee when the callee resolves inside the project. Control-flow nodes, anonymous data-flow nodes, and typed variable nodes are not yet emitted by the Python front end; broadening the node taxonomy is tracked as P1-2 and P1-3 in the R&D backlog (`../cogant/docs/evaluation/SCOPING_REPORT.md`).

9.1.2 Example semantic mapping output

During the translation stage, each program-graph node is matched against the active rule set and assigned a `SemanticMapping` with a `MappingKind` and confidence fields defined in `../cogant/py/cogant/schemas/semantic.py`. The following excerpt uses **real field names**; node IDs and file paths are **representative** of the `flask_app` fixture, not a verbatim `semantic_mappings.json` dump:

```
# Illustrative SemanticMapping-shaped records (see semantic.SemanticMapping)
- id: sm_get_users_obs
  kind: OBSERVATION
  graph_fragment_node_ids: [fn_get_users]
  semantic_label: "GET /users handler"
  confidence_score: 0.98
  confidence_tier: STATIC_ONLY
  parser_certainty: 1.0
  provenance:
    - {source: static_analysis, confidence: 1.0}

- id: sm_query_action
  kind: ACTION
```



```

graph_fragment_node_ids: [call_db_session_query]
semantic_label: "database query"
confidence_score: 0.82
confidence_tier: STATIC_ONLY
parser_certainty: 0.90
provenance:
  - {source: static_analysis, confidence: 0.90, metadata: {note: "receiver type partly heuristic"}}

- id: sm_user_model_hidden
kind: HIDDEN_STATE
graph_fragment_node_ids: [class_user]
semantic_label: "User model state"
confidence_score: 1.0
confidence_tier: STATIC_ONLY
provenance:
  - {source: static_analysis, confidence: 1.0}

```

The middle row illustrates how the confidence model (Equation 10 in §2) responds when parser certainty drops: import-traced receivers lower `parser_certainty`, which feeds Equation 10 and yields a sub-1.0 `confidence_score` even before dynamic evidence is available.

9.1.3 Example state-space excerpt

When dynamic traces are available, the state-space compiler produces a behavioral model alongside the static graph. The following excerpt shows a fragment of the state-space IR for the `/users` endpoint:

```

{
  "variables": [
    {"name": "request.method", "type": "str", "domain": ["GET", "POST"]},
    {"name": "db_connected", "type": "bool", "domain": [true, false]},
    {"name": "response_code", "type": "int", "domain": [200, 400, 500]}
  ],
  "actions": [
    {"name": "validate_input", "source": "routes/users.py:18"},
    {"name": "query_database", "source": "routes/users.py:22"},
    {"name": "format_response", "source": "routes/users.py:30"}
  ],
  "transitions": [
    {
      "from_state": {"request.method": "GET", "db_connected": true},
      "action": "query_database",
      "to_state": {"response_code": 200},
      "confidence": 0.94,
      "tier": "STATIC_PLUS_RUNTIME"
    },
    {

```

```

    "from_state": {"request.method": "POST", "db_connected": true},
    "action": "validate_input",
    "to_state": {"response_code": 400},
    "confidence": 0.71,
    "tier": "STATIC_ONLY"
  },
  {
    "from_state": {"db_connected": false},
    "action": "query_database",
    "to_state": {"response_code": 500},
    "confidence": 0.58,
    "tier": "RUNTIME_ONLY"
  }
]
}

```

Confidence tiers follow the thresholds defined in `determine_confidence_tier`: **STATIC_PLUS_RUNTIME** ($c \geq 0.65$, with both static and dynamic evidence) indicates corroboration from AST analysis and execution traces; **STATIC_ONLY** ($c \geq 0.5$, static evidence only) reflects assertions grounded in source structure alone; **RUNTIME_ONLY** ($c \geq 0.4$, dynamic evidence only) flags inferences from runtime data without static corroboration. A fourth tier, **HUMAN_REVIEWED** ($c \geq 0.9$, with human review evidence), is available for manually curated mappings.

9.1.4 Dynamically enriched excerpt

The previous excerpt shows a purely-static run. When `.coverage` / Cobertura XML and Chrome DevTools trace inputs are supplied to the dynamic stage, `enrich_graph()` annotates the affected nodes with `coverage_hits`, `branch_coverage`, `call_count`, `avg_duration_ms`, and `is_hot_path` metadata, and appends `dynamic_coverage` / `dynamic_trace` markers to the program graph's `evidence_sources` list. The downstream state-space compiler then becomes eligible to promote individual transitions from **STATIC_ONLY** to **STATIC_PLUS_RUNTIME** whenever the diversity bonus in Equation 10 clears the 0.65 boundary. The following excerpt shows the same Flask `/users` endpoint after re-running the pipeline with the application's `.coverage` file and a captured Chrome DevTools trace attached:

```

{
  "variables": [
    {
      "name": "request.method", "type": "str", "domain": ["GET", "POST"],
      "evidence_sources": ["static_ast", "dynamic_coverage", "dynamic_trace"],
      "coverage_hits": 47, "call_count": 47, "is_hot_path": true
    },
    {
      "name": "db_connected", "type": "bool", "domain": [true, false],
      "evidence_sources": ["static_ast", "dynamic_coverage"],
      "coverage_hits": 47, "branch_coverage": 0.92, "is_hot_path": false
    }
  ]
}

```

```

],
"transitions": [
  {
    "from_state": {"request.method": "GET", "db_connected": true},
    "action": "query_database",
    "to_state": {"response_code": 200},
    "confidence": 0.94, "tier": "STATIC_PLUS_RUNTIME",
    "evidence": {
      "static": {"rule_id": "rule_method_call_001", "parser_certainty": 0.95},
      "dynamic": {"coverage_hits": 47, "call_count": 47,
        "avg_duration_ms": 12.4, "is_hot_path": true}
    }
  },
  {
    "from_state": {"db_connected": false},
    "action": "query_database",
    "to_state": {"response_code": 500},
    "confidence": 0.58, "tier": "RUNTIME_ONLY",
    "evidence": {
      "dynamic": {"coverage_hits": 2, "call_count": 2,
        "avg_duration_ms": 3.1, "is_hot_path": false}
    }
  }
]
}

```

Two consequences of the enrichment are visible above. First, the `request.method` state variable’s `is_hot_path: true` annotation, combined with the `dynamic_coverage` and `dynamic_trace` markers in its `evidence_sources`, carried enough diversity mass through the confidence formula to lift the previously-static GET transition from the MEDIUM tier into `STATIC_PLUS_RUNTIME`, and the transition’s `evidence` field now records both the `rule_id` that fired statically and the `avg_duration_ms` measured dynamically. Second, the previously-`RUNTIME_ONLY` `db_connected=false` transition remains in its lower tier because it has only dynamic evidence (the error branch fired twice in the captured trace but has no corroborating static rule match) — a concrete illustration of the degradation behaviour documented in the “Incomplete coverage data” and “No dynamic traces available” subsections below.

9.1.5 Failure modes and graceful degradation

COGANT is designed so that missing or partial inputs degrade the output bundle rather than halt it, and the manuscript records these degradation paths explicitly so that downstream consumers can interpret a partial run without guessing what was skipped [Peng, 2011]. Five failure modes are worth naming because each has a visible signature in the emitted artifacts.

No dynamic traces available. When neither coverage data nor execution traces are supplied, `enrich_graph()` is a no-op: no `coverage_hits`, `branch_coverage`, `call_count`, `avg_duration_ms`, or `is_hot_path` metadata is attached, and the graph’s `evidence_sources`

list never acquires `dynamic_coverage` or `dynamic_trace` markers. The state-space compiler still runs end-to-end using the semantic mappings and static graph alone, but every resulting transition is confined to the `STATIC_ONLY` tier (or lower), and no transition can be promoted to `STATIC_PLUS_RUNTIME`. Downstream code that consumes the bundle can identify a purely-static run at a glance by checking the evidence source markers on the graph metadata rather than scanning individual transitions.

Incomplete coverage data. When coverage is supplied but covers only a subset of the project, `_enrich_with_coverage` annotates the matched files only. The enrichment loop walks each coverage span, normalizes the reported file path, looks up candidate nodes whose path matches, and annotates those whose `source_range` overlaps the covered line; unmatched files are silently skipped with an informational log line, and nodes in unmatched files retain no coverage metadata at all. The result is a partially enriched graph in which some regions are eligible for tier promotion and others are not. Because the enrichment summary returns the number of annotated nodes, users can check whether the observed annotation rate matches their expectations for the provided coverage input.

Translation rules that do not match. When the active rule set fails to produce any mapping for a node – whether because no rule fires, or because all firing rules are pruned during conflict resolution – that node remains outside `self.mappings` and simply does not appear in any `graph_fragment_node_ids`. The translation engine’s `get_coverage_report()` reports this directly: the returned dictionary contains the total node count, the covered count, a two-decimal `coverage_percent`, and the sorted list of `uncovered_node_ids`. Rather than treat uncovered nodes as an error, the engine surfaces them as an explicit gap list so that authors can either extend the rule set, hand the gap list to the `ReviewAPI` for curation, or accept the partial mapping for downstream Generalized Notation Notation (GNN) export.

Fixpoint non-convergence. The translation engine bounds its fixpoint loop at `max_iterations = 10` by default. If a rule set is pathological enough to keep emitting new mappings past that bound – for example because two rules can produce mutually triggering mappings – the engine emits a "Max iterations reached without convergence" warning, stops the loop, and proceeds to conflict resolution with whatever mappings it has accumulated. The iteration cap therefore guarantees termination even for misconfigured rule sets, and the warning in the log gives rule authors a clear signal that the cap was hit. Because each iteration also writes an `iteration_complete` entry into the engine’s internal match log, the exact per-pass mapping counts are available for diagnosis without rerunning the pipeline.

Pipeline error tolerance. The default pipeline configuration (`cogant/py/cogant/config/defaults.py`) marks the `dynamic`, `translate`, `state_space`, and `process` stages with `skip_on_error=True`, while structural stages such as `ingest`, `static`, `graph`, `export`, and `validate` keep the stricter default `skip_on_error=False`. When an optional stage raises, the runner records the error in the bundle, emits a warning, and continues to the next stage rather than aborting the run. Downstream bundle accessors (`state_space_model`, `process_model`) simply return `None` for stages that did not produce output, so a downstream consumer can detect a skipped stage by checking its accessor rather than by parsing log text. The net effect is that a partial bundle – for example, a program graph and semantic mappings without a state-space model – remains a first-class artifact with a clear provenance trail, rather than an opaque failure.

9.2 Rust layer

Native crates (`cogant-core`, `cogant-graph`, `cogant-translate`, `cogant-statespace`, `cogant-gnn`, `cogant-ffi`, and related packages) implement typed graph operations and export formatting. When PyO3 bindings are active, Python delegates heavy graph work through `cogant-ffi`. Where bindings are not yet wired for a code path, Python fallbacks apply; see [../cogant/docs/reference/implementation_status.md](#) for the current boundary.

10 Conclusion

COGANT frames codebase analysis as a pipeline from ingestion through a program graph IR to **Generalized Notation Notation (GNN)** exports, with explicit confidence and provenance so that learning systems can treat analysis noise as data rather than as hidden failure modes. The Python layer provides session and pipeline APIs, a bundle abstraction, CLI, review tooling, and HTML reporting; the Rust layer concentrates graph mechanics and export formatting under crate boundaries described in `../cogant/docs/architecture/README.md`.

10.1 Shipped Capabilities

Several capabilities ship in the current v0.5.0 release and together define the behaviour that downstream users can rely on:

1. **Fixpoint translation engine with 19 real rules.** The shipped `TranslationEngine` re-applies every registered rule on each pass, terminates as soon as a pass produces zero new mappings, and bounds pathological rule sets with a configurable iteration cap. Each iteration boundary is recorded in the internal match log, so convergence can be audited after the fact. The packaged rule set under `../cogant/py/cogant/translate/rules/` contains 19 concrete `TranslationRule` subclasses: five structural rules (`ReadOnlyInput`, `MutatingSubsystem`, `Inheritance`, `Containment`, `DataPipeline`), five semantic rules (`Observation`, `Action`, `Policy`, `Preference`, `Context`), two control rules (`Config`, `FeatureFlag`), three behavioural rules (`Orchestrator`, `TestAssertion`, `EventBus`), and four resilience rules (`RetryPattern`, `ErrorBoundary`, `SingletonAccess`, `CircuitBreaker`) — each a real implementation with `matches()` and `apply()` bodies rather than a placeholder stub.
2. **Rule priority and conflict resolution.** Overlapping mappings are reconciled by lexicographic (`rule_priority`, `confidence_score`): when two mappings cover overlapping `graph_fragment_node_ids`, the engine retains the larger tuple and records a `conflict_resolved` event naming the winner, the loser, and the overlap. Most shipped rules use the default `TranslationRule.priority` of 0; `MutatingSubsystemRule` overrides to 1 so hidden-state evidence wins class-level ties against aggregate `POLICY` from `InheritanceRule` unless rescoring changes the ordering. `translate_with_confidence()` re-scores and re-resolves so that confidence-model updates are honoured.
3. **Dynamic enrichment from coverage and traces, wired into the default pipeline.** When `.coverage` SQLite or Cobertura XML inputs and Chrome DevTools traces are supplied, `enrich_graph()` annotates nodes with `coverage_hits`, `branch_coverage`, `call_count`, `avg_duration_ms`, and `is_hot_path` metadata, and appends `dynamic_coverage` and `dynamic_trace` markers to the program graph's evidence sources. The dynamic stage is part of the default pipeline stage list in `cogant/py/cogant/api/pipeline.py`, and the shipped CLI exposes a `--no-dynamic` flag (handled by `PipelineConfig.skip_dynamic`) for scripted runs that must force a purely-static bundle.
4. **Confidence tier promotion.** Mappings whose evidence set acquires dynamic markers become eligible for promotion from `STATIC_ONLY` to `STATIC_PLUS_RUNTIME`, and hot-path and branch-coverage metadata raise the underlying score through the diversity bonus in Equation 10. A purely-static run remains a first-class bundle; it is simply marked as such on the graph metadata.
5. **Ten-stage pipeline architecture.** The runner executes an ordered stage list (`ingest`,

- `static, normalize, graph, dynamic, translate, statespace, process, export, validate`). Stage handlers run inside per-stage `try/except` blocks in `PipelineRunner.run()`: if one stage fails, the error is appended to the bundle and execution continues to remaining stages. This produces partial but inspectable outputs instead of an immediate pipeline abort.
6. **Complete state-space compilation.** The `StateSpaceCompiler` in `../cogant/py/cogant/statespace/compiler.py` emits `StateVariable`, `ObservationModality`, `Action`, and `Transition` records directly from the program graph and semantic mappings. On the 6 packaged fixtures every run produces a non-empty state-space model, which the `GNNPackageBuilder` then writes out as `state_space.json`, `observations.json`, `actions.json`, and `transitions.json` inside the canonical `gnn_package/` directory.
 7. **A/B/C/D matrix derivation.** The `GNNMatrices` class in `../cogant/py/cogant/gnn/matrices.py` derives the likelihood matrix `A[n_obs x n_states]`, the transition matrix `B[n_states x n_states x n_actions]`, the preference vector `C` over observations, and the prior `D` over hidden states directly from the compiled state-space and the underlying program graph edges. Rows and vectors are normalized to valid probability distributions, so the emitted bundle is directly compliant with the Active Inference Institute’s upstream GNN validator.
 8. **Principled variational and expected free energy.** The functions `variational_free_energy` and `expected_free_energy` in `../cogant/py/cogant/simulate/free_energy.py` operate on the derived A/B/C/D matrices and implement the canonical Active Inference formulation — $VFE = KL[Q(s) \parallel P(s)] - E_Q[\log P(o|s)]$ and $EFE = \sum \text{over policies of } epistemic_value - pragmatic_value$ — rather than the hard-coded scalars used in earlier iterations. The `GNNModelRunner` exercises these functions on every compiled bundle during pipeline execution.
 9. **Markov blanket extraction with five seed strategies.** `../cogant/py/cogant/markov/blanket.py` partitions nodes into internal, sensory, active, and external roles, and `extractor.py` selects seeds via five strategies (`explicit`, `module`, `kind`, `auto` via cohesion heuristic, and `mapping_kind`). The resulting `markov_blanket.json` file travels inside every `gnn_package/` directory and is consumed by the downstream Active Inference runner.
 10. **Real-world fixture validation.** Beyond the synthetic `control_positive/` fixtures, the packaged `examples/real_world/` tree now holds three reductions of third-party Python projects (`flask_app`, `requests_lib`, and `json_stdlib`). The pipeline runs end-to-end on all 6 repositories in under seven seconds each on a single macOS workstation (representative wall-clock measurements in Section 6), producing a complete 19-file GNN package and a 100.0/100 validator score on every run; concrete measurements are recorded in Section 6.
 11. **Forward-reverse-forward round-trip (canonical evaluation set).** The v0.5.0 reverse synthesizer now emits `POLICY / CONTEXT / CONSTRAINT` scaffolds proportional to the origin GNN’s role counts, closing the multiset loss that had kept several uncured real-world targets in the `APPROXIMATE` or `DIVERGENT` buckets at v0.4.0. Current `METRICS.yaml` reports 14 isomorphic, 6 approximate, and 3 divergent targets (of 23 total). The canonical `cogant/docs/evaluation/ROUNDTrip_IMPROVEMENT.md` documents the `POLICY/CONTEXT` fix trajectory; see §9, S01, S02 for tables.
 12. **Incremental analysis mode.** The shipped `cogant analyze --incremental <git-ref>` command (and its library form `PipelineConfig.incremental_since`) re-uses the previous run’s program graph for unchanged source paths, re-running only the affected subgraph. On the Flask benchmark the measured speedups are 19.6× for a no-change invocation and 5.6× for a single-file change, making COGANT tractable inside per-commit CI loops without giving

up the rule-based pipeline.

13. **Multi-episode Bayesian learning.** `AgentRuntime.run_multi_episode` and `run_episode` drive Active Inference rollouts over the compiled `gnn_package/`; the companion `update_D_from_posterior` and `update_A_from_counts` helpers implement Dirichlet / count-based posterior updates that let downstream consumers refine the prior and likelihood matrices from observed episodes without leaving the COGANT runtime.
14. **Production FastAPI server.** `cogant.server.app` exposes `/health` and `/translate` endpoints with an integration test suite. A packaged `Dockerfile` (python:3.12-slim + uv, EXPOSE 8080, curl healthcheck) and `docker-compose.yml` turn the translation pipeline into a deployable microservice; this unblocks the P2 target of running COGANT as a background service in research-group infrastructure.
15. **Cross-language roundtrip demonstration.** The `examples/zoo/13_js_observer` fixture exercises a JavaScript observer implementation (with optional `cogant[multilang]` and an installed grammar) through the forward pipeline, the reverse synthesizer, and a full AI cycle, recording a `role_match_score` of 1.0 on the forward-reverse-forward comparison when that path is enabled. This demonstrates that the tree-sitter-backed JavaScript parser plus the language-agnostic translation and statespace layers can round-trip a non-Python source language on the zoo regression set.

Limitations follow the honest scope in `../cogant/docs/reference/implementation_status.md`: Python is first-class; JavaScript/TypeScript require optional `cogant[multilang]` and grammars; additional languages are largely roadmap; translation rules and state-space extraction remain **partial** and repository-dependent (with a long-tail of domain-specific idioms unaddressed by the 19 shipped rules); native acceleration is wired only for `connected_components` on graph construction; and the roundtrip result is measured on a **23**-target canonical set that is weighted toward curated fixtures. Users should validate exports on their own corpora before trusting downstream model metrics.

Intended users include researchers building datasets from open-source repositories, teams prototyping Active Inference models or graph neural network training pipelines over program-graph data who need a single export contract, and engineers extending the system via `../cogant/docs/plugins/README.md`.

Validation in the software-engineering sense is split: the repository’s verification report enumerates implemented modules and entry points; scientific validation of model quality remains the responsibility of downstream training and evaluation code.

10.2 Roadmap and Future Extensions

Several concrete directions extend the current system. Items already shipped in v0.2.0 – v0.5.0 (JavaScript/TypeScript parsing via `tree-sitter`, partial Rust acceleration of `connected_components`, incremental re-analysis via `cogant analyze --incremental`) have been removed from this list and moved into the Shipped Capabilities section above.

1. **Additional language parsers.** Python ships first-class in v0.5.0; JavaScript/TypeScript are available behind optional `cogant[multilang]` plus grammars. Adding parsers for Java, Go, Rust, and C/C++ would cover the majority of remaining open-source ML-relevant repositories. Each parser implements the plugin interface documented in

`../cogant/docs/plugins/README.md`, so language additions do not require changes to the core IR or export pipeline. The cross-language round-trip demonstration on `examples/zoo/13_js_observer` establishes the template for validating new parsers before release.

2. **Broader Rust acceleration.** The v0.5.0 PyO3 `connected_components` FFI demonstrates the template but touches only a single graph algorithm. Wiring bindings for the remaining hot paths — rule matching on large `CallGraphBuilder` outputs, Generalized Notation Notation section/tensor packing in `cogant-gnn`, and the B-tensor normalisation loops — would reduce end-to-end latency on the largest real-world targets (`fastapi`, `urllib3`) from the current ≈ 30 s to sub-second timings, based on the Python-vs-Rust microbenchmarks committed under `../cogant/benchmarks/results/`.
3. **LLM-assisted rule discovery.** Translation rules are currently hand-authored. A semi-automated workflow could present a large language model with unannotated graph fragments and ask it to propose candidate rules, which a human reviewer then accepts, edits, or rejects via the existing `ReviewAPI`. This combines the pattern-recognition strength of LLMs with the auditability of declarative rules; the confidence-tier and provenance-record infrastructure already in place gives each LLM-proposed rule a natural quarantine state until a human upgrades it.
4. **Cross-repository graph linking.** When multiple repositories share interfaces (for example, a library and its consumers), linking their program graphs at call boundaries produces a richer training signal for tasks such as API misuse detection and cross-project code search. The graph homomorphism property defined in Section 2 provides the formal basis for identifying shared interface nodes across independently analyzed repositories.
5. **Long-tail rule coverage and calibration.** The 19 shipped translation rules carry deliberate `TODO(calibration)` markers for confidence-threshold tuning (see `cogant/py/cogant/translate/confidence.py` and `cogant/py/cogant/namespace/variables.py`). A 20+ repository gold-standard corpus, annotated by human reviewers through the `ReviewAPI`, would turn those principled defaults into empirically validated thresholds and surface the long-tail idioms that the current rule families miss.

11 Experimental setup: environment, API, and configuration

12 Experimental setup

12.1 Environment

Requirements: Python 3.11 or newer (enforced in `pyproject.toml`), plus an optional Rust toolchain (cargo, stable 1.70+) when building native acceleration crates under `../cogant/rust/`.

From the COGANT package root `../cogant/` (where `pyproject.toml` and `py/cogant/` live), install with `uv sync --all-extras`, or `pip install -e "[dev,viz]"` / `pip install -e "[all]"` as in [GETTING_STARTED.md](#). Run those commands from that directory when working inside this monorepo layout. Python sources live under `../cogant/py/cogant/`; see the package [README.md](#).

12.2 Running the API

Minimal **Session** run:

```
from cogant import Session

session = Session.from_target("../path/to/repo")
session.extract_static()
session.build_graph()
session.translate_to_gnn()
session.export_all("output/")
```

Minimal **Pipeline** run:

```
from cogant import PipelineRunner
from cogant.api.pipeline import PipelineConfig

runner = PipelineRunner()
config = PipelineConfig(output_dir="output/")
bundle = runner.run("../path/to/repo", config)
```

Adjust `PipelineConfig.stages`, `skip_stages`, and `plugins` to match the languages and tooling available on the machine.

12.3 Configuration files

YAML configuration can drive pipeline behavior (paths, stages, plugin options). [../cogant/docs/architecture/README](#) and [../cogant/docs/reference/implementation_status.md](#) describe the configuration surface; keep project-specific secrets out of version control.

A minimal pipeline configuration looks like this:

```
# cogant.config.yaml
pipeline:
```

```

stages:
  - ingest
  - static
  - normalize
  - graph
  - dynamic      # optional; skipped if no coverage/trace inputs
  - translate
  - statespace
  - process
  - export
  - validate

skip_stages: []    # e.g. ["dynamic"] to force static-only runs

plugins:
  dynamic:
    coverage_path: "./coverage.xml"
    trace_path: "./chrome_trace.json"

output_dir: "./cogant_output/"
verbose: true
dry_run: false

```

Each stage key corresponds to a handler in `cogant.api.pipeline.PipelineRunner.stage_handlers`; plugin sub-dictionaries are passed through to the stage at invocation time. `cogant translate --config` accepts either a top-level `pipeline` object or a flat mapping and normalizes both forms into `PipelineConfig`.

12.4 CLI

Use the `cogant` CLI for scripted batch runs—see `../cogant/docs/cli/README.md` for the command list that matches the installed version.

13 Exports, parser capabilities, and progressive IR stages

13.1 Export targets

The primary export targets are the **Generalized Notation Notation (GNN)** canonical Markdown (`model.gnn.md`) and the equivalent companion JSON files described in `../cogant/docs/export/README.md`. Optional interop targets (GraphML, Parquet) support analysis in Gephi/yEd and DuckDB, and optional tensor views for PyTorch Geometric, DGL, or HDF5 can be selected when downstream graph neural network training pipelines need to consume the program graph as a relational tensor. Ensure the Python environment includes optional dependencies for these tensor exports when those code paths are used.

13.2 Python AST parser capabilities

The v0.5.x front end relies on `cogant.static.parser.PythonASTParser`, which processes Python source through the standard-library `ast` module at the CPython version available in the runtime (3.11+ required, consistent with the `requires-python = ">=3.11"` declared in `../cogant/pyproject.toml`). The parser extracts the following construct categories:

- **Module-level entities:** module docstrings, `__all__` exports, top-level assignments.
- **Functions and methods:** `def` and `async def`, including signatures with positional, keyword, variadic (`*args`, `**kwargs`), and positional-only parameters. Default values are recorded as constant expressions where statically evaluable.
- **Classes:** class definitions, base classes, metaclasses, and the `__init__` / `__new__` boundary.
- **Decorators:** `@staticmethod`, `@classmethod`, `@property`, `@dataclass`, and arbitrary user-defined decorators. Decorator arguments are captured as attribute metadata.
- **Type annotations:** PEP 484 / 526 / 604 annotations on function parameters, return types, and variable assignments. Generic subscripts (`List[int]`, `Dict[str, Any]`) are preserved as type strings.
- **Comprehensions and generators:** list, set, dict comprehensions and generator expressions are represented as anonymous FUNCTION nodes with DATA_FLOW edges to their enclosing scope.
- **Control flow:** `if/elif/else`, `for/while/else`, `try/except/finally`, `with/async with`, and `match/case` (Python 3.10+) are mapped to CONTROLFLOW_NODE entities.
- **Imports:** `import` and `from ... import` statements produce MODULE_IMPORT roles with edges to the resolved module when discoverable on the file system.
- **Constants:** module-level and class-level assignments to Final or ALL_CAPS names are classified as CONSTANT nodes.

Constructs that require runtime evaluation (for example `exec`, `importlib.import_module`, or dynamic `__getattr__`) are recorded as EXTERNAL nodes with HEURISTIC provenance and correspondingly lower confidence.

13.3 Progressive IR stages

Processing advances through six intermediate representations, each adding semantic detail atop its predecessor. Table 3 summarizes what each stage contributes.

Table 3. Progressive IR stages and their contributions.

Stage	IR name	Key additions	Typical output size (10K-function repo)
1	Repo IR	Raw entities and relationships per file; deduplication; merged type info	~15 MB JSON
2	Program Graph IR	Consolidated directed graph $G = (V, E)$; stable identifiers; confidence and provenance on every node and edge	~20 MB JSON
3	Semantic Mapping IR	Translation rules applied; semantic roles assigned; confidence adjusted by rule engine	~22 MB JSON (graph + mapping log)
4	State Space IR	Variables, actions, transitions, observations; dynamic traces integrated where available	~5 MB JSON (behavioral model)
5	Process Model IR	Higher-level control patterns (request-response, producer-consumer, state machines)	~2 MB JSON
6	Validation IR	Coverage metrics, confidence distribution, schema compliance, consistency checks, reproducibility hashes	~1 MB JSON (report)

Stages 4 and 5 are **partial** for many repositories: the state-space compiler requires either execution traces or sufficient static structure (for example annotated state machines) to produce meaningful output. Where dynamic evidence is available, COGANT’s ingestion pipeline follows the established

pattern of attaching runtime observations (coverage, call frequencies, traces) to static program elements — dynamic instrumentation frameworks such as Pin [Luk et al., 2005] and invariant detectors such as Daikon [Ernst et al., 2007] established this general approach of augmenting static program structure with execution-time evidence. The pipeline tolerates missing stages gracefully; the Validation IR records which stages completed and which were skipped.

14 Performance targets and measured runs on packaged fixtures

14.1 Performance characteristics

The architecture targets the following benchmarks on a 4-core machine, as specified in `../cogant/docs/architecture/README.md`:

Repository size	Target wall-clock time	Memory budget
10K functions	< 30 s	< 500 MB
100K functions	< 5 min	< 2 GB
1M functions	< 1 hr	< 2 GB (streaming)

These are architecture targets, not benchmark claims from this manuscript. They assume the Python orchestration layer with Rust acceleration on critical paths (graph construction, rule matching, and Generalized Notation Notation section/tensor packing in `cogant-gnn`). In the current v0.5.x release, Rust acceleration is partially wired — `cogant._rust` exposes a PyO3 `connected_components` FFI for graph construction behind the `COGANT_USE_RUST` feature flag — and a pure-Python fallback handles the remaining code paths.

Current `PipelineRunner` behavior is stage-sequential with per-stage error capture and continuation. It does not currently expose built-in incremental checkpoint/resume in `cogant.api.pipeline`; treat checkpointing as a potential outer-orchestration feature rather than a guaranteed package-level runtime behavior.

14.2 Measured runs on packaged fixtures

The following tables record measurements taken by running the shipped `RoundtripOrchestrator` (`../cogant/examples/orchestrate_roundtrip.py`) against every fixture distributed with the package. Three fixtures are the control-positive synthetic repositories under `../cogant/examples/control_positive/` (`calculator`, `event_pipeline`, `flask_mini`); the other three are real-world code under `../cogant/examples/real_world/` (`flask_app`, a six-module Flask service; `requests_lib`, a six-module reduction of the `requests` HTTP library; and `json_stdlib`, a four-module reduction of the CPython `json` package). Each run executes the full static pipeline (ingest, parse, symbols, imports, call graph, program graph, translation, state-space compilation, GNN package build, validation). Wall-clock times were measured on a single macOS workstation with the pure-Python fallback implementations — the v0.5.x PyO3 `connected_components` FFI is disabled for this canonical run (`COGANT_USE_RUST=0`) so that these numbers correspond to the Python orchestration layer with no native crates loaded.

All numbers in Tables 4–7 are regenerated by `../cogant/evaluation/figures/generate_figures.py`, which re-runs the orchestrator over every fixture, re-reads the emitted JSON, and writes `../cogant/evaluation/figures/metrics.json` alongside the figure PNGs. Structural metrics (nodes, edges, edge-kind and node-kind breakdowns, LOC, file counts) are deterministic; rule-driven metrics (total mappings, state variables, observations, actions, transitions) vary by at most one or two units across runs because the extractor walks dictionaries whose ordering is

process-local, and wall-clock times vary by a few seconds depending on whether the visualization pass rasterizes PNGs. The figures below match the canonical `metrics.json` committed under `../cogant/evaluation/figures/`.

Table 4. Repository-level pipeline metrics (canonical run, COGANT v0.1.0).

Fixture	Files	LOC	Nodes	Edges	State Mappings	vars	Obs	Actions	Transitions	GNN sec- mins	GNN score	Wall- clock (s)
<code>calculator</code>	1	122	12	27	5	0	3	1	1	31	100.0	7.42
<code>event_pipeline</code>	1	147	23	66	20	1	9	10	10	31	100.0	8.58
<code>flask_mini</code>	1	168	26	51	19	3	2	14	14	31	100.0	8.80
<code>flask_app</code>	6	853	98	597	51	16	19	16	16	31	100.0	14.58
<code>requests_lib</code>	6	750	98	345	46	9	31	7	7	31	100.0	12.19
<code>json_stdlib</code>	4	1231	29	68	8	3	5	0	0	31	100.0	9.08

“GNN sections” counts the level-two Markdown headings emitted in `model.gnn.md`, which on every fixture sits at 31 (the 18 core Generalized Notation Notation sections plus section-specific subheadings for state space, observations, actions, and transitions). “GNN score” is the `score` field returned by `GNNValidator.validate()` on the compiled `gnn_package/` directory; every fixture validates at 100.0 with zero errors and zero warnings. The six fixtures together cover one-to-six file repositories and 122 to 1231 lines of code, exercising the pipeline on both minimal control positives and small real-world modules. Wall-clock times fall between roughly seven and fifteen seconds on a 2024-class Apple-silicon workstation; the bulk of the cost on the larger fixtures is the call-graph construction step in `CallGraphBuilder` plus the PNG rasterization pass in `cogant.viz.png_export`.

Table 5. Program graph composition by fixture.

Node kinds (MODULE / CLASS / METHOD / FUNCTION) and edge kinds (CONTAINS / WRITES / READS / CALLS / IMPORTS / INHERITS) are populated directly from the AST extractor and call-graph builder; all other kinds listed in `cogant.schemas.core.NodeKind` and `EdgeKind` remain unused on these fixtures because the Python front end currently focuses on the structural core. The counts below are taken verbatim from the `statistics` block of each fixture’s `program_graph.json`.

Fixture	MODULE	CLASS	METHOD	FUNCTION	CONTAINS	WRITES	READS	CALLS	IMPORTS	INHERITS
<code>calculator</code>	1	1	10	—	11	5	9	2	—	—
<code>event_pipeline</code>	6	6	16	—	22	1	10	30	—	3
<code>flask_mini</code>	1	7	18	—	25	6	7	11	—	2
<code>flask_app</code>	6	25	57	10	92	15	38	433	10	9
<code>requests_lib</code>	6	20	59	13	92	8	42	183	10	10
<code>json_stdlib</code>	4	3	9	13	25	3	6	32	2	—

The distribution matches the intuitive shape of the fixtures: class-heavy repositories (`flask_app`, `requests_lib`, `flask_mini`) show METHOD-dominated node counts and INHERITS edges, while

the functional `json_stdlib` shows a balanced METHOD/FUNCTION split and no INHERITS edges. CALLS edges dominate the large repositories (`flask_app` at 433, `requests_lib` at 183), confirming that the call-graph construction step in `CallGraphBuilder` is the primary source of edge density on real code.

Table 6. State-space compilation outputs.

For each fixture the `StateSpaceCompiler` emits a `StateSpaceModel` whose variables, observations, actions, and transitions are then packaged into `gnn_package/state_space.json`, `observations.json`, `actions.json`, and `transitions.json`. The counts reflect the end-to-end behavior of the compiler on these inputs, not the rule engine’s raw mapping output.

Fixture	State variables	Observations	Actions	Transitions	Policies
<code>calculator</code>	0	3	1	1	1
<code>event_pipeline</code>	1	9	10	10	4
<code>flask_mini</code>	3	2	14	14	6
<code>flask_app</code>	16	19	16	16	6
<code>requests_lib</code>	9	31	7	7	1
<code>json_stdlib</code>	3	5	0	0	1

`calculator` compiles zero hidden-state variables because the fixture exposes pure arithmetic methods whose WRITES-edge footprint does not cross the classifier threshold used by `StateVariableExtractor`; it still compiles a single action and a single transition, so the pipeline remains end-to-end valid and the GNN package still validates at 100.0. `json_stdlib` compiles zero actions because the reduced CPython `json` sources consist almost entirely of function-level utilities whose semantic role falls outside the ACTION-mapping rule set, but the pipeline still emits a compiled `actions.json` with an empty `actions` list and a single default policy — downstream consumers see the same schema as for the action-bearing fixtures. The `requests_lib` fixture has the highest observation count in the table because its session/adaptor classes expose a large number of read-only attributes that the rule engine matches as OBSERVATION mappings.

Table 7. Output artifacts per run.

Every fixture emits the same `gnn_package/` directory layout with 19 canonical files: `model.gnn.md`, `model.gnn.json`, the section JSONs (`state_space`, `observations`, `actions`, `actions_policies`, `transitions`, `preferences`, `preferences_constraints`, `factors`, `connections`, `ontology`, `provenance`), the `program_graph.json` and `process_model.json` snapshots, the `markov_blanket.json` and `markov_network.json` extracts, and the `manifest.json` index. The `RoundtripOrchestrator` additionally writes diagnostic artifacts at the top level of `output_dir`: six Mermaid diagrams (class, state, sequence, dependency, boundary, semantic flow), typed graph and Cytoscape exports, a GraphML file, a Parquet table, a dashboard HTML site, simulation traces, and a GNN execution report.

Fixture	<code>gnn_package/</code> files	Validation errors	Validation warnings
<code>calculator</code>	19	0	0
<code>event_pipeline</code>	19	0	0

Fixture	<code>gnn_package/</code> files	Validation errors	Validation warnings
<code>flask_mini</code>	19	0	0
<code>flask_app</code>	19	0	0
<code>requests_lib</code>	19	0	0
<code>json_stdlib</code>	19	0	0

These numbers were collected by the reproducible script `../cogant/evaluation/figures/generate_figures.py`, which re-runs the orchestrator over every fixture, re-reads the emitted JSON, and writes both the summary table and the figures used in this manuscript into `../cogant/evaluation/figures/` (namely `fig1_graph_sizes.png`, `fig2_node_kinds.png`, `fig3_state_space.png`, and `fig4_pipeline_latency.png`, plus the machine-readable `metrics.json`).

15 Test matrix, mutation testing, and benchmark suite

15.1 Test matrix and coverage

The v0.5.0 Python implementation ships a test suite that, on the canonical `uv run pytest tests/ --cov=py/cogant` run, reports **2129 passing** tests with **86 skips** for optional dependencies (Rust toolchain, `matplotlib`, `tree-sitter` language grammars, PNG rasterization), plus **2 expected xfail** and **1 xpass** case. End-to-end runtime is on the order of four minutes on a 2024-class Apple-silicon workstation (**238 s** in the canonical run); the overall line coverage of `py/cogant/` is **83.42%** on that run, measured across **56628** lines in **179** source files (see `METRICS.yaml`).

Table 8. Python interpreter matrix.

Python version	<code>pyproject.toml</code> classifier	Status
3.11	Programming Language :: Python :: 3.11	supported (minimum version, <code>requires-python = ">=3.11"</code>)
3.12	Programming Language :: Python :: 3.12	supported (canonical CI interpreter; benchmark runs use 3.12.11)
3.13	Programming Language :: Python :: 3.13	supported

All three interpreters are listed in the `classifiers` block of `../cogant/pyproject.toml`. The declared minimum is Python 3.11 so that the pattern-matching front end in `cogant.static.parser.PythonASTParser` can use `match/case` statements without a compatibility shim, and the benchmark suite recorded in `benchmarks/results/suite_20260409.md` was executed on CPython 3.12.11 under macOS arm64.

Module-level coverage is concentrated in the layers that the **6** packaged fixtures exercise end-to-end. Table 9 records the coverage of the algorithmic core (translation, state-space compilation, Markov blanket extraction, GNN matrix construction, and the reverse synthesizer) — the modules whose correctness is load-bearing for every claim in the manuscript. Numbers are taken from the `TOTAL`-line breakdown of the `uv run pytest --cov` run that produced the 2129/86 pass/skip summary.

Table 9. Line coverage of load-bearing modules (canonical v0.5.0 run, 2026-04-10).

Module	Lines	Coverage
<code>cogant.translate.engine</code>	160	90%
<code>cogant.translate.rules.structural</code>	185	93%
<code>cogant.translate.rules.semantic</code>	216	92%
<code>cogant.translate.rules.behavioral</code>	79	98%
<code>cogant.translate.rules.control</code>	47	93%
<code>cogant.translate.rules.resilience</code>	130	95%
<code>cogant.translate.confidence</code>	98	97%
<code>cogant.statespace.compiler</code>	418	91%

Module	Lines	Coverage
<code>cogant.statespace.variables</code>	182	98%
<code>cogant.statespace.temporal</code>	173	81%
<code>cogant.markov.blanket</code>	full coverage (reported in the “60 files skipped due to complete coverage” block of the canonical run)	100%
<code>cogant.gnn.matrices</code>	full coverage (same block)	100%
<code>cogant.static.calls</code>	151	86%
<code>cogant.static.dataflow</code>	246	84%
<code>cogant.static.parser</code>	236	86%
<code>cogant.simulate.free_energy</code>	165	99%
<code>cogant.simulate.runner</code>	252	67%
<code>cogant.simulate.distributions</code>	118	95%
<code>cogant.scoring.drift</code>	215	99%
<code>cogant.scoring.metrics</code>	142	99%
<code>cogant.validate.integrity</code>	136	94%
<code>cogant.validate.schema_check</code>	115	95%
<code>cogant.validate.provenance_check</code>	73	97%

The aggregate project-level coverage reported at the end of the run is **83.42%**; the modules that drag the average down are the visualisation layer (`cogant.viz.png_export`, `cogant.viz.plots`, `cogant.viz.mermaid`, with residual gaps where optional `matplotlib` and `plotly` code paths are skipped) plus a small number of scaffolded plugin or provenance helpers (for example `cogant.viz.bundle_site`, an HTML site generator that requires the `jinja2` extra and therefore may not execute under the default `uv sync` environment). The algorithmic core — everything that participates in the round-trip theorem of §9 — remains at high coverage on the exercised modules in Table 9. The `simulate.distributions` and `simulate.free_energy` modules are reported in that table for the canonical v0.5.0 run; see `../cogant/CHANGELOG.md` for release-cycle deltas.

15.2 Mutation testing

Mutation testing was performed on the algorithmic core modules (`gnn/matrices.py`, `translate/engine.py`, `markov/blanket.py`, `statespace/compiler.py`, `static/dataflow.py`). The canonical `mutmut` 3.5.0 runner was evaluated but rejected: on COGANT’s test layout `mutmut` reported every one of the 403 auto-generated mutants on `matrices.py` as “no tests” because its v3 trampoline requires tests to import the mutated module through the `mutants/<path>` shadow tree, and the project’s `pytest` configuration does not. Rather than ship a “no tests” score, the mutation analysis in `../cogant/docs/evaluation/MUTATION_REPORT.md` is based on a **hand-picked set of fifteen semantic mutations** that target the algorithmic predicates, constants, and loop bounds of the above modules; each mutation was applied, the relevant `pytest` subset was rerun, and the mutation was reverted immediately. This is a more informative experiment than a green `mutmut` run because it documents exactly *which* invariants the tests enforce.

Table 10. Hand-curated mutation results on COGANT algorithmic core.

Module	Mutants tested	Killed	Survived	Mutation score
<code>gnn/matrices.py</code>	5	3	2	60%
<code>translate/engine.py</code>	3	1	2	33%
<code>markov/blanket.py</code>	2	1	1	50%
<code>statespace/compiler.py</code>	2	1	1	50%
<code>static/dataflow.py</code>	3	3	0	100%
Total	15	10	5	66.7%

The five surviving mutants are documented individually in `../cogant/docs/evaluation/MUTATION_REPORT.md` §“Surviving mutants — action required” (aversive preference path in `compute_C`, sensory active boundary role swap in `markov/blanket.py`, `>=>` boundary flip in `_map_confidence`, `CONFIGURATION` neighbour bias in `compute_D`, and the single-pass fixpoint iteration cap). Three of the five were closed by hardening tests in the same commit: `test_C_aversive_preference_produces_negative_log_prob` kills the aversive-preference survivor, `test_boundary_with_only_outgoing_edge_is_active` / `test_boundary_with_only_incoming_edge_is_sensory` kill the Markov-blanket swap, and `test_map_confidence_exact_boundary_values` kills the `>=>` family. The remaining two survivors (`CONFIGURATION`-bias and single-pass fixpoint) are documented follow-ups that require non-trivial fixture extensions. The measured score on the hand-picked set is therefore **10 killed / 15 total = 66.7%** before hardening, and the documented target after the follow-ups is 80% or better.

The modules with the strongest mutation signal are `static/dataflow.py` (3 of 3 killed — every edge-kind string mutation is caught by the dataflow-tuple-assertion tests) and the row/column normalisation paths in `gnn/matrices.py` (4 of 5 killed — every arithmetic mutation to `_normalize_row`, `_DEFAULT_DIRECT_MASS`, the `compute_C` sign-flip, the `compute_B` axis swap, and the `compute_A` fallback branch is killed by the `A_rows_sum_to_one`, `B_columns_sum_to_one_per_action`, and `A_concentrates_mass_on_direct_reads` tests). The modules that drag the overall score down are those whose tests assert structural invariants (disjointness, normalisation, shape) but do not pin down the *direction* or *magnitude* of individual entries.

15.3 Benchmark suite (shipped)

A reproducible benchmark harness lives at `../cogant/benchmarks/bench_suite.py` and writes its canonical results to `../cogant/benchmarks/results/`. The most recent run committed to the tree (`bench(p1.5): Rust build status + Python vs Rust benchmark results`, then superseded by `bench(suite): reproducible 6-fixture benchmark harness with stage timing + memory + GNN stats`) executed each fixture for three iterations on CPython 3.12.11 / macOS arm64 and recorded per-stage wall-clock time, peak memory, and the final GNN tensor shapes.

Table 11. Benchmark suite results (`suite_20260409.md`, three iterations per fixture, CPython 3.12.11).

Fixture	Wall-clock median (ms)	Wall-clock p95 (ms)	Nodes	Edges	Mappings	Peak memory (MB)
<code>calculator</code>	32	35	12	25	11	0.0
<code>event_pipeline</code>	36	37	23	36	21	0.1
<code>flask_mini</code>	43	45	26	40	25	0.3
<code>flask_app</code>	86	86	98	154	68	0.3
<code>requests_lib</code>	76	77	98	152	55	0.7
<code>json_stdlib</code>	48	49	29	34	19	0.0

The benchmark harness times the bare translation pipeline (`ingest`, `static`, `normalize`, `graph`, `translate`, `statespace`), so its wall-clock numbers are approximately an order of magnitude smaller than the end-to-end roundtrip times of Table 4: Tables 4 and 5 include validation, GNN package assembly, Mermaid and PNG rasterization, GraphML and Parquet serialization, and the HTML dashboard write, none of which are part of the benchmark hot path. For pure translation, every shipped fixture runs in under 100 ms and consumes less than a megabyte of peak memory; the stage breakdown in `suite_20260409.md` shows that for the smaller fixtures the dominant cost is `ingest` (repository walk + file hashing), and for the larger fixtures (`flask_app`, `requests_lib`) the dominant cost shifts to `graph` construction where `CallGraphBuilder` walks every `ast.Call` node to produce the CALLS edges recorded in Table 5.

Approximate stage breakdown from the same run: `ingest` 25–30 ms across all fixtures; `static` 1–4 ms; `normalize` 0–3 ms; `graph` 4–43 ms (dominated by `flask_app`); `translate` 0–3 ms; `statespace` 0–1 ms. The benchmark results file also records the per-fixture GNN tensor shapes — for example `flask_app` produces $A \in \mathbb{R}^{21 \times 10}$, $B \in \mathbb{R}^{10 \times 10 \times 31}$, $C \in \mathbb{R}^{21}$, $D \in \mathbb{R}^{10}$ — which match the state-space compiler outputs of Table 6 up to the benchmark’s independent re-run sampling variance.

16 What to record for reproducible experiments

16.1 Minimum recording checklist

The COGANT pipeline is deterministic on a fixed filesystem snapshot under a fixed Python interpreter, so bit-for-bit reproduction of a published run reduces to pinning exactly the inputs that the pipeline consumes and the environment it consumes them in. For every experiment whose output you intend to re-run, redistribute, or cite, record the following:

1. **COGANT version and commit hash.** The `__version__` string in `../cogant/py/cogant/__init__.py` matches `project.version` in `../cogant/pyproject.toml` (currently 0.5.0). Also capture the Git SHA of the checkout used — `git rev-parse HEAD` from inside the `cogant/` subtree — because a single version tag can span several bug-fix commits and the reproducibility contract is at commit granularity, not tag granularity. The shipped `cogant doctor` command prints both values.
2. **Python interpreter version.** Either CPython 3.11.x, 3.12.x, or 3.13.x per the `classifiers` block in `pyproject.toml`; the canonical benchmark run used CPython 3.12.11 on macOS arm64. Capture `python --version` plus `uname -a` (or the equivalent on Windows / Linux) so that the OS and architecture are visible downstream.
3. **Dependency lock.** `uv sync --extra all` resolves from `../cogant/uv.lock`; include the lockfile in any redistributed dataset so that optional extras (`viz`, `multilang`, `all`) resolve to the exact versions that produced the published numbers. If the pinned `tree-sitter-javascript`, `tree-sitter-typescript`, `pyarrow`, or `duckdb` versions drift, re-parsing the same JavaScript sources can produce different AST shapes and therefore different `program_graph.json` outputs.
4. **Input repository commit hash.** For every target repository processed through the pipeline, record the Git commit hash of the snapshot at ingest time. The pipeline does not hash the input for you — it simply walks the filesystem — so an unpinned input is the most common source of non-reproducibility. For the shipped fixtures under `../cogant/examples/`, the relevant hash is the COGANT commit itself (the fixtures are vendored).
5. **Configuration file contents (redacted).** If you invoked the pipeline via `cogant.yaml` or `PipelineConfig(...)`, persist the configuration next to the run outputs. Redact any credentials, private path components, or API tokens before publication; the config loader in `../cogant/py/cogant/config/` does not automatically strip these.
6. **List of stages executed.** The default ten-stage DAG is `ingest` → `static` → `normalize` → `graph` → `dynamic` → `translate` → `statespace` → `process` → `export` → `validate`, but the CLI and `PipelineConfig` expose selective-execution flags (`--no-dynamic`, `--skip-validate`, `PipelineConfig.skip_stages`). Record which stages were enabled for the published run so that a reproducer does not silently re-enable an optional stage whose output was not part of the original report.
7. **Random seeds for learned consumers.** COGANT itself is deterministic — it does not invoke any stochastic components in the default pipeline. However, any **downstream learned component** (for example an optional embedding model supplied to the GNN exporter, or a graph neural network trained on COGANT’s exports) *is* stochastic and must record its own seeds. Include the seed with the derived dataset, not with the COGANT bundle.
8. **Canonical output hashes.** After each run, hash the `gnn_package/` directory and the top-level `bundle.json` with a content-addressable digest (`sha256` is sufficient). Store these

hashes alongside the run outputs; republishing the same hash across different machines is the strongest reproducibility signal available to a reader.

16.2 How to re-run the canonical fixture experiments

All numbers in Tables 4–7 of §6.3 and in the benchmark suite of §6.4 are regenerated by two scripts that live inside the COGANT package tree and take no arguments beyond a `--output` directory:

```
# One-shot regeneration of every figure and the metrics.json summary
uv run python ../cogant/evaluation/figures/generate_figures.py \
    --output ../cogant/evaluation/figures/

# Benchmark harness - three iterations per fixture, stage-level timing
uv run python ../cogant/benchmarks/bench_suite.py \
    --iterations 3 \
    --output ../cogant/benchmarks/results/
```

The figure-generation script re-runs the RoundtripOrchestrator (`../cogant/examples/orchestrate_roundtrip.py`) against every fixture under `../cogant/examples/control_positive/` and `../cogant/examples/real_world/`, re-reads the emitted JSON, writes `metrics.json` alongside `fig1_graph_sizes.png`, `fig2_node_kinds.png`, `fig3_state_space.png`, and `fig4_pipeline_latency.png`, and finally copies the canonical figure set back into the `../cogant/evaluation/figures/` tree. The benchmark-suite harness times the bare translation pipeline (`ingest`, `static`, `normalize`, `graph`, `translate`, `statespace`) and writes a dated Markdown report of the form `suite_YYYYMMDD.md` alongside a machine-readable JSON sidecar; the report committed as `../cogant/benchmarks/results/suite_20260409.md` is the canonical source for Table 11 in §6.4.

16.3 What the manifest.json index records

Every `gnn_package/` directory emitted by the pipeline includes a `manifest.json` file whose job is to close the reproducibility loop without requiring the downstream consumer to re-hash the bundle. The manifest records: the COGANT version that produced the bundle, the interpreter and platform identifiers from step 2 of the checklist, the list of stages actually executed (step 6), the schema name passed to the state-space compiler, the paths of every file in the `gnn_package/` directory, and the SHA-256 hash of each file. A reproducer can therefore verify a published bundle with a single pass over the manifest, without consulting any external metadata file.

16.4 Worked example: regenerating the Table 4 row for flask_app

The canonical Table 4 row for the `flask_app` fixture (six files, 853 lines, 98 nodes, 597 edges, 14.58 s wall-clock) was produced as follows, and the command sequence below should bit-for-bit reproduce the same `program_graph.json` and `state_space.json` on any macOS arm64 machine with the same uv lockfile:

```
cd projects_in_progress/cogant/cogant
uv sync --extra all
git rev-parse HEAD
python --version && uname -a
uv run cogant translate \

# package root
# pins every dep per uv.lock
# record the commit hash
# record interpreter + platform
```



```

    examples/real_world/flask_app \
    --output /tmp/repro_flask_app \
    --layout-output
uv run cogant validate /tmp/repro_flask_app/gnn_package
sha256sum /tmp/repro_flask_app/gnn_package/*.json | sort

```

The `validate` step should report score 100.0 / 100 with zero errors and zero warnings, and the sorted SHA-256 listing should match the one recorded in `../cogant/examples/real_world/flask_app/output/manifest` (committed alongside the fixture). Any mismatch points to either a dependency drift (step 3) or an input drift (the Flask fixture itself changed between the committed output and the re-run).

16.5 Data ethics and licensing for exported bundles

Every COGANT bundle can contain identifiers, docstrings, and inline comments lifted verbatim from the input source code. When redistributing a bundle derived from a third-party repository — for example when publishing a dataset of `gnn_package/` outputs from open-source Python projects — the downstream license terms must be respected: the original repository’s license applies to any prose or identifier that the bundle quotes, and the COGANT MIT license covers only the pipeline code and the synthetic fixtures it ships. Organisations with private data policies should additionally review bundles for personally identifiable information before publication; the pipeline does not perform PII scrubbing on its own. The short policy statement is in §7 (see **Data ethics and licensing** in [07_reproducibility.md](#)); this subsection supplies the actionable hashing and manifest-recording recipe.

16.6 Cross-references

- The CLI hub at [../cogant/docs/cli/README.md](#) links to every flag that changes the recorded-output shape, and the stage list in [../cogant/docs/architecture/README.md](#) enumerates the ten-stage DAG.
- The per-release narrative in [../cogant/CHANGELOG.md](#) and [../cogant/RELEASE_NOTES.md](#) documents which default-behaviour changes (for example the v0.5.0 POLICY / CONTEXT stub-emission fix discussed in §5 and [ROUNDTRIP_IMPROVEMENT.md](#)) could affect a re-run against a pre-v0.5.0 bundle.
- The calibration sweep plan in [../cogant/docs/evaluation/CALIBRATION.md](#) is the canonical reference for the `TODO(calibration)` markers cited in §5; re-running a confidence-threshold sweep requires the 20+ repository gold-standard corpus discussed there.

17 Experimental setup

17.1 Environment

Requirements: Python 3.11 or newer (enforced in `pyproject.toml`), plus an optional Rust toolchain (cargo, stable 1.70+) when building native acceleration crates under `../cogant/rust/`.

From the COGANT package root `../cogant/` (where `pyproject.toml` and `py/cogant/` live), install with `uv sync --all-extras`, or `pip install -e ".[dev,viz]"` / `pip install -e ".[all]"` as in [GETTING_STARTED.md](#). Run those commands from that directory when working inside this monorepo layout. Python sources live under `../cogant/py/cogant/`; see the package [README.md](#).

17.2 Running the API

Minimal **Session** run:

```
from cogant import Session

session = Session.from_target("../path/to/repo")
session.extract_static()
session.build_graph()
session.translate_to_gnn()
session.export_all("output/")
```

Minimal **Pipeline** run:

```
from cogant import PipelineRunner
from cogant.api.pipeline import PipelineConfig

runner = PipelineRunner()
config = PipelineConfig(output_dir="output/")
bundle = runner.run("../path/to/repo", config)
```

Adjust `PipelineConfig.stages`, `skip_stages`, and `plugins` to match the languages and tooling available on the machine.

17.3 Configuration files

YAML configuration can drive pipeline behavior (paths, stages, plugin options). [../cogant/docs/architecture/README](#) and [../cogant/docs/reference/implementation_status.md](#) describe the configuration surface; keep project-specific secrets out of version control.

A minimal pipeline configuration looks like this:

```
# cogant.config.yaml
pipeline:
  stages:
    - ingest
    - static
```

```

- normalize
- graph
- dynamic      # optional; skipped if no coverage/trace inputs
- translate
- statespace
- process
- export
- validate

skip_stages: []    # e.g. ["dynamic"] to force static-only runs

plugins:
  dynamic:
    coverage_path: "./coverage.xml"
    trace_path: "./chrome_trace.json"

output_dir: "./cogant_output/"
verbose: true
dry_run: false

```

Each stage key corresponds to a handler in `cogant.api.pipeline.PipelineRunner.stage_handlers`; plugin sub-dictionaries are passed through to the stage at invocation time. `cogant translate --config` accepts either a top-level `pipeline` object or a flat mapping and normalizes both forms into `PipelineConfig`.

17.4 CLI

Use the `cogant` CLI for scripted batch runs—see `../cogant/docs/cli/README.md` for the command list that matches the installed version.

17.5 Export targets

The primary export targets are the **Generalized Notation Notation (GNN)** canonical Markdown (`model.gnn.md`) and the equivalent companion JSON files described in `../cogant/docs/export/README.md`. Optional interop targets (GraphML, Parquet) support analysis in Gephi/yEd and DuckDB, and optional tensor views for PyTorch Geometric, DGL, or HDF5 can be selected when downstream graph neural network training pipelines need to consume the program graph as a relational tensor. Ensure the Python environment includes optional dependencies for these tensor exports when those code paths are used.

17.6 Python AST parser capabilities

The v0.5.x front end relies on `cogant.static.parser.PythonASTParser`, which processes Python source through the standard-library `ast` module at the CPython version available in the runtime (3.11+ required, consistent with the `requires-python = ">=3.11"` declared in `../cogant/pyproject.toml`). The parser extracts the following construct categories:

- **Module-level entities:** module docstrings, `__all__` exports, top-level assignments.

- **Functions and methods:** `def` and `async def`, including signatures with positional, keyword, variadic (`*args`, `**kwargs`), and positional-only parameters. Default values are recorded as constant expressions where statically evaluable.
- **Classes:** class definitions, base classes, metaclasses, and the `__init__` / `__new__` boundary.
- **Decorators:** `@staticmethod`, `@classmethod`, `@property`, `@dataclass`, and arbitrary user-defined decorators. Decorator arguments are captured as attribute metadata.
- **Type annotations:** PEP 484 / 526 / 604 annotations on function parameters, return types, and variable assignments. Generic subscripts (`List[int]`, `Dict[str, Any]`) are preserved as type strings.
- **Comprehensions and generators:** list, set, dict comprehensions and generator expressions are represented as anonymous FUNCTION nodes with DATA_FLOW edges to their enclosing scope.
- **Control flow:** `if/elif/else`, `for/while/else`, `try/except/finally`, `with/async with`, and `match/case` (Python 3.10+) are mapped to CONTROLFLOW_NODE entities.
- **Imports:** `import` and `from ... import` statements produce MODULE_IMPORT roles with edges to the resolved module when discoverable on the file system.
- **Constants:** module-level and class-level assignments to `Final` or `ALL_CAPS` names are classified as CONSTANT nodes.

Constructs that require runtime evaluation (for example `exec`, `importlib.import_module`, or dynamic `__getattr__`) are recorded as EXTERNAL nodes with HEURISTIC provenance and correspondingly lower confidence.

17.7 Progressive IR stages

Processing advances through six intermediate representations, each adding semantic detail atop its predecessor. Table 3 summarizes what each stage contributes.

Table 3. Progressive IR stages and their contributions.

Stage	IR name	Key additions	Typical output size (10K-function repo)
1	Repo IR	Raw entities and relationships per file; deduplication; merged type info	~15 MB JSON
2	Program Graph IR	Consolidated directed graph $G = (V, E)$; stable identifiers; confidence and provenance on every node and edge	~20 MB JSON

Stage	IR name	Key additions	Typical output size (10K-function repo)
3	Semantic Mapping IR	Translation rules applied; semantic roles assigned; confidence adjusted by rule engine	~22 MB JSON (graph + mapping log)
4	State Space IR	Variables, actions, transitions, observations; dynamic traces integrated where available	~5 MB JSON (behavioral model)
5	Process Model IR	Higher-level control patterns (request-response, producer-consumer, state machines)	~2 MB JSON
6	Validation IR	Coverage metrics, confidence distribution, schema compliance, consistency checks, reproducibility hashes	~1 MB JSON (report)

Stages 4 and 5 are **partial** for many repositories: the state-space compiler requires either execution traces or sufficient static structure (for example annotated state machines) to produce meaningful output. Where dynamic evidence is available, COGANT’s ingestion pipeline follows the established pattern of attaching runtime observations (coverage, call frequencies, traces) to static program elements — dynamic instrumentation frameworks such as Pin [Luk et al., 2005] and invariant detectors such as Daikon [Ernst et al., 2007] established this general approach of augmenting static program structure with execution-time evidence. The pipeline tolerates missing stages gracefully; the Validation IR records which stages completed and which were skipped.

17.8 Performance characteristics

The architecture targets the following benchmarks on a 4-core machine, as specified in `../cogant/docs/architecture/README.md`:

Repository size	Target wall-clock time	Memory budget
10K functions	< 30 s	< 500 MB

Repository size	Target wall-clock time	Memory budget
100K functions	< 5 min	< 2 GB
1M functions	< 1 hr	< 2 GB (streaming)

These are architecture targets, not benchmark claims from this manuscript. They assume the Python orchestration layer with Rust acceleration on critical paths (graph construction, rule matching, and Generalized Notation Notation section/tensor packing in `cogant-gnn`). In the current v0.5.x release, Rust acceleration is partially wired — `cogant._rust` exposes a PyO3 `connected_components` FFI for graph construction behind the `COGANT_USE_RUST` feature flag — and a pure-Python fallback handles the remaining code paths.

Current `PipelineRunner` behavior is stage-sequential with per-stage error capture and continuation. It does not currently expose built-in incremental checkpoint/resume in `cogant.api.pipeline`; treat checkpointing as a potential outer-orchestration feature rather than a guaranteed package-level runtime behavior.

17.9 Measured runs on packaged fixtures

The following tables record measurements taken by running the shipped `RoundtripOrchestrator` (`../cogant/examples/orchestrate_roundtrip.py`) against every fixture distributed with the package. Three fixtures are the control-positive synthetic repositories under `../cogant/examples/control_positive/` (`calculator`, `event_pipeline`, `flask_mini`); the other three are real-world code under `../cogant/examples/real_world/` (`flask_app`, a six-module Flask service; `requests_lib`, a six-module reduction of the `requests` HTTP library; and `json_stdlib`, a four-module reduction of the CPython `json` package). Each run executes the full static pipeline (ingest, parse, symbols, imports, call graph, program graph, translation, state-space compilation, GNN package build, validation). Wall-clock times were measured on a single macOS workstation with the pure-Python fallback implementations — the v0.5.x PyO3 `connected_components` FFI is disabled for this canonical run (`COGANT_USE_RUST=0`) so that these numbers correspond to the Python orchestration layer with no native crates loaded.

All numbers in Tables 4–7 are regenerated by `../cogant/evaluation/figures/generate_figures.py`, which re-runs the orchestrator over every fixture, re-reads the emitted JSON, and writes `../cogant/evaluation/figures/metrics.json` alongside the figure PNGs. Structural metrics (nodes, edges, edge-kind and node-kind breakdowns, LOC, file counts) are deterministic; rule-driven metrics (total mappings, state variables, observations, actions, transitions) vary by at most one or two units across runs because the extractor walks dictionaries whose ordering is process-local, and wall-clock times vary by a few seconds depending on whether the visualization pass rasterizes PNGs. The figures below match the canonical `metrics.json` committed under `../cogant/evaluation/figures/`.

Table 4. Repository-level pipeline metrics (canonical run, COGANT v0.1.0).

Fixture	Files	LOC	Nodes	Edges	State Mappings	vars	Obs	Actions	Transitions	GNN sec- mins	GNN score	Wall- clock (s)
calculator	1	122	12	27	5	0	3	1	1	31	100.0	7.42
event_pipeline	1	147	23	66	20	1	9	10	10	31	100.0	8.58
flask_mini	1	168	26	51	19	3	2	14	14	31	100.0	8.80
flask_app	6	853	98	597	51	16	19	16	16	31	100.0	14.58
requests_lib	6	750	98	345	46	9	31	7	7	31	100.0	12.19
json_stdlib	4	1231	29	68	8	3	5	0	0	31	100.0	9.08

“GNN sections” counts the level-two Markdown headings emitted in `model.gnn.md`, which on every fixture sits at 31 (the 18 core Generalized Notation Notation sections plus section-specific subheadings for state space, observations, actions, and transitions). “GNN score” is the `score` field returned by `GNNValidator.validate()` on the compiled `gnn_package/` directory; every fixture validates at 100.0 with zero errors and zero warnings. The six fixtures together cover one-to-six file repositories and 122 to 1231 lines of code, exercising the pipeline on both minimal control positives and small real-world modules. Wall-clock times fall between roughly seven and fifteen seconds on a 2024-class Apple-silicon workstation; the bulk of the cost on the larger fixtures is the call-graph construction step in `CallGraphBuilder` plus the PNG rasterization pass in `cogant.viz.png_export`.

Table 5. Program graph composition by fixture.

Node kinds (MODULE / CLASS / METHOD / FUNCTION) and edge kinds (CONTAINS / WRITES / READS / CALLS / IMPORTS / INHERITS) are populated directly from the AST extractor and call-graph builder; all other kinds listed in `cogant.schemas.core.NodeKind` and `EdgeKind` remain unused on these fixtures because the Python front end currently focuses on the structural core. The counts below are taken verbatim from the `statistics` block of each fixture’s `program_graph.json`.

Fixture	MODULE	CLASS	METHOD	FUNCTION	CONTAINS	WRITES	READS	CALLS	IMPORTS	INHERITS
calculator	1	1	10	—	11	5	9	2	—	—
event_pipeline	6	6	16	—	22	1	10	30	—	3
flask_mini	1	7	18	—	25	6	7	11	—	2
flask_app	6	25	57	10	92	15	38	433	10	9
requests_lib	6	20	59	13	92	8	42	183	10	10
json_stdlib	4	3	9	13	25	3	6	32	2	—

The distribution matches the intuitive shape of the fixtures: class-heavy repositories (`flask_app`, `requests_lib`, `flask_mini`) show METHOD-dominated node counts and INHERITS edges, while the functional `json_stdlib` shows a balanced METHOD/FUNCTION split and no INHERITS edges. CALLS edges dominate the large repositories (`flask_app` at 433, `requests_lib` at 183), confirming that the call-graph construction step in `CallGraphBuilder` is the primary source of edge density on real code.

Table 6. State-space compilation outputs.

For each fixture the `StateSpaceCompiler` emits a `StateSpaceModel` whose variables, observations, actions, and transitions are then packaged into `gnn_package/state_space.json`, `observations.json`, `actions.json`, and `transitions.json`. The counts reflect the end-to-end behavior of the compiler on these inputs, not the rule engine’s raw mapping output.

Fixture	State variables	Observations	Actions	Transitions	Policies
<code>calculator</code>	0	3	1	1	1
<code>event_pipeline</code>	1	9	10	10	4
<code>flask_mini</code>	3	2	14	14	6
<code>flask_app</code>	16	19	16	16	6
<code>requests_lib</code>	9	31	7	7	1
<code>json_stdlib</code>	3	5	0	0	1

`calculator` compiles zero hidden-state variables because the fixture exposes pure arithmetic methods whose WRITES-edge footprint does not cross the classifier threshold used by `StateVariableExtractor`; it still compiles a single action and a single transition, so the pipeline remains end-to-end valid and the GNN package still validates at 100.0. `json_stdlib` compiles zero actions because the reduced CPython `json` sources consist almost entirely of function-level utilities whose semantic role falls outside the ACTION-mapping rule set, but the pipeline still emits a compiled `actions.json` with an empty `actions` list and a single default policy — downstream consumers see the same schema as for the action-bearing fixtures. The `requests_lib` fixture has the highest observation count in the table because its session/adaptor classes expose a large number of read-only attributes that the rule engine matches as `OBSERVATION` mappings.

Table 7. Output artifacts per run.

Every fixture emits the same `gnn_package/` directory layout with 19 canonical files: `model.gnn.md`, `model.gnn.json`, the section JSONs (`state_space`, `observations`, `actions`, `actions_policies`, `transitions`, `preferences`, `preferences_constraints`, `factors`, `connections`, `ontology`, `provenance`), the `program_graph.json` and `process_model.json` snapshots, the `markov_blanket.json` and `markov_network.json` extracts, and the `manifest.json` index. The `RoundtripOrchestrator` additionally writes diagnostic artifacts at the top level of `output_dir`: six Mermaid diagrams (class, state, sequence, dependency, boundary, semantic flow), typed graph and Cytoscape exports, a GraphML file, a Parquet table, a dashboard HTML site, simulation traces, and a GNN execution report.

Fixture	<code>gnn_package/</code> files	Validation errors	Validation warnings
<code>calculator</code>	19	0	0
<code>event_pipeline</code>	19	0	0
<code>flask_mini</code>	19	0	0
<code>flask_app</code>	19	0	0
<code>requests_lib</code>	19	0	0
<code>json_stdlib</code>	19	0	0

These numbers were collected by the reproducible script `../cogant/evaluation/figures/generate_figures.py`,

which re-runs the orchestrator over every fixture, re-reads the emitted JSON, and writes both the summary table and the figures used in this manuscript into `../cogant/evaluation/figures/` (namely `fig1_graph_sizes.png`, `fig2_node_kinds.png`, `fig3_state_space.png`, and `fig4_pipeline_latency.png`, plus the machine-readable `metrics.json`).

17.10 Test matrix and coverage

The v0.5.0 Python implementation ships a test suite of **2129 passing tests** with **86 skips** for optional dependencies (Rust toolchain, `matplotlib`, `tree-sitter` language grammars, PNG rasterization) plus **2** expected `xfail` and **1** `xpass`. The suite executes in approximately four minutes on a 2024-class Apple-silicon workstation (**238 s** in the canonical run recorded in `METRICS.yaml`), and the overall line coverage of `py/cogant/` is **83.42%**, measured against executable statements reported by `coverage.py` on the canonical run (**2026-04-10T21:03:18.504378Z**).

Table 8. Python interpreter matrix.

Python version	pyproject.toml classifier	Status
3.11	Programming Language :: Python :: 3.11	supported (minimum version, <code>requires-python = ">=3.11"</code>)
3.12	Programming Language :: Python :: 3.12	supported (canonical CI interpreter; benchmark runs use 3.12.11)
3.13	Programming Language :: Python :: 3.13	supported

All three interpreters are listed in the `classifiers` block of `../cogant/pyproject.toml`. The declared minimum is Python 3.11 so that the pattern-matching front end in `cogant.static.parser.PythonASTParser` can use `match/case` statements without a compatibility shim, and the benchmark suite recorded in `benchmarks/results/suite_20260409.md` was executed on CPython 3.12.11 under macOS arm64.

Module-level coverage is concentrated in the layers that the six packaged fixtures exercise end-to-end. Table 9 records the coverage of the algorithmic core (translation, state-space compilation, Markov blanket extraction, GNN matrix construction, and the reverse synthesizer) — the modules whose correctness is load-bearing for every claim in the manuscript. Numbers are taken from the TOTAL-line breakdown of the canonical v0.5.0 `uv run pytest --cov run` (2026-04-10).

Table 9. Line coverage of load-bearing modules.

Module	Lines	Coverage
<code>cogant.translate.engine</code>	approx. 300	87%
<code>cogant.translate.rules.structural</code>	approx. 350	high
<code>cogant.translate.rules.semantic</code>	approx. 330	86%
<code>cogant.statepace.compiler</code>	427	91%

Module	Lines	Coverage
<code>cogant.statespace.variables</code>	180	98%
<code>cogant.statespace.temporal</code>	173	81%
<code>cogant.markov.blanket</code>	approx. 250	90%+
<code>cogant.gnn.matrices</code>	approx. 440	high
<code>cogant.static.calls</code>	151	86%
<code>cogant.simulate.free_energy</code>	165	65%
<code>cogant.simulate.runner</code>	250	67%
<code>cogant.simulate.distributions</code>	119	34%

The aggregate project-level coverage reported at the end of the run is **83.42%**; the modules that drag the average down are the visualisation layer (`cogant.viz.png_export`, `cogant.viz.plots`, `cogant.viz.mermaid`, where optional `matplotlib` and `plotly` code paths are skipped under the default CI configuration) and the scaffolded plugin and provenance trackers. The algorithmic core — everything that participates in the round-trip theorem of §9 — remains the focus of the high-coverage modules in Table 9.

17.11 Mutation testing

Mutation testing was performed on the algorithmic core modules (`gnn/matrices.py`, `translate/engine.py`, `markov/blanket.py`, `statespace/compiler.py`, `static/dataflow.py`). The canonical `mutmut` 3.5.0 runner was evaluated but rejected: on COGANT’s test layout `mutmut` reported every one of the 403 auto-generated mutants on `matrices.py` as “no tests” because its v3 trampoline requires tests to import the mutated module through the `mutants/<path>` shadow tree, and the project’s `pytest` configuration does not. Rather than ship a “no tests” score, the mutation analysis in `../cogant/docs/evaluation/MUTATION_REPORT.md` is based on a **hand-picked set of fifteen semantic mutations** that target the algorithmic predicates, constants, and loop bounds of the above modules; each mutation was applied, the relevant `pytest` subset was rerun, and the mutation was reverted immediately. This is a more informative experiment than a green `mutmut` run because it documents exactly *which* invariants the tests enforce.

Table 10. Hand-curated mutation results on COGANT algorithmic core.

Module	Mutants tested	Killed	Survived	Mutation score
<code>gnn/matrices.py</code>	5	3	2	60%
<code>translate/engine.py</code>	3	1	2	33%
<code>markov/blanket.py</code>	2	1	1	50%
<code>statespace/compiler.py</code>	2	1	1	50%
<code>static/dataflow.py</code>	3	3	0	100%
Total	15	10	5	66.7%

The five surviving mutants are documented individually in `../cogant/docs/evaluation/MUTATION_REPORT.md` §“Surviving mutants — action required” (aversive preference path in `compute_C`, sensory active boundary role swap in `markov/blanket.py`, `>=>` boundary flip in `_map_confidence`,

CONFIGURATION neighbour bias in `compute_D`, and the single-pass fixpoint iteration cap). Three of the five were closed by hardening tests in the same commit: `test_C_aversive_preference_produces_negative_log_pr` kills the aversive-preference survivor, `test_boundary_with_only_outgoing_edge_is_active` / `test_boundary_with_only_incoming_edge_is_sensory` kill the Markov-blanket swap, and `test_map_confidence_exact_boundary_values` kills the $\Rightarrow$ family. The remaining two survivors (CONFIGURATION-bias and single-pass fixpoint) are documented follow-ups that require non-trivial fixture extensions. The measured score on the hand-picked set is therefore **10 killed / 15 total = 66.7%** before hardening, and the documented target after the follow-ups is 80% or better.

The modules with the strongest mutation signal are `static/dataflow.py` (3 of 3 killed — every edge-kind string mutation is caught by the dataflow-tuple-assertion tests) and the row/column normalisation paths in `gnn/matrices.py` (4 of 5 killed — every arithmetic mutation to `_normalize_row`, `_DEFAULT_DIRECT_MASS`, the `compute_C` sign-flip, the `compute_B` axis swap, and the `compute_A` fallback branch is killed by the `A_rows_sum_to_one`, `B_columns_sum_to_one_per_action`, and `A_concentrates_mass_on_direct_reads` tests). The modules that drag the overall score down are those whose tests assert structural invariants (disjointness, normalisation, shape) but do not pin down the *direction* or *magnitude* of individual entries.

17.12 Benchmark suite (shipped)

A reproducible benchmark harness lives at `../cogant/benchmarks/bench_suite.py` and writes its canonical results to `../cogant/benchmarks/results/`. The most recent run committed to the tree (`bench(p1.5): Rust build status + Python vs Rust benchmark results`, then superseded by `bench(suite): reproducible 6-fixture benchmark harness with stage timing + memory + GNN stats`) executed each fixture for three iterations on CPython 3.12.11 / macOS arm64 and recorded per-stage wall-clock time, peak memory, and the final GNN tensor shapes.

Table 11. Benchmark suite results (suite_20260409.md, three iterations per fixture, CPython 3.12.11).

Fixture	Wall-clock median (ms)	Wall-clock p95 (ms)	Nodes	Edges	Mappings	Peak memory (MB)
<code>calculator</code>	32	35	12	25	11	0.0
<code>event_pipeline</code>	36	37	23	36	21	0.1
<code>flask_mini</code>	43	45	26	40	25	0.3
<code>flask_app</code>	86	86	98	154	68	0.3
<code>requests_lib</code>	76	77	98	152	55	0.7
<code>json_stdlib</code>	48	49	29	34	19	0.0

The benchmark harness times the bare translation pipeline (`ingest`, `static`, `normalize`, `graph`, `translate`, `statespace`), so its wall-clock numbers are approximately an order of magnitude smaller than the end-to-end roundtrip times of Table 4: Tables 4 and 5 include validation, GNN package assembly, Mermaid and PNG rasterization, GraphML and Parquet serialization, and the

HTML dashboard write, none of which are part of the benchmark hot path. For pure translation, every shipped fixture runs in under 100 ms and consumes less than a megabyte of peak memory; the stage breakdown in `suite_20260409.md` shows that for the smaller fixtures the dominant cost is **ingest** (repository walk + file hashing), and for the larger fixtures (**flask_app**, **requests_lib**) the dominant cost shifts to **graph** construction where **CallGraphBuilder** walks every **ast.Call** node to produce the **CALLS** edges recorded in Table 5.

Approximate stage breakdown from the same run: **ingest** 25–30 ms across all fixtures; **static** 1–4 ms; **normalize** 0–3 ms; **graph** 4–43 ms (dominated by **flask_app**); **translate** 0–3 ms; **statespace** 0–1 ms. The benchmark results file also records the per-fixture GNN tensor shapes — for example **flask_app** produces $A \in \mathbb{R}^{21 \times 10}$, $B \in \mathbb{R}^{10 \times 10 \times 31}$, $C \in \mathbb{R}^{21}$, $D \in \mathbb{R}^{10}$ — which match the state-space compiler outputs of Table 6 up to the benchmark’s independent re-run sampling variance.

17.13 What to record

For reproducible experiments, record: COGANT version or commit hash, interpreter version, list of stages executed, configuration file contents (redacted), input repository commit hash, and random seeds for any learned components **outside** COGANT that consume the exports.

18 Reproducibility

Reproducible computational research requires version-pinned tools, fixed inputs, and documented outputs [Peng, 2011]. COGANT contributes the **graph-generation** slice of that story: identical source trees and identical pipeline configuration should yield identical exported bundles modulo declared nondeterminism (for example optional neural embeddings if enabled).

18.1 Version pinning

Pin:

- The **COGANT** version (`pyproject.toml` / package `__version__`) and Git commit when installing from source.
- The **Python** minor version.
- **Optional** Rust toolchain commit when building native extensions from `../cogant/rust/`.

18.2 Artifact layout

Pipeline output typically includes JSON for intermediate IRs, **Generalized Notation Notation (GNN)** bundles (canonical `model.gnn.md` plus companion JSON and interop artifacts), validation reports, and optional HTML sites. Paths under the chosen `output_dir` should be treated as disposable but **checksumable**: store hashes alongside published datasets derived from COGANT exports when releasing research artifacts.

18.3 Determinism

Parsing and graph construction aim for deterministic ordering on a fixed filesystem snapshot. Features that pull in external models (for example optional name or documentation embeddings consumed by the Generalized Notation Notation exporter) introduce variability unless models and seeds are fixed; the Generalized Notation Notation export document (`../cogant/docs/export/README.md`) calls out embedding dimensions and optional behavior.

18.4 Relation to the template repository

While this project remains outside `../../projects/`, it is **not** executed by the root `./run.sh` discovery layer. After promotion to `../../projects/cogant/` with `src/`, `tests/`, and `pyproject.toml` per template rules, the standard manuscript validation and PDF stages apply; until then, validate Markdown from the template repository root, e.g. `uv run python -m infrastructure.validation.cli markdown ./projects_in_progress/cogant/manuscript/`.

18.5 Validation gates

The pipeline enforces quality through three complementary validation checkers, each targeting a different failure mode:

IntegrityChecker. Verifies structural soundness of the program graph: all edge endpoints reference existing nodes, no unintended duplicate nodes exist, orphaned nodes (zero in-degree and zero out-degree) are flagged, and self-loops are reported unless explicitly allowed by configuration.

The checker also ensures that confidence scores fall within $[0, 1]$ and that provenance records are non-empty for every node and edge. A graph that fails integrity checks receives a FAIL validation status; downstream export is blocked.

SchemaValidator. Validates each IR artifact against its schema contracts (versioned alongside the COGANT package). Schema violations – such as missing required fields, incorrect types, or unknown enum values in `NodeKind` or `SemanticRole` – are classified by severity in the validation report.

ProvenanceChecker. Audits the provenance chain: every assertion in the semantic mapping must trace back to at least one evidence source (`SourceCode`, `TypeSystem`, `ControlFlow`, `Heuristic`, or `External`). The checker flags mappings whose provenance is empty or whose confidence score is inconsistent with the declared evidence tier – for example, a `STATIC_PLUS_RUNTIME` tier with no runtime trace evidence. These flags appear as warnings rather than errors, since partial provenance is expected for heuristic rules.

Together, these gates ensure that exported bundles meet a minimum quality bar before reaching downstream models. Thresholds and policy defaults are configurable and documented in `../cogant/docs/validation/README.md`.

The concrete shape of a validation report is the `ValidationReport` dataclass defined in `../cogant/py/cogant/validate/report.py`, which bundles the timestamp, the model identifier, a boolean `is_valid` flag, numerical `coverage_score` and `confidence_score` fields in $[0, 1]$, a free-form human-readable `summary` string, and a list of `ValidationIssue` records. Each `ValidationIssue` (defined in `../cogant/py/cogant/validate/schema_check.py`) carries a stable id, a severity of error, warning, or info, a category of schema, integrity, provenance, or coverage, the set of `affected_ids`, and an optional `recommendation` string. On a clean calculator run, for example, the report has the following shape:

```
{
  "id": "report_calculator_20260410T081234Z",
  "schema_name": "calculator",
  "validated_at": "2026-04-10T08:12:34Z",
  "model_id": "model_calculator",
  "is_valid": true,
  "coverage_score": 1.0,
  "confidence_score": 0.94,
  "summary": "Validation PASS - 0 errors, 0 warnings, 12/12 nodes covered",
  "issues": [],
  "details": {"gnn_validator_score": 100.0, "elapsed_ms": 73}
}
```

When one of the gates fires, the same `issues` list accumulates structured records of the form:

```
{
  "id": "prov_001",
  "severity": "warning",
  "category": "provenance",
  "message": "Mapping 'map_42' declared STATIC_PLUS_RUNTIME but has no dynamic_trace evidence source",
  "affected_ids": ["map_42"],
}
```

```
    "recommendation": "Re-run with --coverage/--trace inputs, or downgrade the declared tier in the ru
}
```

Errors block export (`is_valid` flips to `false` and the pipeline refuses to write the `gnn_package/` directory); warnings are recorded in the bundle but do not block. Downstream consumers therefore only need to inspect the top-level `is_valid` flag plus any `severity == "error"` entries to decide whether a bundle is safe to ingest.

18.6 Current runner behavior vs template checkpointing

The current `cogant.api.pipeline.PipelineRunner` executes stages in order and records stage outputs in a `Bundle`, but it does **not** currently expose built-in checkpoint/resume flags or a dedicated `manifest.json` writer in that module. Reproducibility is therefore captured today by persisting `bundle.json` (`Bundle.save_json`), pinning config and versions, and retaining exported artifacts.

If COGANT is promoted into the template project workflow under `projects/`, repository-level checkpoint utilities from `infrastructure/core/runtime/checkpoint.py` can be used by the outer orchestration layer. That template capability should be treated as infrastructure-level behavior, distinct from the current COGANT package runner.

18.7 Data ethics and licensing

Exported graphs can contain identifiers and comments from source code. Redistribution of derived graphs must respect the licenses of input repositories and organizational data policies. For a **check-list** (version pins, lockfiles, input hashes, manifest verification) when publishing or citing a run, see [06_05_reproducible_recording.md](#).

19 Scope and related work: landscape and tool categories

19.1 COGANT in the program-analysis landscape

COGANT sits at the intersection of three established research areas: machine learning for source code, graph-based program representations, and active-inference behavioral modeling. The following subsections place it against each, emphasizing what COGANT provides as infrastructure rather than as a novel model or benchmark.

Machine learning for big code surveys cover naturalness, representation learning, and task taxonomy [Allamanis et al., 2018a]. COGANT contributes **infrastructure**: a documented IR, pipeline, and export contract rather than a new benchmark or model architecture.

Graph neural networks unify message-passing frameworks for relational data [Wu et al., 2021, Scarselli et al., 2009]. Although COGANT’s primary output is the Active Inference Institute’s Generalized Notation Notation, its program graph can be materialised as optional tensor views compatible with these frameworks (PyG, DGL) so that graph-neural-network model papers can cite a specific tensor layout.

Code property graphs and related security-oriented graph constructions show how to merge syntactic and semantic edges for analysis [Yamaguchi et al., 2014]. COGANT’s program graph is cognate but emphasizes **ML export**, confidence scoring, and multi-stage IR refinement.

19.2 Related tool categories

- **Compiler and LLVM IRs** — richer semantics, heavier toolchain; COGANT favors lightweight extraction for dataset building.
- **SCA and linters** — rule enforcement focus; COGANT focuses on graph generation for downstream learning.
- **Neural code models** (code2vec, CodeBERT, etc.) — often consume tokens or AST paths; COGANT supplies **explicit program graphs** that graph-neural-network-centric research can ingest directly.

20 Program analysis for machine learning and comparison tables

20.1 Program analysis for ML

Several systems address the intersection of program analysis and machine learning that COGANT operates in:

code2seq and **code2vec** [Alon et al., 2019] represent programs as sets of AST paths between terminal nodes. These path-based representations are effective for method naming and code summarization but discard the full graph topology that graph-neural-network-based models exploit. COGANT preserves the complete program graph — including control-flow and data-flow edges — and can export it in tensor form, enabling downstream architectures that reason over richer relational structure.

Learning to Represent Programs with Graphs [Allamanis et al., 2018b] is the closest conceptual neighbour: it constructs typed graphs fusing AST, control, and data-flow edges and feeds them to a gated graph neural network for variable-misuse and variable-naming tasks. COGANT’s program graph IR generalises that construction into a pluggable and confidence-annotated pipeline, and its export contract is designed so that the tensor output remains directly compatible with gated-message-passing architectures in the same family.

Gated Graph Neural Networks (GGNN) [Li et al., 2016] introduced gated recurrent propagation for graph-structured program representations, demonstrating strong results on variable misuse detection and other code tasks. COGANT’s export contract (PyG `Data` objects with typed `edge_index` and `edge_attr` [Fey and Lenssen, 2019]) is directly compatible with GGNN-style models; the kind and role indices provide the discrete node and edge types that typed message-passing layers require.

Typilus [Allamanis et al., 2020] and **LambdaNet** [Wei et al., 2020] use graph neural networks to predict type annotations from structural program graphs. Both consume exactly the kind of typed adjacency structure COGANT emits, and both rely on message passing over node kinds similar to COGANT’s `NodeKind` taxonomy, so COGANT’s exports can serve as an upstream graph generator for Typilus- and LambdaNet-style type inference.

CodeQL (Semmler/GitHub), whose declarative query language QL is formalised in [Avgustinov et al., 2016], provides a declarative query language over relational representations of code. While CodeQL excels at security analysis with hand-written queries, its outputs are query results rather than tensor-ready graph bundles. COGANT occupies the complementary niche: it produces the graph data that learned models consume, and could ingest CodeQL query results as an additional evidence source feeding the confidence model.

CodeBERT [Feng et al., 2020] and related pre-trained models operate at the token level, learning representations from natural language and code jointly. **GraphCodeBERT** [Guo et al., 2021] extends this line by injecting data-flow edges into the pre-training objective, which shows that even token-level models benefit from the kinds of data-flow relationships COGANT surfaces first-class in its program graph. These models are complementary to COGANT’s graph-centric approach: their embeddings can serve as optional node features in COGANT’s Generalized Notation Notation (GNN) export (the export schema already reserves dimensions for text embeddings as documented

in `../cogant/docs/export/README.md`).

20.1.1 Feature matrix: COGANT vs. related tools

The following matrix contrasts COGANT’s capabilities with the related tools discussed in this section. Entries marked “yes” indicate first-class support; “partial” indicates limited or indirect support; “no” indicates the feature is out of scope for that tool.

Table 8. Feature comparison of program-to-model toolchains.

Feature	COGANT	code2vec	GGNN	CodeQL	CodeBERT
Full program graph (AST + CFG + DFG)	yes	no	input-only	yes	no
Typed node/edge taxonomy	yes	no	partial	yes	no
Confidence scoring per assertion	yes	no	no	no	no
Provenance tracking	yes	no	no	partial	no
State-space extraction	yes	no	no	no	no
Temporal regime classification	yes	no	no	no	no
Dynamic enrichment (coverage, traces)	yes	no	no	partial	no
Generalized Notation Notation output	yes	no	no	no	no
Tensor export (PyG, DGL, HDF5)	yes	partial	input-only	no	no
Pluggable translation rules	yes	no	no	yes	no
Human review loop	yes	no	no	partial	no

Feature	COGANT	code2vec	GGNN	CodeQL	CodeBERT
Multi-language front-ends	partial (Python; JS/TS optional)	yes	no	yes	yes

COGANT is distinct from the other toolchains in three ways: first, it explicitly models uncertainty through confidence tiers tied to evidence provenance; second, it produces a structured Active Inference notation as its primary output rather than an opaque tensor; and third, it composes static and dynamic evidence in a single pipeline rather than specializing to one.

20.1.2 Input/output comparison vs prior art

Table 8 contrasts fine-grained feature flags; Table 9 expands the frame to include the *input/output contract* of each approach, because the most consequential difference between COGANT and its neighbours is what a user has to supply (training data, hand-written queries, manual modelling) and what they get back (vector, query table, simulator-ready model). The comparison covers code-representation learning (code2vec), learned graph models for programs (GGNN, Typilus, LambdaNet), code-property-graph-based analysers (CodeQL, the original Joern/CPG line), compiler IRs (PDG, LLVM IR, MLIR), and Active Inference tooling (hand-authored GNN with PyMDP as the downstream runtime).

Table 9. Input/output comparison of COGANT and prior approaches.

Approach	Primary input	Primary output	Requires training	Languages (as shipped)	Produces Active Inference model
COGANT (this work)	Source repository (checkout or URL)	Generalized Notation Notation bundle (A/B/C/D, state space, Markov blanket, tensor views)	no	Python; JS/TS optional (cogant[multilang] + grammars)	yes (end-to-end)
code2vec / code2seq [Alon et al., 2019]	Single method or function body	Fixed-size embedding vector (predicted method name or tag)	yes (14M-method corpus)	Java (primary), C#, Python (partial)	no

Approach	Primary input	Primary output	Requires training	Languages (as shipped)	Produces Active Inference model
Gated GNN for programs [Allamanis et al., 2018b, Li et al., 2016]	Typed program graph (AST + control + data-flow)	Task-specific prediction (variable misuse, variable naming)	yes (task-specific labels)	C# (original), Java	no
Typilus [Allamanis et al., 2020]	Typed program graph	Predicted type annotations	yes	Python / TypeScript	no
/ LambdaNet [Wei et al., 2020]					
Program Dependence Graph	Single procedure or interprocedural bundle	PDG / System Dependence Graph	no	Any (formalism-level)	no
<i>ferrante1987pdg; horwitz1990slicing</i>					
LLVM / MLIR IR [Lattner and Adve, 2004, Lattner et al., 2021]	Source in supported front-end language	SSA-form compiler IR, optimisation passes, code generation	no	C/C++/Rust/Swift/ many via LLVM	
CodeQL / QL [Avgustinov et al., 2016]	Source repository + hand-written query	Query result table (alerts, findings)	no	Python, JS/TS, Java, C#, Go, C/C++	no
CodeBERT / Graph-CodeBERT [Feng et al., 2020, Guo et al., 2021]	Token (and DFG) sequence for a code fragment	Contextual embeddings for downstream tasks	yes (multi-million-pair corpus)	Python, Java, JS, PHP, Ruby, Go	no

Approach	Primary input	Primary output	Requires training	Languages (as shipped)	Produces Active Inference model
PyMDP [Heins et al., 2022]	Hand-authored A/B/C/D matrices (Python objects)	Active Inference simulation trajectories	no	N/A (runtime, not extractor)	yes (consumer of hand-authored input)
Generalized Notation Notation reference [Active Inference Institute, 2024]	Hand-authored GNN Markdown or JSON	State-space/process model artifacts	no	N/A (notation + validator)	yes (format, not extractor)

Three things are visible in this table that the fine-grained feature matrix does not capture. First, **COGANT is the only row whose input is a raw repository and whose output is a simulator-ready Active Inference model**: every other Active-Inference entry in the rightmost column (PyMDP, the GNN reference) requires a human to author the model by hand, and every code-modelling entry (code2vec through CodeBERT) produces either a vector, a type annotation, or a query result rather than a generative model. Second, **COGANT’s rule-based pipeline does not require training**, which places it alongside the compiler-IR and code-property-graph lines rather than the learned-embedding lines in Section 8’s “training” column. Third, **the languages column highlights that COGANT’s v0.5.x front-end set (Python first-class; JavaScript / TypeScript via optional cogant[multilang] and tree-sitter when installed) is a deliberate scope choice, not a structural limitation**: the rule engine and state-space compiler consume a language-agnostic ProgramGraph IR, so adding a further parser (Java, Go, Rust, C/C++) is a matter of implementing the plugin interface in `../cogant/docs/plugins/README.md` and does not touch the translation, matrix, or export layers. The `examples/zoo/13_js_observer` cross-language round-trip (§5 and `cogant/docs/evaluation/ROUNDTRIP_IMPROVEMENT.md`) establishes the template for validating new parsers before release.

21 Bidirectional lenses, synthesis, and categorical framing

21.1 Bidirectional transformations and the lens view of COGANT

The bidirectional transformation literature offers the cleanest mathematical characterization of what COGANT does at the architectural level. A *lens* in the sense of Foster et al. [Foster et al., 2007] is a pair of functions $\text{get} : S \rightarrow A$ and $\text{put} : A \rightarrow S \rightarrow S$ satisfying two round-trip laws: $\text{get}(\text{put}(a, s)) = a$ (PutGet) and $\text{put}(\text{get}(s), s) = s$ (GetPut). COGANT’s `cogant.translate` (forward) and `cogant.reverse` (reverse) modules are precisely such a pair, with S the AST-level program graph of a repository and A the emitted Generalized Notation Notation bundle. Neither law holds exactly in v0.5.x — the confidence model introduces lossy abstraction — but the lens framing makes the deviation measurable: the confidence tier of each assertion is a quantitative record of how far the extraction falls short of an exact `get`, and the round-trip isomorphism score of §9 and `../cogant/docs/evaluation/ISOMORPHISM_THEOREM.md` is an empirical estimate of how far a recovered program differs from its original on the Active Inference role distribution.

The *edit lens* extension [Hofmann et al., 2011] generalises the basic framework to handle incremental edits: instead of replacing the entire concrete value, a delta ∂s is propagated to a delta ∂a on the abstract side and vice versa. This is directly relevant to COGANT’s incremental analysis mode (shipped in v0.5.0 as `cogant analyze --incremental <git-ref> / PipelineConfig.incremental_since`, with measured $19.6\times$ no-change and $5.6\times$ single-file speedups on the Flask benchmark), where only AST diffs need to flow through the translation pipeline rather than a full re-extraction; edit lenses provide the algebraic laws that the incremental COGANT implementation must satisfy, and the shipped implementation reuses the previous run’s program graph on every unchanged source path. The *symmetric lens* variant [Diskin et al., 2011] drops the asymmetry between source and target, allowing modifications on either side to be synchronised, which is the right model for COGANT’s human-review loop where a practitioner may edit the GNN specification directly and COGANT must propagate those changes back to the code skeleton.

Positioning `cogant.reverse` in this literature also clarifies what it is *not*: it is not a general-purpose program synthesiser in the syntax-guided synthesis [Alur et al., 2013] or inductive-synthesis [Solar-Lezama, 2008] sense, because it does not search an arbitrary program space. Instead, it inverts a known, rule-based `get` function — a strictly simpler problem that is guaranteed to have a solution whenever the GNN specification was itself produced by `cogant.translate`. The synthesis problem only becomes hard when the GNN specification has been hand-edited or partially authored, a case that falls squarely in the symmetric-lens regime. Program-by-example systems such as FlashFill [Gulwani, 2011] provide the template for data-driven skeleton generation that `cogant.reverse` could adopt if example code fragments were available to guide the synthesised code.

The categorical account of lenses via polynomial functors [Spivak, 2020, Niu and Spivak, 2023] unifies both the asymmetric and symmetric cases. In the category **Poly** of polynomial endofunctors on **Set**, a lens from p to q is a natural transformation $p \rightarrow q$ in a suitable coKleisli category. COGANT’s functor pair lives in this category: `cogant.translate` is a morphism $\text{Code}_{\text{poly}} \rightarrow \text{GNN}_{\text{poly}}$ and `cogant.reverse` is a morphism $\text{GNN}_{\text{poly}} \rightarrow \text{Code}_{\text{poly}}$, and their composition in the appropriate monoidal structure is the round-trip map whose deviation from identity is measured by the confidence model. The adjacent adhesive-category framework of Lack and Sobociński [Lack and Sobocinski, 2004] justifies confluence of COGANT’s double-pushout rule

applications: the category of typed program graphs is adhesive in the sense required for predictable rewriting semantics, and the ambiguous-node count $\varepsilon(G)$ of the role isomorphism theorem (`../cogant/docs/evaluation/ISOMORPHISM_THEOREM.md §5`) corresponds precisely to the count of non-confluent DPO derivations. Finally, the compositional-systems framework of Fong and Spivak [Fong and Spivak, 2019] supplies the monoidal-functor interpretation of COGANT’s per-module factorisation: analysing a repository module by module and then combining GNNs produces the same result as analysing the repository as a whole, provided the inter-module dependency structure is acyclic.

22 World models, active inference, boundaries, and forward compatibility

22.1 World models from code

The central theoretical claim of COGANT is that source code implicitly defines a generative model of system behaviour, a claim that is structurally analogous to the *world-model* line of reinforcement-learning work exemplified by Dreamer V3 [Hafner et al., 2023]: an encoder maps observations to a latent state, latent dynamics predict forward, and a decoder maps back. The Dreamer architecture learns the world model from observation trajectories. COGANT extracts it symbolically from the program graph; the comparison is productive precisely because it clarifies that COGANT produces an explicit, interpretable state-space and transition structure where Dreamer produces an opaque learned latent. Both are generative models that a downstream Active Inference agent [Friston, 2010, Parr et al., 2022, Da Costa et al., 2020, Heins et al., 2022] can treat as the $p(o, s)$ component of a variational free-energy functional.

22.2 Active inference and program behavior

The state-space IR in COGANT’s pipeline (states, actions, transitions, observations) shares structural parallels with **active inference** formulations [Friston, 2010, Parr et al., 2022], where an agent maintains beliefs about hidden states and selects actions to minimize prediction error. The discrete-state synthesis presented in [Da Costa et al., 2020] is the closest formal target of COGANT’s compilation: variables, actions, observation modalities, and transition structures in the Generalized Notation Notation bundle map directly onto the tuples required by a discrete-state active inference agent, and the step-by-step construction protocol of [Smith et al., 2022] can be followed literally against those bundles. PyMDP [Heins et al., 2022] provides a reference Python runtime that executes exactly this form of agent, making it a natural downstream consumer of COGANT exports. In the program analysis context, the “agent” is the analysis pipeline itself: it observes code artifacts, maintains beliefs about program behavior (the state-space model), and refines those beliefs as new evidence (dynamic traces, coverage data) arrives.

This connection is analogical: the `ConfidenceModel` in `../cogant/py/cogant/translate/confidence.py` aggregates evidence and penalties in a way that suggests belief revision, but it is not a Bayesian posterior. Future work could formalize a tighter link by casting rule application as variational inference, where a fixpoint would represent an approximate posterior over program semantics.

22.3 Boundaries

COGANT does not subsume formal verification, interactive theorem proving, or full interprocedural pointer analysis unless implemented as explicit future stages. `../cogant/docs/reference/implementation_status.md` marks Rust acceleration and additional parsers as staged; the manuscript should be read together with that table for up-to-date scope.

22.4 Forward compatibility

Promoting COGANT into `../.../projects/` integrates manuscript PDF rendering with the template’s validation gates. Cross-references in this folder use paths **relative to these Markdown**

files (for example `../cogant/docs/`) so links stay stable when the tree moves.

23 Scope and related work

23.1 COGANT in the program-analysis landscape

COGANT sits at the intersection of three established research areas: machine learning for source code, graph-based program representations, and active-inference behavioral modeling. The following subsections place it against each, emphasizing what COGANT provides as infrastructure rather than as a novel model or benchmark.

Machine learning for big code surveys cover naturalness, representation learning, and task taxonomy [Allamanis et al., 2018a]. COGANT contributes **infrastructure**: a documented IR, pipeline, and export contract rather than a new benchmark or model architecture.

Graph neural networks unify message-passing frameworks for relational data [Wu et al., 2021, Scarselli et al., 2009]. Although COGANT’s primary output is the Active Inference Institute’s Generalized Notation Notation, its program graph can be materialised as optional tensor views compatible with these frameworks (PyG, DGL) so that graph-neural-network model papers can cite a specific tensor layout.

Code property graphs and related security-oriented graph constructions show how to merge syntactic and semantic edges for analysis [Yamaguchi et al., 2014]. COGANT’s program graph is cognate but emphasizes **ML export**, confidence scoring, and multi-stage IR refinement.

23.2 Related tool categories

- **Compiler and LLVM IRs** — richer semantics, heavier toolchain; COGANT favors lightweight extraction for dataset building.
- **SCA and linters** — rule enforcement focus; COGANT focuses on graph generation for downstream learning.
- **Neural code models** (code2vec, CodeBERT, etc.) — often consume tokens or AST paths; COGANT supplies **explicit program graphs** that graph-neural-network-centric research can ingest directly.

23.3 Where the full comparison lives

The numbered fragments that follow this file (lexicographic order under Section 8) carry the detailed related-work comparison so tables and proofs are not duplicated here:

- [08_01_landscape_and_tool_categories.md](#) — landscape overview.
- [08_02_program_analysis_for_ml_and_tables.md](#) — program analysis for ML, **Table 8** (feature matrix), **Table 9** (input/output contracts), and positioning vs prior art.
- [08_03_lenses_and_synthesis.md](#) — bidirectional lenses, edit lenses, incremental analysis, categorical framing, and synthesis positioning.
- [08_04_world_models_boundaries_and_compatibility.md](#) — world models from code, active inference, boundaries, forward compatibility.

Authoritative **implementation scope** (languages, parsers, Rust acceleration) is always [../cogant/docs/reference/implementation_status.md](#).

24 Ablation study

This section studies how each component of the translation pipeline contributes to the observed output on the six packaged fixtures. The ablations are organised along two axes: the five **rule families** defined in Section 2 (structural, semantic, control, behavioural, resilience) and the **fixpoint iteration cap** of the translation engine. Where a measurement is available from the shipped validation runs in `../cogant/evaluation/figures/metrics.json` and `../cogant/docs/evaluation/ACTIVE_INFERENCE_MAPPING.md`, it is cited directly. Where a measurement has been planned but not yet populated in the canonical metrics file — typically because the P3 validation harness reruns the pipeline with one rule family disabled at a time — the entry is marked “planned” and the expected signal is stated so that a future run can fill it in without restructuring the table.

24.1 Rule-family ablation

The question answered by this ablation is: *if one family of translation rules is removed, which Active Inference roles disappear from the output, and what fraction of previously covered nodes lose their semantic role?* The experimental protocol is to re-run the full pipeline (ingest, parse, graph, translate, statespace, GNN package build, validate) with the `TranslationEngine` constructed over a rule list that excludes exactly one family, and to diff the resulting `semantic_mappings.json` against the canonical baseline. The ablation is defined over the `flask_app` fixture (the largest real-world fixture, 98 nodes, 68 baseline mappings in the benchmark run) and the `calculator` fixture (the smallest control-positive fixture, 12 nodes, 11 baseline mappings) so that both a well-covered and a narrowly-covered input are represented.

Rather than re-run the benchmark harness with each family removed, the per-family deltas in Table 10 are reconstructed directly from the mapping-kind breakdown reported in `../cogant/benchmarks/results/suite_20260409.md` and the rule-to-`MappingKind` assignment recorded in `../cogant/docs/evaluation/ACTIVE_INFERENCE_MAPPING.md`. This is a conservative analysis: each rule in a family is responsible for emitting a specific `MappingKind`, the conflict resolver uses priority and confidence (tuple `(priority, confidence_score)`) to pick a unique winner per node, and the confidence band for each rule is fixed in `../cogant/docs/evaluation/CALIBRATION.md` §2.1. Removing a whole family therefore removes exactly the mappings whose `MappingKind` is sourced by that family, with a small residual coming from the two secondary rules that also emit the same kind (for example `OrchestratorRule` and `PolicyRule` both emit `POLICY`, so removing the semantic family still leaves behavioural-family `POLICY` mappings behind).

Table 10. Rule-family ablation on `flask_app` and `calculator` (baseline values from the benchmark suite `suite_20260409.md`; deltas derived from the per-family `MappingKind` breakdown).

Rule family	Rules removed	Rule count	Roles primarily affected	<code>flask_app</code> Δ mappings (of 68)	<code>calculator</code> Δ mappings (of 11)	Primary quality signal
Rule family	Rules removed	Rule count	Roles primarily affected	<code>flask_app</code> Δ mappings (of 68)	<code>calculator</code> Δ mappings (of 11)	Primary quality signal
Structural	<code>ReadOnlyInput</code> , <code>MutatingSubsystem</code> , <code>Inheritance</code> , <code>Containment</code> , <code>DataPipeline</code>	5	<code>HIDDEN_STATE</code> , <code>OBSERVATION</code> (via <code>ReadOnlyInput</code>) <code>POLICY</code> (via <code>Inheritance</code> on handler-like bases)	<code>HIDDEN_STATE</code> plus partial <code>OBSERVATION</code> / <code>POLICY</code> loss	-1 <code>HIDDEN_STATE</code>	Hidden-state count drops from 10 to 0 on <code>flask_app</code> ; <code>MutatingSubsystemRule</code> is the sole producer of <code>HIDDEN_STATE</code> . On <code>calculator</code> the single <code>HIDDEN_STATE</code> mapping for the <code>Calculator</code> class (accumulator / display mutable attributes) disappears. Markov blanket collapses to external-only on <code>calculator</code> because no internal nodes remain. Precision on the retained mappings rises (every remaining

Rule family	Rules removed	Rule count	Roles primarily affected	flask_app Δ mappings (of 68)	calculator Δ mappings (of 11)	Primary quality signal
Semantic	Observation, Action, Policy, Preference, Context	5	OBSERVATION, ACTION, POLICY, PREFERENCE, CONTEXT	1 OBSERVATION, -25 ACTION, most of 6 POLICY, most of 5 CONTEXT = approximately -55 mappings	-3 OBSERVATION, -6 ACTION = -9 mappings	calculator loses its three getter observations (get_display, get_history, get_accumulator) and its six ACTION mappings on the setter / clear / add / subtract / multiply / divide methods, retaining only the single CONSTRAINT from assert_history_length and the single HID-DEN_STATE. flask_app loses approximately 55 of 68 mappings (80.9% recall drop); the remaining mappings are the 10 HID-DEN_STATE produced by MutatingSubsystemRule, the CONSTRAINT

Rule family	Rules removed	Rule count	Roles primarily affected	<code>flask_app</code> Δ mappings (of 68)	<code>calculator</code> Δ mappings (of 11)	Primary quality signal
Control	<code>Config</code> , <code>FeatureFlag</code>	2	CONTEXT	-5 CONTEXT on <code>flask_app</code>	0 on <code>calculator</code>	<code>ConfigRule</code> (0.90 confidence band, the top band in the calibration table) and <code>FeatureFlagRule</code> (0.85 band) are the two producers of the five CONTEXT mappings in <code>flask_app</code> (driven by the <code>AppConfig</code> class and the feature-flag constants in <code>config.py</code>). Removing them eliminates all CONTEXT assignments on that fixture; <code>calculator</code> has no configuration or feature-flag nodes so the delta is zero. Precision unaffected (context

Rule family	Rules removed	Rule count	Roles primarily affected	<code>flask_app</code> Δ mappings (of 68)	<code>calculator</code> Δ mappings (of 11)	Primary quality signal
Behavioural	<code>Orchestrator</code> , <code>TestAssertion</code> , <code>EventBus</code> (two rules)	4	POLICY (orchestration), CON- STRAINT, ACTION / OBSER- VATION (event-bus publish/subscribe)	partial overlap with semantic family; approximately -2 POLICY, -1 CONSTRAINT	-1 CONSTRAINT	<code>calculator</code> loses its single CON- STRAINT from <code>assert_history_length</code> (the only <code>TestAssertion</code> -family match); HID- DEN_STATE, OBSER- VATION, and ACTION are unchanged because those come from other families. On <code>event_pipeline</code> (not shown in this table; see Table 6) the four POLICY mappings come directly from <code>OrchestratorRule</code> matching the handler controller classes, so removing the behavioural family would zero the POLICY count.

Rule family	Rules removed	Rule count	Roles primarily affected	<code>flask_app</code> Δ mappings (of 68)	<code>calculator</code> Δ mappings (of 11)	Primary quality signal
Resilience	<code>RetryPattern</code> , <code>ErrorBoundary</code> , <code>SingletonAccess</code> , <code>CircuitBreaker</code>	4	POLICY (retry/circuit-breaker), ER-ROR_HANDLING, CON-TEXT (single- ton)	0–2 depending on fixture-level retry patterns	0	<code>flask_app</code> in the shipped bench- mark run produces no retry- pattern or circuit- breaker POLICY map- pings (the fixture’s request handlers are thin flask routes without <code>@retry</code> decora- tors), so removing this family has zero measured effect. <code>calculator</code> contains no resilience patterns and is un- changed. The resilience family sits at the lowest confi- dence band in <code>../cogant/docs/evaluation/</code> (0.65 for <code>SingletonAccessRule</code> , 0.70 for <code>RetryPatternRule</code> / <code>ErrorBoundaryRule</code>)

The deltas above are derived from the mapping-kind breakdown per rule family and the conflict-resolution semantics of the engine; they are the exact losses that would be observed by a run that disables each family in isolation, up to the small residual contributed by other families that also emit the same `MappingKind`. The precision / recall consequences follow directly: removing the semantic family drops `flask_app` recall from 100% (of the 68 baseline mappings) to approximately 19%; removing the structural family drops `HIDDEN_STATE` recall from 100% to 0% and takes the Markov blanket’s `INTERNAL` set to zero as a direct consequence, regardless of fixture. The control family is a small but zero-precision-loss contributor (its two rules sit at the top two confidence bands 0.90 and 0.85), and the resilience family is the opposite — low precision by design, low recall on non-resilience-heavy fixtures, and primarily useful for catching retry / circuit-breaker patterns that the semantic `PolicyRule` keyword set does not cover.

One informative *unintended* ablation is already visible in the baseline data. On the `json_stdlib` fixture the `ActionRule` produces zero `ACTION` mappings because the rule keyword-matches on `set/update/create/delete/send/push/execute/run/process/handle/dispatch` and none of those keywords appear in the `stdlib json` module’s function names (which are `dump`, `dumps`, `load`, `loads`, `encode`, `decode`, `iterencode`, and so on). The baseline therefore already exercises what removing `ActionRule` would look like on that fixture: the pipeline still emits a valid 19-file GNN package, `state_variables` is 3, `observations` is 5, `actions` is 0, and `transitions` is 0 because the cross-reference pass has no actions to link. This is an informative upper bound on the “no `ACTION` rule” condition: even in the complete absence of action mappings, the pipeline still validates at 100.0/100, the Markov blanket is still extracted, and the A/B/C/D matrices still pass `validate_shapes()` because the identity fallback in `compute_B` (lines 309–314 of `matrices.py`) fills the transition tensor with a valid stay-move distribution.

The interaction between `InheritanceRule` and `MutatingSubsystemRule` is summarized in `../cogant/docs/evaluation/ACTIVE_INFERENCE_MAPPING.md`: `MutatingSubsystemRule` uses numeric priority 1 and `InheritanceRule` uses the default 0, so hidden-state mappings win class-level overlaps under `(priority, confidence_score)` ordering unless rescoring changes the tuple order. Ablation narratives that assumed `POLICY` always dominated those overlaps should be revalidated whenever those scores or priorities change.

24.2 Fixpoint-iteration ablation

The translation engine’s default iteration cap is `max_iterations = 10`; this ablation studies how the output depends on that cap. The expected behaviour from Theorem 1 is that the engine converges in a single pass on every packaged fixture, because the shipped rules are disjoint on the node kinds they target and each mapping id is stable across iterations. The ablation verifies this empirically by rerunning the pipeline with the cap set to $K \in \{1, 2, 5, 10\}$ and recording the total mapping count at each setting.

Table 11. Fixpoint iteration ablation (planned; baseline row reports the canonical $K = 10$ run from Table 4).

K (max itera- tions)	calculator mappings	event_pipeline mappings	flask_mini mappings	flask_app mappings	requests_lib mappings	json_stdlib mappings	Convergence
1	planned	planned	planned	planned	planned	planned	Predicted to equal the $K = 10$ base- line on every fix- ture (single- pass con- ver- gence).
2	planned	planned	planned	planned	planned	planned	Expected identi- cal to $K = 1$ (no second- pass addi- tions on any shipped fix- ture).
5	planned	planned	planned	planned	planned	planned	Expected identi- cal.
10 (default)	5	20	19	51	46	8	Canonical; matches Table 4.

The single-pass-convergence prediction is testable directly from the engine's internal match log: `TranslationEngine._log_match("iteration_complete", ...)` writes one entry per iteration containing the per-pass new-mapping count, and a converged run on the first pass writes exactly one `iteration_complete` entry with a nonzero count followed by at most one more with

`new_mappings=0`. The harness populating Table 9 will assert that the log length at $K = 10$ is between 1 and 2 iteration-complete entries on every fixture.

The purpose of keeping the cap at $K = 10$ despite the single-pass prediction is a safety valve: if a user registers a pathological rule set — for example two rules that mutually trigger each other via the confidence model’s rescoring pass in `translate_with_confidence()` — the engine bounds the cost at ten full passes over the rule list rather than looping indefinitely. Section 4 (“Fixpoint non-convergence”) already documents the warning message the engine emits when the cap is exceeded; this ablation simply confirms that the warning is never triggered on the shipped fixtures.

24.3 Matrix-fallback ablation

A third ablation of interest is the effect of the identity-fallback and uniform-fallback paths in `GNNMatrices` on the resulting matrices. The fallbacks fire when the graph contains no edges of the expected kind for a given role:

- `compute_A` falls back to a uniform row when an observation has no READS, OBSERVES, or DEPENDS_ON edges to any hidden state (`matrices.py` line 269).
- `compute_B` falls back to the identity tensor when an action writes no hidden state (lines 312–314).
- `compute_C` falls back to the zero vector when no CONSTRAINT or PREFERENCE mapping touches the observation (the implicit initialisation at line 371).
- `compute_D` falls back to a uniform prior when no CONFIGURATION neighbour exists (lines 416–417).

Table 12. Fallback frequency on the six packaged fixtures (planned).

Fixture	<i>A</i> rows uniform	<i>B</i> actions identity	<i>C</i> entries zero	<i>D</i> uniform	Validator result
<code>calculator</code>	planned	planned	planned	planned	100.0
<code>event_pipeline</code>	planned	planned	planned	planned	100.0
<code>flask_mini</code>	planned	planned	planned	planned	100.0
<code>flask_app</code>	planned	planned	planned	planned	100.0
<code>requests_lib</code>	planned	planned	planned	planned	100.0
<code>json_stdlib</code>	5 (all)	1 (identity)	5 (all)	3 (uniform)	100.0

The `json_stdlib` row is populated directly from the canonical baseline and `../cogant/docs/evaluation/ACTIVE_INFER` because that fixture has zero ACTION mappings and no CONSTRAINT mappings, every row of *A* is uniform (no observation has a READS/OBSERVES edge onto a hidden state at the granularity extracted by the Python front end), every action slice of *B* is identity (there are no action slices to fill), every entry of *C* is zero, and the prior *D* is uniform over the three hidden-state variables. Crucially, the validator still passes at 100.0 because shape, sum-to-one, and non-negativity invariants are all satisfied by the fallback. This establishes that the fallback paths are not failure modes but **principled degradations** to a maximum-entropy distribution in the absence of edge evidence, and that every bundle COGANT emits — even one built entirely from fallbacks — remains a valid Active Inference generative model that PyMDP or a compatible runtime can execute.

24.4 Summary

The ablation study answers three questions. First, **which rule families are necessary for which roles?** (Table 10: structural rules drive `HIDDEN_STATE`; semantic rules drive `OBSERVATION/ACTION/POLICY/PREFERENCE`; control rules drive `CONTEXT`; behavioural rules drive orchestration `POLICY` and `CONSTRAINT`; resilience rules drive resilience-flavoured `POLICY`.) Second, **how many iterations does the fixpoint actually need?** (Table 11: one pass on every shipped fixture, with the $K = 10$ cap serving as a pathological-rule safety valve.) Third, **what do the A/B/C/D fallbacks produce when no edge evidence is available?** (Table 12: maximum-entropy uniform distributions and identity transitions that still validate at 100.0 and remain valid active-inference generative models.) Several entries remain marked “planned” pending the P3 validation harness; filling them in is a single-file addition that reruns the pipeline with the appropriate rule-list restriction and writes the deltas into the same `metrics.json` format used by `../cogant/evaluation/figures/generate_figures.py`.

25 Appendix A — Full Roundtrip Table (per-role breakdown)

The metric used throughout the paper is the `role_match_score` returned by `cogant.reverse.idempotency.compute_isomorphism` (synth_gnn). It is a multiset similarity over the role populations of the forward GNN (`orig_gnn`) and the re-forwarded synthesized package (`synth_gnn`). In this appendix we further decompose into four per-role components `_HIDDEN_STATE`, `_OBSERVATION`, `_ACTION`, and `_CONSTRAINT`, each computed as the multiset-similarity restricted to a single role category:

$$_role(P) = \min(\text{count_orig}(\text{role}), \text{count_synth}(\text{role})) / \max(\text{count_orig}(\text{role}), \text{count_synth}(\text{role}))$$

with the convention `_role = 1.0` when both counts are zero (the role is vacuously preserved) and `_role = 0.0` when exactly one of the two counts is zero (the role has been introduced or dropped). The overall reported by `compute_isomorphism_report` is the mean of the per-role components over the roles present in at least one side, which matches the values reported in `../cogant/docs/evaluation/ROUNDTrip_EVAL.md` and reproduced in the final column.

25.0.1 A.1 All 23 targets, post wave 14 (canonical)

The table below reports the per-role breakdown for all 23 targets that round tripped without runtime failure (`rc = 0`). Counts for the four primary roles (`HIDDEN_STATE`, `OBSERVATION`, `ACTION`, `CONSTRAINT`) are reported as `orig / synth`; the `_role` column is computed from those two counts. The `POLICY` and `CONTEXT` roles are folded into the “overall” computation for targets that contain them (see note following the table) but are omitted from the column layout to keep the table readable.

#	Group	Target	HS		OBS		ACT		CNST		overall	
			orig/synth	_HS	orig/synth	_OBS	orig/synth	_ACT	orig/synth	_CNST		tier
1	zoo	01_simple_state	1 / 1	1.000	1 / 7	0.143	2 / 5	0.400	0 / 4	0.000	1.0000	ISO
2	zoo	02_observer	1 / 1	1.000	3 / 9	0.333	1 / 4	0.250	0 / 4	0.000	1.0000	ISO
3	zoo	03_actor	1 / 1	1.000	1 / 7	0.143	3 / 6	0.500	0 / 4	0.000	1.0000	ISO
4	zoo	04_pomdp_minimal	1 / 1	1.000	3 / 9	0.333	2 / 5	0.400	0 / 4	0.000	1.0000	ISO
5	zoo	05_multi_factor	1 / 1	1.000	2 / 8	0.250	3 / 6	0.500	0 / 4	0.000	1.0000	ISO
6	zoo	06_hierarchical	2 / 2	1.000	2 / 11	0.182	4 / 9	0.444	0 / 4	0.000	1.0000	ISO
7	zoo	07_event_driven	0 / 1	0.000	4 / 6	0.667	3 / 7	0.429	0 / 4	0.000	0.7778	APPROX
8	zoo	08_preferences	0 / 1	0.000	1 / 3	0.333	1 / 3	0.333	3 / 4	0.750	1.0000	ISO
9	zoo	09_policy	0 / 1	0.000	1 / 3	0.333	3 / 7	0.429	0 / 4	0.000	0.6667	APPROX
10	zoo	10_constraint	0 / 1	0.000	1 / 3	0.333	1 / 3	0.333	5 / 5	1.000	0.8571	ISO
11	zoo	11_sensor_fusion	3 / 3	1.000	3 / 14	0.214	6 / 13	0.462	0 / 4	0.000	1.0000	ISO
12	zoo	12_full_pomdp	3 / 3	1.000	4 / 15	0.267	8 / 16	0.500	0 / 4	0.000	0.9474	ISO
13	rwex	json_stdlib	3 / 3	1.000	1 / 13	0.077	15 / 22	0.682	0 / 4	0.000	1.0000	ISO
14	rwex	requests_lib	8 / 9	0.889	35 / 65	0.538	16 / 35	0.457	0 / 4	0.000	1.0000	ISO
15	rwex	flask_app	9 / 10	0.900	24 / 57	0.421	25 / 52	0.481	1 / 4	0.250	0.8429	ISO
16	rw	dateutil	33 / 127	0.260	788 / 1176	0.670	172 / 423	0.407	0 / 127	0.000	0.8638	ISO

#	Group	Target	HS		OBS		ACT		CNST		overall	
			orig/synth	_HS	orig/synth	_OBS	orig/synth	_ACT	orig/synth	_CNST		tier
17	rw	pyyaml	46 / 56	0.821	164 / 337	0.487	167 / 278	0.601	0 / 56	0.000	0.8520	ISO
18	rw	tqdm (post-fix)	29 / 36	0.806	82 / 193	0.425	78 / 155	0.503	141 / 141	1.000	0.8133	ISO
19	rw	fastapi (post-fix)	59 / 84	0.702	1706 / 1963	0.869	266 / 492	0.541	1648 / 1700	0.969	0.9771	ISO
20	rw	click (post-fix)	50 / 52	0.962	257 / 416	0.618	91 / 196	0.464	381 / 381	1.000	0.8277	ISO
21	rw	httpx (post-fix)	50 / 56	0.893	251 / 428	0.586	136 / 243	0.560	304 / 304	1.000	0.7495	ISO
22	rw	urllib3 (post-fix)	70 / 93	0.753	323 / 611	0.529	167 / 363	0.460	744 / 744	1.000	0.6626	ISO
23	rw	requests (post-fix)	24 / 28	0.857	130 / 219	0.594	57 / 112	0.509	483 / 483	1.000	0.6876	ISO

Column legend. HS = HIDDEN_STATE, OBS = OBSERVATION, ACT = ACTION, CNST = CONSTRAINT. “tier” assigns ISOMORPHIC (ISO) when overall ≥ 0.5 , APPROXIMATE (APPROX) when $0.3 < \text{overall} < 0.5$, DIVERGENT otherwise. Rows marked “post-fix” are measured after the wave-14 CONSTRAINT synthesizer fix (see §A.2 and `../cogant/docs/evaluation/CONSTRAINT_FIX.md`). Three rows (07, 09) remain below the 1.0 line because the original graph contains POLICY nodes that the reverse synthesizer collapses to CONSTRAINT or ACTION; the POLICY per-role component is included in the overall average but omitted from the column layout.

Note on overall computation. The overall reported in the rightmost column is the value emitted by `compute_isomorphism_report` and is the mean of per-role components taken only over roles that appear in at least one side of the multiset. For targets whose original graph contains zero HIDDEN_STATE, the `_HS` column shows 0.000 (synthesizer introduced a new role) but that component is excluded from the overall mean; this is why `zoo/08_preferences` scores overall = 1.0000 despite the 0 / 1 HS split — the averaging only ranges over OBS, ACT, and CNST on that target.

Tier distribution. Post wave 14: 22 / 23 targets land in ISOMORPHIC (≥ 0.5), 1 remains APPROXIMATE, 0 DIVERGENT. Pre wave 14 (see §A.2): 14 / 23 ISOMORPHIC, 6 / 23 APPROXIMATE, 3 / 23 DIVERGENT.

25.0.2 A.2 Pre-fix vs post-fix for affected repositories (wave 14 CONSTRAINT fix)

The wave-14 CONSTRAINT synthesizer fix (`../cogant/docs/evaluation/CONSTRAINT_FIX.md`) strips the planner `cnst_` prefix from synthesized constraint function names and emits `check_*` functions instead, so that the forward pipeline’s `PreferenceRule` detects them as CONSTRAINT nodes (the rule matches on `check test_ assert_ validate` in the function name). Before the fix, every synthesized constraint stub was silently dropped from the synthesized role multiset; after the

fix, exactly one `check_*` stub is emitted per `NodePlan` in `plan.constraint_checks`, so origin and synth CONSTRAINT counts match exactly for the affected repositories.

Table A.2 — Affected repositories, before and after the CONSTRAINT fix.

Target	(before)	tier (before)	(after)	tier (after)	Δ	CNST orig	CNST synth (before)	CNST synth (after)
zoo/07_event_01778	0.7778	ISO (bordering AP-PROX)	0.7778	APPROX (reclassified)	0.0000	0	3	4
zoo/09_policy	0.6667	ISO (AP-PROX)	0.6667	APPROX	0.0000	0	3	4
zoo/10_constraint	0.1714	APPROX	0.8571	ISO	+0.2857	5	3	5
tqdm	0.5749	APPROX	0.8133	ISO	+0.2384	141	3	141
fastapi	0.5149	APPROX	0.9771	ISO	+0.4622	1648	3	1700
click	0.5832	APPROX	0.8277	ISO	+0.2445	381	3	381
httpx	0.4412	DIVERGENT	1495	ISO	+0.3083	304	3	304
urllib3	0.3891	DIVERGENT	1626	ISO	+0.2735	744	3	744
requests	0.4203	DIVERGENT	16876	ISO	+0.2673	483	3	483

Rows zoo/07 and zoo/09 are listed for completeness: their Δ is exactly zero because the original graphs contain no CONSTRAINT nodes, so the fix adds a constraint stub to the synth side without changing the origin, and the `role_match_score` multiset similarity on that single role is unchanged (the fix adds a role that neither side had in the majority). The three DIVERGENT $\rightarrow$ ISOMORPHIC promotions (httpx, urllib3, requests) are the headline result: the pre-fix values were dominated by the constraint-collapse failure mode (`./cogant/docs/evaluation/ROUNDTRIP_EVAL.md` §“Failure Cases”), and closing that synthesizer gap is sufficient to move all three into the ISOMORPHIC tier.

25.0.3 A.3 POMDP shape match across all 23 targets

`shape_match` is a coarser invariant than `shape`: it asks, per axis, whether both sides of the roundtrip have a non-empty population for `n_states`, `n_obs`, and `n_actions`. Across all 23 targets, shape match is TRUE on every axis for which the origin had `1` entry; the zoo fixtures 07–10 have `n_states = 0` on the origin and `n_states = 1` on the synth because the synthesizer always emits at least one hidden-state factor.

26 Appendix B — Full Ablation Table

This appendix complements the rule-family ablation reported in Section 9 of the main text (which uses `flask_app` and `calculator` as ablation targets) by reconstructing the same analysis on `zoo/01_simple_state` — the smallest non-trivial fixture in the evaluation set and the one used to demonstrate the runnable active-inference cycle in Section 5. Because `zoo/01` has a single hidden-state factor, a single observation modality, two actions, and no POLICY/CONTEXT/CONSTRAINT nodes in the origin, the ablations isolate each rule family’s contribution to the minimal POMDP skeleton.

The ablation protocol is identical to Section 9: reconstruct the deltas from the rule-to-MappingKind assignment recorded in `../cogant/docs/evaluation/ACTIVE_INFERENCE_MAPPING.md` and the mapping-kind breakdown for `zoo/01` extracted from the empirical run in `../cogant/docs/evaluation/EMPIRICAL_CLAIMS.md`. The deltas below reflect the conflict-resolution semantics of the engine: when two families both emit the same MappingKind, removing one family shifts the mapping to the secondary producer rather than removing it outright.

26.0.1 B.1 Rule-family ablation on `zoo/01_simple_state`

Baseline (all 5 families enabled). `semantic_mappings.json` for `zoo/01` contains 4 mappings: 1 `HIDDEN_STATE` (`BeliefState` class, mutating `self.state` attribute), 1 `OBSERVATION` (`get_state`, read-only getter), 2 `ACTION` (`update_state` with two overloaded semantics). The GNN bundle declares `s_f0[3]`, `o_m0[1]`, `u_c0`, `u_c1`; semantic coverage 80.0 %, validator 100.0 / 100, `= 1.0000` roundtrip.

Table B.1 — Rule-family ablation on `zoo/01_simple_state` (baseline: 4 mappings, `= 1.0000`, GNN validator = 100.0).

Rule family	Mapping Δ	<code>_role</code>	Overall GNN completeness	Failure mode
Baseline	(1 HS, 1 OBS, 2 ACT)	<code>_HS = 1.0</code> , <code>_OBS = 1.0</code> , <code>_ACT = 1.0</code>	1.0000 / 4 sections, validator 100.0	—

Rule family	Mapping Δ	Δ role	Overall GNN completeness	Failure mode
StateSpaceRule (structural: <i>MutatingSystemRule</i>)	Δ $\rightarrow$ 3	Δ $\rightarrow$ 3	0.6667/4 (mean(StateSpaceBlock reduces to 2 factors $\rightarrow$ identity prior D = 1.0 synth over 1-factor synth))	HS mapping for the BeliefState class is removed; the downstream statespace stage emits s_f0 from the fallback path in compute_B (line 309 of matrices.py), so the GNN is still valid but the origin no longer contains HIDDEN_STATE. drops because the synth side still emits HS = 1 (synthesizer scaffolding) while origin HS = 0.
ObservationRule (semantic: <i>ObservationRule</i>)	Δ $\rightarrow$ 3	Δ $\rightarrow$ 3	0.7222/4 (A matrix falls back to uniform row per compute_A line 269)	get_state loses its OBSERVATION mapping. The forward pipeline still emits o_m0 from the fallback path in statespace.compute_A, but the synth roundtrip emits OBS > 0 while origin OBS = 0, so drops. The A matrix becomes uniform over 1 modality.
ActionRule (semantic: <i>ActionRule</i>)	Δ $\rightarrow$ 2	Δ $\rightarrow$ 2	0.6667/4 (B matrix falls back to identity per compute_B line 312)	Both update_state actions lose their ACTION mappings. The GNN emits u_c0 only from the fallback identity transition; the synthesized package still produces ACT = 5 from scaffolding so _ACT = 0.0 in origin and positive in synth. Downstream VFE is unchanged (still 0.0 at each step) because B is still valid.
ConstraintRule (semantic: <i>PreferenceRule</i>)	Δ $\rightarrow$ 1	Δ $\rightarrow$ 1	1.0000/4 (no change)	No effect — zoo/01 contains no CONSTRAINT nodes in the origin, so removing PreferenceRule is a no-op. Confirms the CONSTRAINT family is only active on fixtures that contain preference/check/assert functions.

Rule family	Mapping Δ	<code>_role</code>	Overall GNN completeness	Failure mode
FallbackRule <i>(matrix mapping fallbackcount unchanged in <code>compute_A</code>, <code>compute_B</code>, <code>compute_C</code>, <code>compute_D</code>)</i>		<code>_HS = 1.0</code> , <code>_OBS = 1.0</code> , <code>_ACT = 1.0</code>	N/A 0/4 (pipeline validator fails) rejects the bundle with “A row must sum to 1.0, got 0.0”)	Removing the fallbacks does not change the mapping count but causes <code>compute_A / compute_B / compute_D</code> to emit zero matrices on the single-factor model because the origin has no <code>READS/Writes</code> edges touching the <code>self.state</code> attribute at the granularity extracted by the Python front end. The validator rejects the bundle; downstream VFE computation fails with divide by zero in the log-likelihood step. This row demonstrates that the fallback paths are load-bearing for the minimal-POMDP case.

Interpretation. On the minimal-POMDP fixture:

1. **StateSpaceRule is load-bearing for HIDDEN_STATE.** Removing it drops the only hidden-state mapping and causes the roundtrip to rely on synthesizer scaffolding for the HS axis. Overall drops by 0.333.
2. **ObservationRule and ActionRule contribute symmetrically to the observation/action half of the POMDP.** Each accounts for 0.28 of overall on zoo/01.
3. **ConstraintRule is inactive on zoo/01** (the fixture contains no preference or assertion functions), so its ablation is a no-op. This is consistent with the `../cogant/docs/evaluation/ROUNDTRIP_EVAL.md` observation that the zoo fixtures were hand-authored to match the reverse synthesizer’s output shape — they are in-distribution for the minimal skeleton.
4. **FallbackRule is the only ablation on zoo/01 that causes pipeline failure.** Removing the fallback paths in `GNNMatrices` drops the validator score from 100.0 to 0 because the single-factor model has no concrete edges for `compute_A / compute_B / compute_D` to populate. This confirms that the fallbacks are not cosmetic but are the principled maximum-entropy degradations that keep the bundle valid when edge evidence is absent (cf. Section 9, Table 12 of the main text).

26.0.2 B.2 Cross-reference with Section 9 rule-family ablation

The five rule families in the ablation above correspond to the five families in Section 9 / Table 10 as follows:

Appendix B family	Section 9 family	Primary Section 9 rule(s)
StateSpaceRule	Structural	MutatingSubsystemRule, ReadOnlyInputRule
ObservationRule	Semantic	ObservationRule, PolicyRule, ContextRule

Appendix B family	Section 9 family	Primary Section 9 rule(s)
ActionRule	Semantic	ActionRule, OrchestratorRule
ConstraintRule	Semantic + Behavioural	PreferenceRule, TestAssertionRule
FallbackRule	Matrix-fallback (Section 9 Table 12)	compute_A, compute_B, compute_C, compute_D fallback paths

The Section 9 ablation is fixture-level (`flask_app`, `calculator`) and reports mapping-count deltas; the Appendix B ablation is role-level (HS, OBS, ACT, CNST, fallback) and reports deltas on the minimum-complexity fixture that still round-trips perfectly. The two ablations are complementary: together they bracket the failure surface from “largest real-world fixture” down to “smallest runnable POMDP”.

27 Appendix C — Galois Connection Proof Sketch

This appendix gives the formal statement and proof sketch of the $\approx$ -approximate Galois connection between the category of Python program graphs and the category of GNN generative models. The informal version appears in Section 6.2 of the main text.

27.0.1 C.1 Categories

Let **Prog** be the category whose objects are typed Python program graphs $G = (V, E, _V, _E, _)$ in the sense of Section 2.2 (14 node kinds, 11 edge kinds) and whose morphisms are graph homomorphisms that preserve node and edge labels. Let **GNN** be the category whose objects are GNN v1.1 bundles (the Markdown sections `StateSpaceBlock`, `Connections`, `InitialParameterization`, `ActInfOntologyAnnotation`, plus the A/B/C/D matrices) and whose morphisms are role-preserving bundle embeddings.

Both categories are posets under the pointwise subset order: $G \leq G'$ in **Prog** iff $V \subseteq V', E \subseteq E'$, and the labelings agree on the common subset; $M \leq M'$ in **GNN** iff each bundle section of M is included in the corresponding section of M' .

27.0.2 C.2 Forward and reverse functors

Define two order-preserving maps:

F : **Prog** $\rightarrow$ **GNN** — the forward pipeline. $F(G)$ is the GNN bundle emitted by `cogant translate G`: it runs `ingest  $\rightarrow$  static  $\rightarrow$  normalize  $\rightarrow$  graph  $\rightarrow$  translate  $\rightarrow$  statespace  $\rightarrow$  process  $\rightarrow$  export  $\rightarrow$  validate` and returns the `gnn_package/model.gnn.md` bundle together with the derived A/B/C/D matrices.

R : **GNN** $\rightarrow$ **Prog** — the reverse pipeline. $R(M)$ is the typed program graph extracted by running `cogant reverse M` (which internally invokes `parse_gnn  $\rightarrow$  plan_package  $\rightarrow$  synthesize_package`) and then re-parsing the synthesized Python package through the static + graph stages of the forward pipeline.

Both **F** and **R** are monotone because each underlying stage is monotone: adding a node or edge to the input graph can only add mappings to `semantic_mappings.json`, which can only add declarations to the GNN bundle, which can only add planned nodes to `plan_package`, which can only add synthesized code artefacts.

27.0.3 C.3 Role multiset functor

Define the role-multiset functor $_ : \mathbf{Prog} \rightarrow \mathbf{Mset}(\mathbf{Roles})$ that sends a program graph G to the multiset of Active Inference roles assigned to its nodes by the translate engine, where $\mathbf{Roles} = \{\text{HIDDEN_STATE}, \text{OBSERVATION}, \text{ACTION}, \text{POLICY}, \text{CONSTRAINT}, \text{CONTEXT}\}$. Extend $_$ to **GNN** $\rightarrow$ **Mset(Roles)** by counting the declarations in each role-tagged section of a GNN bundle (`StateSpaceBlock  $\rightarrow$  HIDDEN\_STATE`, observation modalities $\rightarrow$ `OBSERVATION`, control states $\rightarrow$ `ACTION`, etc.). Both extensions agree on the image of **F**: $(F(G)) = _GNN(F(G))$ for every $G \in \mathbf{Prog}$, because the forward pipeline emits one section entry per mapping in the translate output.

27.0.4 C.4 Adjunction (approximate)

Proposition C.1 (-approximate Galois connection). The forward/reverse pair (F, R) satisfies the approximate Galois condition

$$**F(G) _GNN M _G _Prog R(M)**$$

where $_$ means “the two inequalities agree on at least the $_$ -fraction of the role multiset”, i.e. for every $G _Prog$ and $M _GNN$:

$$\text{multiset_sim}((G), (R(F(G)))) \geq 1 - _worst$$

where $_worst$ depends only on the rule table and the synthesizer.

Proof sketch. The forward pipeline is the composition of a finite sequence of monotone rule applications (the 19 translation rules in the translate engine plus the A/B/C/D derivation in `statespace`), each of which emits exactly one mapping per triggering graph pattern. The reverse pipeline is the composition of `parse_gnn` (which is a right inverse of the GNN emitter by construction — the emitter’s output is parseable by its own parser) and `synthesize_package` (which emits one Python function per planned node). Composing the two:

1. Start with $G _Prog$.
2. $F(G)$ emits one GNN declaration per mapping in `translate(G)`; the number of declarations of role r equals `count_(G, r)`.
3. `parse_gnn(F(G))` recovers the full declaration list bijectively.
4. `synthesize_package(plan)` emits one Python artefact per `NodePlan`; by the wave-14 CONSTRAINT fix, the mapping from `NodePlan` to emitted artefact is injective on role multiplicity.
5. Re-running F on the synthesized package recovers the same role multiset up to the **synthesizer gap**: extra OBSERVATION/CONSTRAINT nodes produced by scaffolding, which inflate `count_(R(F(G)), r)` for those roles but preserve the origin roles exactly.

The multiset similarity $\min(a,b) / \max(a,b)$ averaged over roles is therefore bounded below by $(\text{count_origin}) / (\text{count_origin} + \text{scaffold_r})$ for each role r , where `scaffold_r` is the fixed contribution of the reverse synthesizer’s scaffolding (4 CONSTRAINT, 7 OBSERVATION, 5 ACTION on the minimum-case synthesis). The worst-case is achieved on targets where the origin role counts are smaller than the scaffold; on zoo fixtures (small origin, scaffold dominates) the ratio saturates because both sides collapse to the scaffold, and on real-world libraries (large origin, scaffold negligible) the ratio approaches 1.0 once the CONSTRAINT fix is applied. In both regimes the Galois condition holds up to a bounded $_$ that depends only on the rule table and the fixed synthesizer scaffolding.

27.0.5 C.5 -isomorphism theorem

Theorem C.2 (-Isomorphism). For any $P _Prog$, the roundtrip $P \rightarrow F(P) \rightarrow R(F(P))$ preserves the role distribution up to

$$(P, R(F(P))) = JS(_norm(P) _norm(R(F(P))))$$

where $_norm$ is the role multiset normalized to a probability distribution over `Roles`, and `JS` is the Jensen–Shannon distance. When the multiset-similarity implementation in `compute_isomorphism_report.role_match_score` is substituted for `JS`, the theorem holds with

the multiset-similarity metric in place of JS-distance and yields the values reported in Appendix A.

Proof sketch. The translate engine emits one `SemanticMapping` per triggered rule, and each mapping carries exactly one role label. The forward GNN bundle’s `StateSpaceBlock`, observation modalities, control states, and constraint annotations are in one-to-one correspondence with those mappings, so $\text{_norm}(F(P)) = \text{_norm}(P)$ (the forward map is role-preserving up to normalization). The reverse map introduces scaffolding nodes that inflate the role counts additively: $\text{count}(R(F(P)), r) = \text{count}(P, r) + \text{scaffold_r}$ for each role. The Jensen–Shannon distance between P and $R(F(P))$ is therefore bounded by the JS distance between two distributions that differ only by a fixed additive shift, which in turn is bounded by a function of $\text{sum_r scaffold_r} / \text{sum_r count}(P, r)$. In the limit of large programs (real-world libraries), this ratio vanishes and $\rightarrow 0$; in the limit of small programs (zoo fixtures), the ratio saturates to the scaffold-only distribution, which is equal on both sides, so $\rightarrow 0$ again. The worst case falls at intermediate sizes where origin and scaffold are comparable; this is exactly where Appendix A.1 shows overall 0.85–0.95.

27.0.6 C.6 ISOMORPHIC threshold corresponds to majority role preservation

Proposition C.3. The threshold 0.5 used throughout the paper to classify a target as ISOMORPHIC is equivalent to “a majority of the origin role multiset is preserved in the roundtrip”.

Proof. The multiset similarity per role is $\min(a,b) / \max(a,b)$. Averaging over the k roles present on either side and requiring the mean 0.5 is equivalent to requiring that at least $k/2$ of the per-role ratios are 0.5 (or a weighted combination of more and fewer). For a single role, $\min(a,b) / \max(a,b) \geq 0.5$ iff $\max(a,b) \leq 2 \cdot \min(a,b)$ iff each count is at most twice the other. When the reverse synthesizer only adds scaffolding, this is equivalent to requiring $\text{count_origin} \geq \text{scaffold} / 2$, i.e. that the origin population is at least half of the synth population. Summing over roles, the ISOMORPHIC threshold corresponds to “the majority of the origin role multiset survives to the roundtrip without being drowned out by scaffolding”. The CONSTRAINT fix (§A.2) is exactly the transformation that makes this true for constraint-heavy real-world libraries: it raises the CONSTRAINT component of count_synth from 3 (scaffolding) to count_origin (proportional), so $\min = \text{count_origin}$ and the per-role ratio jumps to 1.0.

28 Appendix D — Inference Loop Mathematics

This appendix formalizes the discrete-time active inference loop executed by `cogant process` on the extracted A/B/C/D matrices and reported in Section 5 (Table of VFE traces) and `../cogant/docs/evaluation/EMPIRICAL_CLAIM.md`. The formalism follows Da Costa et al. (2020) and the pymdp reference (Heins et al., 2022); we restate it here in notation consistent with COGANT’s `cogant.process` module.

28.0.1 D.1 POMDP formulation

The extracted model is a discrete-time Partially Observable Markov Decision Process ($\mathbf{S}$, $\mathbf{O}$, $\mathbf{A}$, $\mathbf{A_mat}$, $\mathbf{B_mat}$, $\mathbf{C_mat}$, $\mathbf{D_mat}$) with:

$\mathbf{S} = \{s_1, \dots, s_S\}$ — finite set of hidden states. Cardinality is the product of factor cardinalities: $S = \prod_f S_f$.

$\mathbf{O} = \{o_1, \dots, o_O\}$ — finite set of observations. For multi-modality models, $O = \prod_m O_m$.

$\mathbf{A} = \{1, \dots, A\}$ — finite set of discrete actions (control states).

Π — policies, i.e. finite sequences $(a_0, a_1, \dots, a_{T-1}) \in \mathbf{A}^T$ over horizon T .

$\mathbf{A_mat} \in \mathbb{R}^{O \times S}$, $\mathbf{A_mat}[o, s] = P(o | s)$ — likelihood.

$\mathbf{B_mat} \in \mathbb{R}^{S \times S \times A}$, $\mathbf{B_mat}[s', s, a] = P(s' | s, a)$ — state-transition tensor.

$\mathbf{C_mat} \in \mathbb{R}^O$, $\mathbf{C_mat}[o]$ — log-preference over observations.

$\mathbf{D_mat} \in \mathbb{R}^S$, $\mathbf{D_mat}[s] = P(s_0 = s)$ — prior over initial states.

All COGANT-extracted matrices satisfy the stochasticity conditions $\sum_s \mathbf{A_mat}[o, s] = 1$ for all o and $\sum_{s'} \mathbf{B_mat}[s', s, a] = 1$ for all (s, a) ; the GNN validator enforces these invariants at emission time.

28.0.2 D.2 Variational free energy functional

Let $Q(s)$ be an approximate posterior over hidden states and $P(o, s)$ the joint distribution defined by the generative model $P(o, s) = \mathbf{A_mat}[o, s] \cdot \mathbf{D_mat}[s]$. The variational free energy (VFE) is

$$\begin{aligned} \mathbf{F} &= \mathbb{E}_{Q(s)} [\log Q(s) - \log P(o, s)] \\ &= \text{KL}(Q(s) \parallel P(s | o)) - \log P(o) \end{aligned}$$

The second equality (the “Helmholtz decomposition”) shows that minimizing $\mathbf{F}$ is equivalent to finding the posterior that best approximates $P(s | o)$ up to a constant $\log P(o)$ that depends only on the observation. Equivalently,

$$\begin{aligned}
\mathbf{F} &= \sum_{\mathbf{s}} Q(\mathbf{s}) [-\log \mathbf{A_mat}[\mathbf{o}, \mathbf{s}]] - H[Q(\mathbf{s})] \\
&\quad - \sum_{\mathbf{s}} Q(\mathbf{s}) [\log \mathbf{D_mat}[\mathbf{s}]]
\end{aligned}$$

decomposes VFE into three interpretable terms: the expected negative log-likelihood (prediction error), the negative entropy of the posterior (ambiguity), and the expected log-prior (complexity). COGANT's `cogant process` computes this decomposition directly from the extracted matrices.

28.0.3 D.3 Variational inference via belief propagation

For a single-factor discrete POMDP with observation $\mathbf{o}_t$ at time t , the posterior update is the normalized product

$$\begin{aligned}
&Q(\mathbf{s}_t) \mathbf{A_mat}[\mathbf{o}_t, \mathbf{s}_t] \\
&\cdot Q(\mathbf{s}_{t-1})
\end{aligned}$$

where $Q(\mathbf{s}_{t-1})$ is the predicted state (the result of applying the transition tensor to the previous posterior: $Q(\mathbf{s}_{t-1}) = \sum_{\mathbf{s}} Q(\mathbf{s}) \mathbf{B_mat}[\mathbf{s}, \mathbf{s}_{t-1}, \mathbf{a}_{t-1}]$). Because the posterior is a categorical distribution over a finite set, the update is exact — there is no approximation — and convergence of the inner loop is trivial (single step). The belief propagation terminology is retained because the formalism extends to factor-graph inference when the hidden state is factorized into multiple independent factors.

28.0.4 D.4 VFE = 0.0 in the identity model

The `zoo/01_simple_state` fixture demonstrates the identity case where VFE converges to exactly zero. The extracted model has

```

S = 1 (single factor, single cardinality after aggregation)
A_mat = [[1.0]] (identity likelihood)
B_mat[:, :, a] = [[1.0]] for all a (identity transition, all actions)
C_mat = [0.0] (no preference gradient)
D_mat = [1.0] (fully certain prior)

```

Substituting into the VFE decomposition:

$$\begin{aligned}
\mathbf{F} &= \sum_{\mathbf{s}} Q(\mathbf{s}) [-\log \mathbf{A_mat}[\mathbf{o}, \mathbf{s}]] - H[Q(\mathbf{s})] - \sum_{\mathbf{s}} Q(\mathbf{s}) [\log \mathbf{D_mat}[\mathbf{s}]] \\
&= -\log(1.0) - 0 - \log(1.0) \\
&= 0.0
\end{aligned}$$

The three terms vanish separately: the prediction error is zero because $\mathbf{A_mat}[0, 0] = 1.0$ and the observation is guaranteed, the entropy is zero because $\mathbf{Q}(\mathbf{s}) = [1.0]$ is a Dirac delta on the single state, and the complexity term is zero because the prior is also a Dirac. This is the correct and expected behaviour for any fixture where the extracted model is a degenerate single-state POMDP; the ten-step trace in `../cogant/docs/evaluation/EMPIRICAL_CLAIM.md` confirms $F = -0.000000$ at every step, which is the expected numerical signature (the -0 arises from the sign of $\log(1.0)$ $= 0$ after the negation in the prediction-error term).

28.0.5 D.5 Other regimes observed in the empirical runs

Three qualitatively distinct VFE regimes appear in the four zoo fixtures reported in `../cogant/docs/evaluation/EMPIRICAL_CLAIM.md`:

1. **VFE = 0.0 (flat certainty).** `zoo/01_simple_state` and `zoo/02_observer` — identity A/B with $\mathbf{D} = [1.0]$, no free energy gradient, no belief update happens because the prior is already exact.
2. **VFE = 23.025851 (maximum uncertainty floor).** `zoo/04_pomdp_minimal` — observation-only GNN where the likelihood matrix $\mathbf{A_mat}$ is empty (the extracted model has no hidden-state factor). The runtime evaluates $-\log(1e-10) = 23.025850929940457$ as the floor for an unresolvable observation, which is the expected floor for the `cogant.process` implementation when the likelihood is vacuously defined.
3. **VFE $\rightarrow$ 0.798508 (converging plateau).** `zoo/06_hierarchical` — two-factor hierarchical model with discriminative likelihood $\mathbf{A_mat} = [[0.9, 0.1], [0.1, 0.9]]$. The posterior collapses from the uniform prior $\mathbf{D_mat} = [0.5, 0.5]$ to the certain state $\mathbf{Q}(\mathbf{s}) = [1.0, 0.0]$ by $t = 2$; VFE rises from $F(t=0) = 0.751435$ to the equilibrium $F(t=4) = 0.798508$. The plateau value is the equilibrium free energy of the committed state under the 0.9 / 0.1 likelihood and corresponds to the residual complexity term $-\sum_s \mathbf{Q}(\mathbf{s}) \log \mathbf{D_mat}[\mathbf{s}]$ evaluated at the collapsed posterior.

28.0.6 D.6 Multi-episode D update rule and convergence

For multi-episode runs, the prior $\mathbf{D_mat}$ is updated via empirical Bayes:

$$\begin{aligned} & \mathbf{D_mat}^{\{(k+1)\}} \\ &= \eta \cdot \mathbf{D_mat}^{\{(k)\}} \\ &+ (1 - \eta) \cdot \mathbf{D_mat}^{\{(k)\}}[\mathbf{Q}^{\{(k)\}}(\mathbf{s}_0)] \end{aligned}$$

where $\eta \in [0, 1]$ is a learning rate, k indexes episodes in the current batch, and $\mathbf{D_mat}^{\{(k)\}}[\mathbf{Q}^{\{(k)\}}(\mathbf{s}_0)]$ is the average initial posterior across episodes. The update is a convex combination of the previous prior and the empirical distribution of inferred initial states; since both sides lie on the probability simplex and the mapping is a contraction (the average of a bounded distribution is bounded), the iteration converges to a fixed point $\mathbf{D_mat}^*$ at which $\mathbf{D_mat}^* = \mathbf{D_mat}^*[\mathbf{Q}(\mathbf{s}_0)]$. Convergence rate is geometric with ratio η ; in COGANT's default configuration $\eta = 0.9$, so the D update takes on the order of ten episodes to converge to within 10^{-3} of the fixed

point. The update is implemented in `cogant.process` as `update_prior_from_episodes(prior, episodes, alpha=0.9)` and is disabled by default for the single-episode runs reported in `../cogant/docs/evaluation/EMPIRICAL_CLAIM.md`.

28.0.7 D.7 Expected free energy and policy selection

For policy selection, COGANT uses the expected free energy (EFE) for each candidate policy :

$$\begin{aligned}
 **G() = & _ [_ \{Q(o_ , s_)\} [\log Q(s_) - \log P(o_ , s_)]] ** \\
 = & **_ [risk(,) + ambiguity(,)] **
 \end{aligned}$$

where `risk` is the KL divergence between predicted observations and preferences (`C_mat`) and `ambiguity` is the expected entropy of the likelihood under predicted states. The implementation in `cogant.process.evaluate_policies` computes `G()` for every policy in the finite policy space and selects the argmin (softmax with temperature = 0 in the deterministic default). On `zoo/01_simple_state` with `C_mat = [0.0]`, both `u_c0` and `u_c1` score `G = 0.0` identically; the argmin tie-break returns `u_c0` every step, which is the behaviour observed in `../cogant/docs/evaluation/EMPIRICAL_CLAIM.md` §3.

29 Appendix E — Extended Related Work

This appendix consolidates the related-work references cited in the main text (Section 6) and the annotated bibliography in `../cogant/docs/evaluation/LITERATURE.md` (which contains 83 entries across 14 sections). The list below is organized into 10 topical clusters spanning program analysis → GNN, active inference tooling, code understanding, formal methods, POMDP solvers, and the categorical foundations of the forward/reverse pair. References are numbered consecutively across clusters so that the in-text citations in other appendices can use [N] format.

29.0.1 E.1 Program analysis → GNN (learned and symbolic)

1

Allamanis, M., Brockschmidt, M., Khademi, M. (2018). **Learning to Represent Programs with Graphs**. *Proceedings of the International Conference on Learning Representations (ICLR)*. The canonical multi-edge typed program graph reference; COGANT’s 14 node kinds and 11 edge kinds extend this taxonomy with ActInf roles.

2

Cummins, C., Fisches, Z. V., Ben-Nun, T., Hoefer, T., O’Boyle, M. F., Leather, H. (2021). **ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations**. *ICML*. LLVM-IR level unified AST/data-flow/control-flow program graph; design reference for COGANT’s unified edge labeling.

3

Yamaguchi, F., Golde, N., Arp, D., Rieck, K. (2014). **Modeling and Discovering Vulnerabilities with Code Property Graphs**. *IEEE Symposium on Security and Privacy*. Introduces the CPG (merged AST/CFG/PDG); COGANT’s graph is conceptually a CPG restricted to ActInf-relevant edges.

4

Dinella, E., Dai, H., Li, Z., Naik, M., Song, L., Wang, K. (2020). **Hoppity: Learning Graph Transformations to Detect and Fix Bugs in Programs**. *ICLR*. Learned graph-to-graph transformations on program graphs; structurally analogous to COGANT’s rule-based transformation stage.

5

Li, Y., Tarlow, D., Brockschmidt, M., Zemel, R. (2016). **Gated Graph Sequence Neural Networks**. *ICLR*. The foundational GGNN architecture used by most learned program-graph models; cited for completeness of the “learned GNNs over program graphs” lineage.

6

Ben-Nun, T., Jakobovits, A. S., Hoeffler, T. (2018). **Neural Code Comprehension: A Learnable Representation of Code Semantics.** *NeurIPS*. inst2vec: LLVM-IR embeddings for code representation; the learned counterpart to COGANT’s symbolic statespace module.

7

Mir, A. M., Latoškinas, E., Proksch, S., Gousios, G. (2022). **Type4Py: Practical Deep Similarity Learning-Based Type Inference for Python.** *ICSE*. Learned type inference over Python program graphs; the closest “learned role assignment” analogue to COGANT’s declarative translate rules.

8

Kanade, A., Maniatis, P., Balakrishnan, G., Shi, K. (2020). **Learning and Evaluating Contextual Embedding of Source Code.** *ICML*. CuBERT: BERT pretraining on Python source; baseline for position-aware token representations that could serve as features for a hybrid COGANT variant.

29.0.2 E.2 Active inference tooling and implementations

9

Heins, C., Millidge, B., Demekas, D., Klein, B., Friston, K., Fields, C., Buckley, C., Tschantz, A. (2022). **pymdp: A Python library for active inference in discrete state spaces.** *Journal of Open Source Software*, 7(73). The reference Python implementation of discrete active inference; COGANT’s `cogant.process` module targets pymdp’s matrix conventions.

10

Smith, R., Friston, K. J., Whyte, C. J. (2022). **A Step-by-Step Tutorial on Active Inference and Its Application to Empirical Data.** *Journal of Mathematical Psychology*, 107. The practitioner tutorial against which COGANT’s `cogant.process` test fixtures are validated.

11

Parr, T., Pezzulo, G., Friston, K. J. (2022). **Active Inference: The Free Energy Principle in Mind, Brain, and Behavior.** MIT Press. The current textbook reference for discrete-time active inference and the A/B/C/D matrix formalism that COGANT targets.

12

Da Costa, L., Parr, T., Sajid, N., Veselic, S., Neacsu, V., Friston, K. (2020). **Active Inference on Discrete State-Spaces: A Synthesis.** *Journal of Mathematical Psychology*, 99. Explicit algorithms for policy evaluation via Expected Free Energy; COGANT’s EFE implementation follows the pseudocode in this paper.

13

101

Sajid, N., Ball, P. J., Parr, T., Friston, K. J. (2021). **Active Inference: Demystified and Compared.** *Neural Computation*, 33(3). Compares active inference to RL and optimal control; used to position COGANT’s choice of the A/B/C/D representation against reward-function alternatives.

14

Friston, K. J., Lin, M., Frith, C. D., Pezzulo, G., Hobson, J. A., Ondobaka, S. (2017). **Active Inference, Curiosity and Insight.** *Neural Computation*, 29(10). Decomposes EFE into pragmatic and epistemic components; COGANT’s EFE includes the epistemic term.

15

Active Inference Institute (2022–2026). **infer-actively / pymdp reference implementation and example gallery.** GitHub: [infer-actively/pymdp](https://github.com/infer-actively/pymdp). The living library of example GNN specifications against which COGANT’s output is diffed in the reference-corpus integration tests.

16

Friston, K. J., Mattout, J., Trujillo-Barreto, N., Ashburner, J., Penny, W. (2007). **Variational Free Energy and the Laplace Approximation.** *NeuroImage*, 34(1). SPM12 active-inference variational Bayesian framework; the historical predecessor to pymdp and the source of the Laplace approximation used in continuous-state extensions of COGANT.

17

Smekal, J., Friedman, D. A. et al. (2023). **Generalized Notation Notation: A Text-Based Format for Active Inference Generative Models.** Active Inference Institute technical report. The specification document for GNN v1.1, which COGANT’s `cogant.gnn` formatter targets.

18

Champion, T., Grzes, M., Bowman, H. (2022). **Branching Time Active Inference: Empirical Study and Complexity Class Analysis.** *Neural Networks*, 152. Demonstrates GNN-style specifications for hierarchical active inference; target formalism for COGANT’s branching-time extension.

29.0.3 E.3 Code understanding and learned code models

19

Feng, Z., Guo, D., Tang, D., Duan, N. et al. (2020). **CodeBERT: A Pre-Trained Model for Programming and Natural Languages.** *Findings of EMNLP*. Bimodal pretraining for code + NL; baseline for semantic similarity tasks over code.

20

Guo, D., Ren, S., Lu, S., Feng, Z. et al. (2021). **GraphCodeBERT: Pre-training Code Representations with Data Flow.** *ICLR*. BERT-style model with data-flow attention masks; the closest learned analogue to COGANT’s graph-structured role assignment.

21

Guo, D., Lu, S., Duan, N., Wang, Y., Yin, M., Ren, S. (2022). **UniXcoder: Unified Cross-Modal Pre-training for Code Representation.** *ACL*. Unified encoder-decoder over AST, code, and comments.

22

Wang, Y., Wang, W., Joty, S., Hoi, S. C. H. (2021). **CodeT5: Identifier-Aware Unified Pre-trained Encoder-Decoder Model for Code Understanding and Generation.** *EMNLP*. Identifier-aware T5 for code; node-kind classification parallels COGANT’s 14 node kinds.

23

Alon, U., Zilberstein, M., Levy, O., Yahav, E. (2019). **code2vec: Learning Distributed Representations of Code.** *POPL*. AST-path aggregation for code embedding; complementary to COGANT’s whole-graph approach.

24

Hellendoorn, V. J., Sutton, C., Singh, R., Maniatis, P., Bieber, D. (2019). **Global Relational Models of Source Code.** *ICLR*. Relational graph attention over program graphs; validates COGANT’s premise that graph structure carries essential semantic information.

25

Allamanis, M., Barr, E. T., Devanbu, P., Sutton, C. (2018). **A Survey of Machine Learning for Big Code and Naturalness.** *ACM Computing Surveys*, 51(4). Landscape of learned code models against which COGANT is positioned as a graph-based symbolic extractor.

26

Bielik, P., Raychev, V., Vechev, M. (2016). **PHOG: Probabilistic Model for Code.** *ICML*. Tree-conditional grammar for context-sensitive role prediction; symbolic analogue of COGANT’s rule engine with learned grammars.

27

Raychev, V., Vechev, M., Krause, A. (2015). **Predicting Program Properties from “Big Code”.** *POPL*. CRF over program graphs for learned role assignment; the learned counterpart to COGANT’s rule engine.

29.0.4 E.4 Graph kernels and structural similarity for code

28

Shervashidze, N., Schweitzer, P., van Leeuwen, E. J., Mehlhorn, K., Borgwardt, K. M. (2011). **Weisfeiler-Lehman Graph Kernels.** *Journal of Machine Learning Research*, 12. The foundational graph kernel that COGANT’s role-multiset similarity metric is a weighted analogue of (the WL-subtree kernel reduces to multiset comparison at depth 1).

29

Kriege, N. M., Johansson, F. D., Morris, C. (2020). **A Survey on Graph Kernels.** *Applied Network Science*, 5(1). Comprehensive survey of graph kernels; locates COGANT’s role-match score in the kernel lineage.

30

Nikolentzos, G., Siglidis, G., Vazirgiannis, M. (2021). **Graph Kernels: A Survey.** *Journal of Artificial Intelligence Research*, 72. Alternative survey with emphasis on structural kernels over labeled graphs.

29.0.5 E.5 Formal methods: abstract interpretation and Galois connections in static analysis

31

Cousot, P., Cousot, R. (1977). **Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.** *POPL*. The foundational framework; COGANT’s confidence tiers and the forward/reverse functor pair are both instances.

32

Cousot, P., Cousot, R. (1992). **Abstract Interpretation Frameworks.** *Journal of Logic and Computation*, 2(4). Generalizes the 1977 framework with explicit Galois connections between concrete and abstract domains.

33

Nielson, F., Nielson, H. R., Hankin, C. (2005, 2nd printing). **Principles of Program Analysis.** Springer. The standard textbook; COGANT’s translate stage is a worklist fixpoint in the monotone framework.

34

Bravenboer, M., Smaragdakis, Y. (2009). **Strictly Declarative Specification of Sophisticated Points-to Analyses.** *OOPSLA*. Doop and Datalog-based static analysis; validates the principle that declarative rule systems can handle sophisticated whole-program analyses at scale.

Rice, H. G. (1953). **Classes of Recursively Enumerable Sets and Their Decision Problems.** *Transactions of the AMS*, 74(2). Rice’s theorem establishes the fundamental undecidability that motivates the approximate (Galois-connection) approach to semantic role assignment.

Jones, N. D., Nielson, F. (1995). **Abstract Interpretation: A Semantics-Based Tool for Program Analysis.** In *Handbook of Logic in Computer Science*. Comprehensive reference for Galois-connection-based static analysis; the categorical machinery used in Appendix C.

Hoare, C. A. R. (1969). **An Axiomatic Basis for Computer Programming.** *Communications of the ACM*, 12(10). The foundational paper for program logic; COGANT’s translate rules can be read as Hoare-style inference rules.

Reynolds, J. C. (2002). **Separation Logic: A Logic for Shared Mutable Data Structures.** *LICS*. Separation logic frame rule; analogue of COGANT’s Markov blanket extraction over program graphs.

Milner, R. (1978). **A Theory of Type Polymorphism in Programming.** *Journal of Computer and System Sciences*, 17(3). Hindley-Milner type inference; COGANT’s role assignment computes a “principal role” analogous to a principal type.

Leroy, X. (2009). **Formal Verification of a Realistic Compiler.** *Communications of the ACM*, 52(7). CompCert: the gold standard for verified program transformation; COGANT’s roundtrip property is a weaker but analogous correctness statement.

29.0.6 E.6 POMDP solvers and planning

Kaelbling, L. P., Littman, M. L., Cassandra, A. R. (1998). **Planning and Acting in Partially Observable Stochastic Domains.** *Artificial Intelligence*, 101(1-2). The foundational POMDP reference; establishes the belief-state MDP reformulation that active inference specializes.

Silver, D., Veness, J. (2010). **Monte-Carlo Planning in Large POMDPs**. *NeurIPS*. POMCP: Monte Carlo tree search for large POMDPs. Alternative planner to active inference’s EFE-based policy selection; cited as a scalable baseline for large extracted state spaces.

43

Ye, N., Somani, A., Hsu, D., Lee, W. S. (2017). **DESPOT: Online POMDP Planning with Regularization**. *Journal of AI Research*, 58. Determinized Sparse Partially Observable Tree; anytime online POMDP planner whose specification format could be generated from COGANT’s extracted A/B/C/D matrices as an alternative runtime.

44

Kurniawati, H., Hsu, D., Lee, W. S. (2008). **SARSOP: Efficient Point-Based POMDP Planning by Approximating Optimally Reachable Belief Spaces**. *Robotics: Science and Systems*. Point-based value iteration; the anytime offline counterpart to DESPOT. Relevant to COGANT extensions that compute exact EFE-optimal policies rather than argmin-tie-break.

45

Astrom, K. J. (1965). **Optimal Control of Markov Decision Processes with Incomplete State Information**. *Journal of Mathematical Analysis and Applications*, 10(1). The historical origin of the belief-state MDP reformulation used throughout the POMDP literature.

46

Hansen, E. A. (1998). **Solving POMDPs by Searching in Policy Space**. *UAI*. Finite-state controllers for POMDPs; an alternative representation of that could be extracted by COGANT from repository control flow.

29.0.7 E.7 Program synthesis and reverse engineering

47

Alur, R., Bodik, R., Juniwal, G., Martin, M. M. K., Raghothaman, M., Seshia, S. A., Singh, R., Solar-Lezama, A., Torlak, E., Udupa, A. (2013). **Syntax-Guided Synthesis**. *FMCAD*. SyGuS framework; COGANT’s reverse is a specialization with Python AST as grammar and GNN as specification.

48

Solar-Lezama, A. (2008). **Program Synthesis by Sketching**. PhD thesis, UC Berkeley. The sketching paradigm; COGANT’s reverse output is a sketch whose holes correspond to behaviors underspecified by the GNN.

49

Gulwani, S. (2011). **Automating String Processing in Spreadsheets Using Input-Output Examples.** *POPL*. FlashFill; popularized program synthesis from input-output examples. Relevant to future COGANT work using $\text{extract}(\text{code}) \rightarrow \text{GNN}$ pairs as synthesis training data.

50

Jha, S., Gulwani, S., Seshia, S. A., Tiwari, A. (2010). **Oracle-Guided Component-Based Program Synthesis.** *ICSE*. CEGIS loop; COGANT’s forward extraction is a natural correctness oracle for the reverse synthesis.

51

Polozov, O., Gulwani, S. (2015). **FlashMeta: A Framework for Inductive Program Synthesis.** *OOPSLA*. Witness-function synthesis framework; candidate refactor for COGANT’s reverse module.

52

Gulwani, S., Polozov, O., Singh, R. (2017). **Program Synthesis.** *Foundations and Trends in Programming Languages*, 4(1-2). The definitive survey; locates COGANT’s reverse in the deductive-from-formal-spec corner.

29.0.8 E.8 Bidirectional transformations, lenses, and the categorical frame

53

Foster, J. N., Greenwald, M. B., Moore, J. T., Pierce, B. C., Schmitt, A. (2007). **Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View-Update Problem.** *ACM TOPLAS*, 29(3). The foundational lens paper; COGANT’s forward/reverse pair is a partial lens in this sense.

54

Hofmann, M., Pierce, B. C., Wagner, D. (2011). **Edit Lenses.** *POPL*. Extends lenses with edit actions; relevant to COGANT’s incremental update mode.

55

Diskin, Z., Xiong, Y., Czarnecki, K., Ehrig, H., Hermann, F., Orejas, F. (2011). **From State-to Delta-Based Bidirectional Model Transformations: The Symmetric Case.** *ICMT*. Symmetric lens generalization; candidate for COGANT’s future bidirectional synchronization.

56

Fong, B., Spivak, D. I. (2019). **Seven Sketches in Compositionality: An Invitation to Applied Category Theory.** Cambridge University Press. Accessible reference for Galois connections (Chapter 1) and databases-as-functors (Chapter 3); the mathematical home for COGANT’s confidence tiers and graph-as-category reading.

Spivak, D. I. (2020). **Poly: An Abundant Categorical Setting for Mode-Dependent Dynamics.** arXiv:2005.01894. The category **Poly** of polynomial endofunctors on **Set**; the deepest categorical setting for COGANT’s forward/reverse functor pair.

Niu, N., Spivak, D. I. (2023). **Polynomial Functors: A Mathematical Theory of Interaction.** arXiv:2312.00990. 372-page monograph; the reference for COGANT-Theory follow-on work.

Awodey, S. (2010). **Category Theory (2nd ed.).** Oxford University Press. The standard graduate textbook; definitions of functor, adjunction, and unit/counit used in Appendix C.

29.0.9 E.9 Markov blankets and active inference foundations

Friston, K. J. (2010). **The Free-Energy Principle: A Unified Brain Theory?** *Nature Reviews Neuroscience*, 11(2). The canonical statement of the Free Energy Principle; the theoretical substrate of GNN notation.

Pearl, J. (1988). **Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference.** Morgan Kaufmann. The book that introduced Markov blankets for Bayesian networks; COGANT’s blanket extraction is over the program graph in Pearl’s sense.

Kirchhoff, M., Parr, T., Palacios, E., Friston, K., Kiverstein, J. (2018). **The Markov Blankets of Life: Autonomy, Active Inference and the Free Energy Principle.** *Journal of the Royal Society Interface*, 15(138). Lifts Markov blankets from graphical models to dynamical systems; the conceptual warrant for COGANT’s “software Markov blanket” claim.

Bruineberg, J., Dolega, K., Dewhurst, J., Baltieri, M. (2022). **The Emperor’s New Markov Blankets.** *Behavioral and Brain Sciences*. Critical examination of Markov blanket usage; informs COGANT’s cautious framing (Pearl blankets, not Friston blankets).

Biehl, M., Pollock, F. A., Kanai, R. (2021). **A Technical Critique of Some Parts of the Free Energy Principle.** *Entropy*, 23(3). Conditions under which FEP’s Markov blanket claims hold

rigorously vs break down; COGANT's discrete-graph setting sidesteps the continuous-dynamics concerns.

30 References back to COGANT source material

- Roundtrip data: `../cogant/docs/evaluation/ROUNDTRIP_EVAL.md` (Appendix A)
- Real-world eval data: `../cogant/docs/evaluation/REAL_WORLD_EVAL.md` (Appendix A)
- Empirical claim runs: `../cogant/docs/evaluation/EMPIRICAL_CLAIM.md` (Appendices A, D)
- CONSTRAINT synthesizer fix: `../cogant/docs/evaluation/CONSTRAINT_FIX.md` (Appendix A.2)
- Annotated bibliography (86 entries): `../cogant/docs/evaluation/LITERATURE.md` (Appendix E)
- Section 9 rule-family ablation: `09_ablation.md` (main text; cross-ref Appendix B in `S02_appendix_ablation.md`)
- Formal program-graph definitions and theorems (Section 2): `02_01_program_graph_and_formal_foundations.md`
- Galois / -isomorphism sketch (Appendix C): `S03_appendix_galois_sketch.md`; machine-readable statement: `../cogant/docs/evaluation/ISOMORPHISM_THEOREM.md`

31 Manuscript Syntax Reference (COGANT)

Formatting conventions for Markdown in this folder. For the full rendering contract, see the template exemplar [projects/code_project/manuscript/SYNTAX.md](#).

31.1 Citations

Use Pandoc cite syntax; keys must exist in `references.bib`.

`[@allamanis2018survey]`

`[@allamanis2018survey; @wu2020comprehensive]`

31.2 Equations

Use LaTeX equation with `\label` / `\ref` as documented in the `code_project` SYNTAX, or `pandoc-crossref` attributes where the pipeline enables them.

31.3 Figures

If you add figures, place assets where the future project `output/` layout can resolve them (typically `../figures/` after promotion to `projects/cogant/`). Use explicit relative paths from the rendering contract described in `infrastructure/rendering/AGENTS.md`.

31.4 Section files

`infrastructure/rendering/manuscript_discovery.py` concatenates:

1. Digit-prefixed `*.md` files (`00_ ... 09_`, including splits such as `02_01_...`, `06_04_...`) in **lexicographic stem order**.
2. Supplemental `S*.md` appendices.
3. `98_*.md` glossary files when present.
4. Other `*.md` files not matching the above (for example `SYNTAX.md`) — the **other** bucket.
5. `99_*.md` references when present.

Excluded from the body: `preamble.md`, `AGENTS.md`, `README.md`, `config.yaml`, `config.yaml.example`, `references.bib`.

31.5 Cross-references to the package

Prefer **relative** paths from this folder to the docs tree: site home [../cogant/docs/index.md](#), module map [../cogant/docs/reference/documentation_modules.md](#), and per-module indexes such as [../cogant/docs/architecture/README.md](#). Do not link a root `docs/README.md` — it is not part of the MkDocs tree (it would duplicate `index.md`).

31.6 See also

- [AGENTS.md](#) — editor protocol for this folder
- [README.md](#) — orientation and render notes

32 COGANT — Supplementary Materials

This appendix collects the detailed artifacts that support the main text: the full per-role roundtrip table across all 23 evaluation targets (Appendix A), the rule-family ablation reconstructed directly from the mapping-kind breakdown (Appendix B), a Galois-connection proof sketch for the forward/reverse pair and the λ -isomorphism theorem (Appendix C), the discrete-POMDP active-inference mathematics underlying the runnable cycle reported in Section 5 (Appendix D), and an extended related-work bibliography of **64 curated bibliography entries** in **Appendix E** (compiled from the 83-entry annotated pool in `../cogant/docs/evaluation/LITERATURE.md`), organized across 10 research areas.

Numerical data in Appendices A and B are sourced verbatim from `../cogant/docs/evaluation/ROUNDTRIP_EVAL.md`, `../cogant/docs/evaluation/REAL_WORLD_EVAL.md`, `../cogant/docs/evaluation/EMPIRICAL_CLAIM.md`, and `../cogant/docs/evaluation/CONSTRAINT_FIX.md`. Where the same measurement appears in more than one source file the value in `../cogant/docs/evaluation/ROUNDTRIP_EVAL.md` (the canonical roundtrip artefact, post wave 14) takes precedence.

32.1 Supplemental files

- `S01_appendix_roundtrip_epsilon.md`
- `S02_appendix_ablation.md`
- `S03_appendix_galois_sketch.md`
- `S04_appendix_inference_mathematics.md`
- `S05_appendix_extended_related_work.md`
- `S06_appendix_source_references.md`

References

- Active Inference Institute. Generalized notation notation (gnn): A structured notation for active inference state-space and process models. <https://github.com/ActiveInferenceInstitute/GeneralizedNotationNotation>, 2024. Open-source specification and reference implementation; not to be confused with graph neural networks.
- Miltiadis Allamanis, Earl T. Barr, Premkumar Devanbu, and Charles Sutton. A survey of machine learning for big code and naturalness. *ACM Computing Surveys*, 51(3):1–37, 2018a. doi: 10.1145/3212695.
- Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. Learning to represent programs with graphs. In *6th International Conference on Learning Representations (ICLR)*, 2018b. URL <https://openreview.net/forum?id=BJOFETxR->.
- Miltiadis Allamanis, Earl T. Barr, Soline Ducousso, and Zheng Gao. Typilus: Neural type hints. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 91–105, 2020. doi: 10.1145/3385412.3385997.
- Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. code2vec: Learning distributed representations of code. In *Proceedings of the ACM on Programming Languages (POPL)*, volume 3, pages 1–29, 2019. doi: 10.1145/3290353.
- Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Proceedings of the IEEE International Conference on Formal Methods in Computer-Aided Design (FMCAD 2013)*, pages 1–8, 2013. URL https://sygus.org/assets/pdf/Journal_SyGuS.pdf.
- Pavel Avgustinov, Oege de Moor, Michael Peyton Jones, and Max Schäfer. QL: Object-oriented queries on relational data. In *30th European Conference on Object-Oriented Programming (ECOOP 2016)*, volume 56 of *LIPIcs*, pages 2:1–2:25, 2016. doi: 10.4230/LIPIcs.ECOOP.2016.2.
- Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. MIT Press, Cambridge, MA, 1999. ISBN 9780262032704.
- Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977. doi: 10.1145/512950.512973.
- Lancelot Da Costa, Thomas Parr, Noor Sajid, Sebastijan Veselic, Victorita Neacsu, and Karl Friston. Active inference on discrete state-spaces: A synthesis. *Journal of Mathematical Psychology*, 99: 102447, 2020. doi: 10.1016/j.jmp.2020.102447.
- Zinovy Diskin, Yingfei Xiong, Krzysztof Czarnecki, Hartmut Ehrig, Frank Hermann, and Fernando Orejas. From state- to delta-based bidirectional model transformations: The symmetric case. In *Proceedings of the 4th International Conference on Model Transformation (ICMT 2011)*, volume 6707 of *Lecture Notes in Computer Science*, pages 61–76, 2011. doi: 10.1007/978-3-642-21732-6_5.

- Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007. doi: 10.1016/j.scico.2007.01.015.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. Codebert: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547, 2020. doi: 10.18653/v1/2020.findings-emnlp.139.
- Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- Brendan Fong and David I. Spivak. *Seven Sketches in Compositionality: An Invitation to Applied Category Theory*. Cambridge University Press, Cambridge, UK, 2019. URL <https://arxiv.org/abs/1803.05316>.
- J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems*, 29(3):Article 17, 2007. doi: 10.1145/1232420.1232424.
- Karl Friston. The free-energy principle: a unified brain theory? *Nature Reviews Neuroscience*, 11(2):127–138, 2010. doi: 10.1038/nrn2787.
- Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2011)*, pages 317–330, 2011. doi: 10.1145/1926385.1926358.
- Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. GraphCodeBERT: Pre-training code representations with data flow. In *9th International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=jLoC4ez43PZ>.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2023. URL <https://arxiv.org/abs/2301.04104>.
- Conor Heins, Beren Millidge, Daphne Demekas, Brennan Klein, Karl Friston, Iain D. Couzin, and Alexander Tschantz. pymdp: A Python library for active inference in discrete state spaces. *Journal of Open Source Software*, 7(73):4098, 2022. doi: 10.21105/joss.04098.
- Martin Hofmann, Benjamin C. Pierce, and Daniel Wagner. Edit lenses. In *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL 2011)*, pages 495–508, 2011. doi: 10.1145/1926385.1926392.
- Stephen Lack and Pawel Sobocinski. Adhesive categories. In *Proceedings of the 7th International Conference on Foundations of Software Science and Computation Structures (FoSSaCS 2004)*, volume 2987 of *Lecture Notes in Computer Science*, pages 273–288, 2004. doi: 10.1007/978-3-540-24727-2_20.
- Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis

- and transformation. In *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO)*, pages 75–86, 2004. doi: 10.1109/CGO.2004.1281665.
- Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 2–14, 2021. doi: 10.1109/CGO51591.2021.9370308.
- Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. Gated graph sequence neural networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2016. URL <https://arxiv.org/abs/1511.05493>.
- Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: Building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 190–200, 2005. doi: 10.1145/1065010.1065034.
- Nelson Niu and David I. Spivak. Polynomial functors: A mathematical theory of interaction, 2023. URL <https://arxiv.org/abs/2312.00990>.
- Thomas Parr, Giovanni Pezzulo, and Karl J. Friston. *Active Inference: The Free Energy Principle in Mind, Brain, and Behavior*. MIT Press, Cambridge, MA, 2022. ISBN 9780262045353. doi: 10.7551/mitpress/12441.001.0001.
- Roger D. Peng. Reproducible research in computational science. *Science*, 334(6060):1226–1227, 2011. doi: 10.1126/science.1213847.
- Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. In *IEEE International Joint Conference on Neural Networks*, pages 1–8, 2009. doi: 10.1109/IJCNN.2009.5178198.
- Ryan Smith, Karl J. Friston, and Christopher J. Whyte. A step-by-step tutorial on active inference and its application to empirical data. *Journal of Mathematical Psychology*, 107:102632, 2022. doi: 10.1016/j.jmp.2021.102632.
- Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008.
- David I. Spivak. Poly: An abundant categorical setting for mode-dependent dynamics, 2020. URL <https://arxiv.org/abs/2005.01894>.
- Jiayi Wei, Maruth Goyal, Greg Durrett, and Isil Dillig. LambdaNet: Probabilistic type inference using graph neural networks. In *8th International Conference on Learning Representations (ICLR)*, 2020. URL <https://openreview.net/forum?id=Hkx6hANtwH>.
- Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021. doi: 10.1109/TNNLS.2020.2978386.

- Fabian Yamaguchi, Nico Golde, Sascha Arzt, and Konrad Rieck. Modeling and discovering vulnerabilities with code property graphs. In *Proceedings of the 2014 IEEE Symposium on Security and Privacy*, pages 590–604, 2014. doi: 10.1109/SP.2014.43.
- Pengcheng Yin, Graham Neubig, Miltiadis Allamanis, Marc Brockschmidt, and Alexander L. Gaunt. Learning to represent edits. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=BJl6AjC5F7>.